

CANN 商用版
8.0.RC3

集合通信用户指南

文档版本 01
发布日期 2025-02-20



版权所有 © 华为技术有限公司 2025。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

安全声明

产品生命周期政策

华为公司对产品生命周期的规定以“产品生命周期终止政策”为准，该政策的详细内容请参见如下网址：
<https://support.huawei.com/ecolumnsweb/zh/warranty-policy>

漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
<https://www.huawei.com/cn/psirt/vul-response-process>
如企业客户须获取漏洞信息，请参见如下网址：
<https://securitybulletin.huawei.com/enterprise/cn/security-advisory>

华为初始证书权责说明

华为公司对随设备出厂的初始数字证书，发布了“华为设备初始数字证书权责说明”，该说明的详细内容请参见如下网址：
<https://support.huawei.com/enterprise/zh/bulletins-service/ENEWS2000015766>

华为企业业务最终用户许可协议(EULA)

本最终用户许可协议是最终用户（个人、公司或其他任何实体）与华为公司就华为软件的使用所缔结的协议。最终用户对华为软件的使用受本协议约束，该协议的详细内容请参见如下网址：
<https://e.huawei.com/cn/about/eula>

产品资料生命周期策略

华为公司针对随产品版本发布的售后客户资料（产品资料），发布了“产品资料生命周期策略”，该策略的详细内容请参见如下网址：
<https://support.huawei.com/enterprise/zh/bulletins-website/ENEWS2000017760>

目 录

- 1 HCCL 概述..... 1
- 2 术语与相关概念..... 5
- 3 集合通信原语..... 7
- 4 分级通信原理..... 11
- 5 集合通信算法..... 15
- 6 通信功能开发..... 17
 - 6.1 简介..... 17
 - 6.2 通信域管理..... 18
 - 6.3 集合通信..... 20
 - 6.4 点对点通信..... 21
 - 6.5 集群信息配置..... 23
 - 6.5.1 简介..... 23
 - 6.5.2 ranktable 文件配置资源信息..... 23
 - 6.5.3 环境变量配置资源信息..... 30
 - 6.6 样例代码..... 31
 - 6.6.1 安装配置 MPI..... 31
 - 6.6.2 HcclCommInitClusterInfo 初始化方式..... 31
 - 6.6.3 HcclCommInitClusterInfoConfig 初始化方式..... 35
 - 6.6.4 HcclCommInitRootInfo 初始化方式..... 40
 - 6.6.5 HcclCommInitRootInfoConfig 初始化方式..... 43
 - 6.6.6 HcclCommInitAll 初始化..... 47
 - 6.6.7 HcclCreateSubCommConfig 方式创建子通信域..... 51
 - 6.6.8 样例编译运行..... 56
- 7 常用工具与配置..... 58
- 8 接口参考..... 59
 - 8.1 HCCL 接口简介..... 59
 - 8.2 HCCL API (C) 59
 - 8.2.1 接口列表..... 60
 - 8.2.2 通信域管理..... 62
 - 8.2.2.1 HcclCommInitClusterInfo..... 62

8.2.2.2 HcclCommInitClusterInfoConfig.....	63
8.2.2.3 HcclGetRootInfo.....	64
8.2.2.4 HcclCommInitRootInfo.....	65
8.2.2.5 HcclCommInitRootInfoConfig.....	66
8.2.2.6 HcclCommConfigInit.....	68
8.2.2.7 HcclCommInitAll.....	68
8.2.2.8 HcclCommDestroy.....	69
8.2.2.9 HcclGetRankSize.....	70
8.2.2.10 HcclGetRankId.....	71
8.2.2.11 HcclSetConfig.....	72
8.2.2.12 HcclGetConfig.....	72
8.2.2.13 HcclGetCommName.....	73
8.2.2.14 HcclGetCommConfigCapability.....	74
8.2.2.15 HcclCreateSubCommConfig.....	75
8.2.3 集合通信.....	77
8.2.3.1 HcclAllReduce.....	77
8.2.3.2 HcclBroadcast.....	78
8.2.3.3 HcclAllGather.....	79
8.2.3.4 HcclReduceScatter.....	81
8.2.3.5 HcclReduce.....	82
8.2.3.6 HcclAlltoAll.....	83
8.2.3.7 HcclAlltoAllV.....	84
8.2.3.8 HcclBarrier.....	86
8.2.3.9 HcclScatter.....	87
8.2.4 点对点通信.....	88
8.2.4.1 HcclSend.....	88
8.2.4.2 HcclRecv.....	89
8.2.4.3 HcclBatchSendRecv.....	90
8.2.5 异常处理.....	91
8.2.5.1 HcclGetCommAsyncError.....	92
8.2.5.2 HcclGetErrorString.....	92
8.2.6 数据类型.....	93
8.2.6.1 HcclResult.....	93
8.2.6.2 HcclReduceOp.....	94
8.2.6.3 HcclDataType.....	94
8.2.6.4 HcclConfig.....	94
8.2.6.5 HcclConfigValue.....	95
8.2.6.6 HcclRootInfo.....	95
8.2.6.7 HcclComm.....	95
8.2.6.8 HcclSendRecvType.....	95
8.2.6.9 HcclSendRecvItem.....	96
8.2.6.10 HcclCommConfig.....	96

8.2.6.11 HcclCommConfigCapability.....	96
8.2.7 预留接口.....	96
8.3 HCCL API (Python)	97
8.3.1 接口列表.....	97
8.3.2 hccl.manage.api.....	99
8.3.2.1 create_group.....	99
8.3.2.2 destroy_group.....	102
8.3.2.3 get_rank_size.....	102
8.3.2.4 get_local_rank_size.....	103
8.3.2.5 get_rank_id.....	104
8.3.2.6 get_local_rank_id.....	105
8.3.2.7 get_world_rank_from_group_rank.....	106
8.3.2.8 get_group_rank_from_world_rank.....	107
8.3.3 hccl.split.api.....	108
8.3.3.1 set_split_strategy_by_idx.....	108
8.3.3.2 set_split_strategy_by_size.....	110
8.3.4 npu_bridge.hccl.hccl_ops.....	111
8.3.4.1 allreduce.....	111
8.3.4.2 allgather.....	112
8.3.4.3 broadcast.....	114
8.3.4.4 reduce_scatter.....	115
8.3.4.5 reduce.....	117
8.3.4.6 send.....	118
8.3.4.7 receive.....	120
8.3.4.8 alltoallv.....	121
8.3.4.9 alltoallvc.....	123
9 FAQ.....	126
9.1 HCCL 常见问题总结.....	126
9.1.1 环境变量配置错误 (EI0001)	126
9.1.2 执行通信操作超时 (EI0002)	128
9.1.3 集合通信操作参数无效 (EI0003)	129
9.1.4 无效的 RankTable 配置 (EI00004)	130
9.1.5 卡间集合通信参数不一致 (EI0005)	131
9.1.6 建链接超时 (EI0006)	132
9.1.7 CANN 版本不一致, HCCL 返回错误码 EI0008.....	133
9.2 HCCP 常见问题总结.....	134
9.2.1 EJ0001 打屏报错.....	134
9.2.2 EJ0002 打屏报错.....	136
9.2.3 EJ0003 打屏报错.....	136
9.2.4 EJ0004 打屏报错.....	138
9.2.5 参与集合通信的服务器 TLS 信息不一致, HCCL 初始化失败.....	139
9.3 HCCL Test 常见问题总结.....	141

9.3.1 单机带宽低.....	141
9.3.2 多机峰值带宽低.....	142
9.3.3 小数据量时延大.....	143
9.3.4 多机测试大数据量带宽骤降为 0.....	144
9.3.5 测试命令卡数与实际卡数不一致，返回 retcode 11.....	145
9.3.6 未设置 hostname 与 ip 对应关系导致 MPI error 报错.....	146
9.3.7 HCCLComm 初始化失败.....	146
9.3.8 防火墙未关闭.....	147
9.3.9 SSH 连接超时错误.....	148
9.3.10 多台机器 MPI 版本不一致问题.....	148
9.3.11 Device 网络不通报错 retcode 4.....	148
9.3.12 host ip 之间两两不通报 MPI 错误.....	149
9.3.13 网口 down 导致报错 ret(-67).....	150
9.3.14 Device IP 冲突导致 get socket 超时.....	150
9.3.15 未指定网卡名报错 retcode 9.....	151
9.3.16 hostfile 与测试命令不匹配报错 retcode 11.....	151
9.3.17 大规模集群场景报建链失败错误.....	152
9.3.18 大规模集群场景 HCCP 报 errno 24.....	153

1 HCCL 概述

集合通信库HCCL（Huawei Collective Communication Library）是基于昇腾AI处理器的高性能集合通信库，提供单机多卡以及多机多卡间的数据并行、模型并行集合通信方案。

HCCL支持AllReduce、Broadcast、AllGather、ReduceScatter、AlltoAll等通信原语，Ring、Mesh、Halving-Doubling（HD）等通信算法，基于HCCS、RoCE和PCIe高速链路实现集合通信。

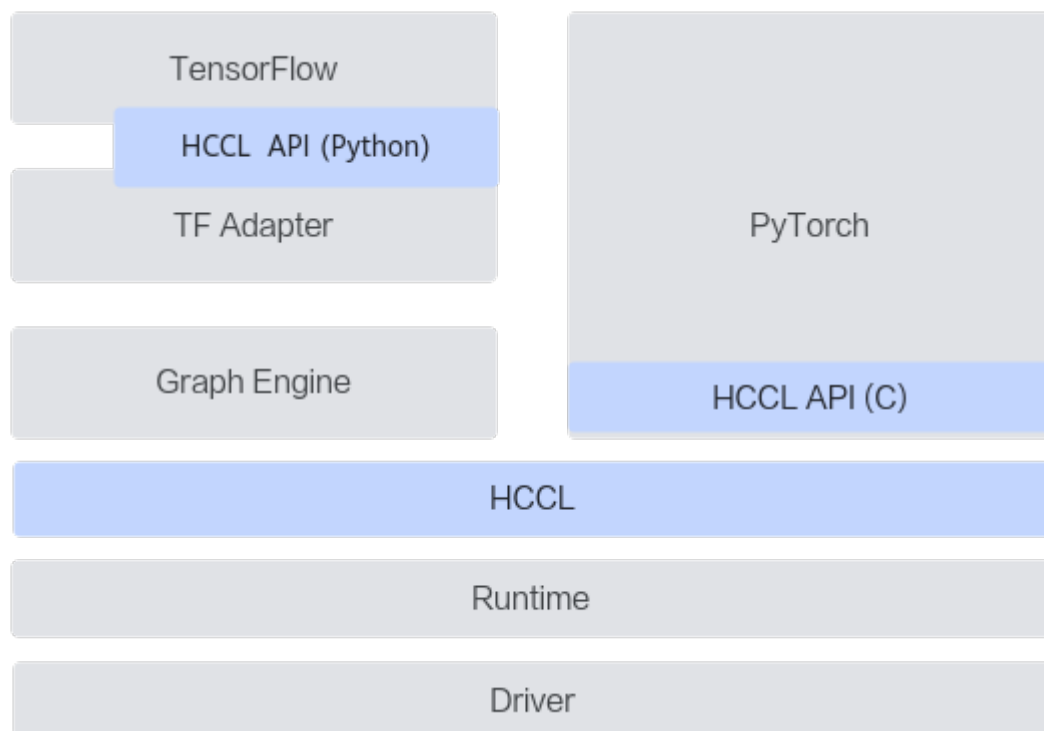
支持的产品型号

Atlas 训练系列产品

Atlas A2 训练系列产品

Atlas 300I Duo 推理卡

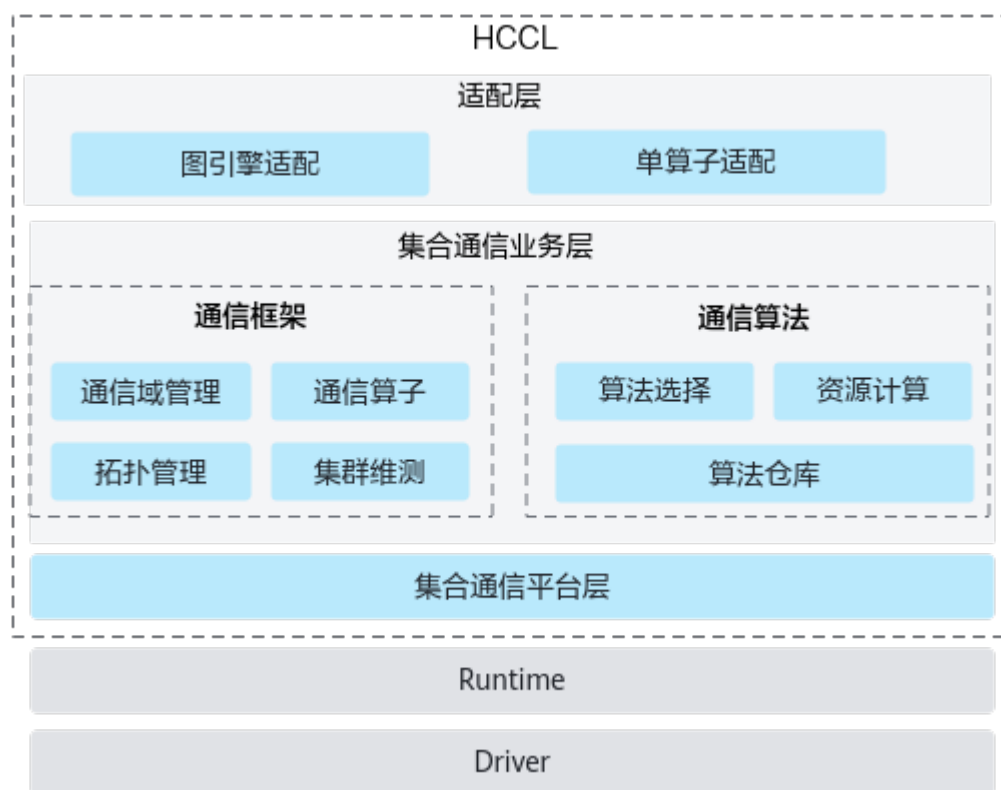
HCCL 在系统中的位置



HCCL提供了Python与C++两种语言的接口，其中Python语言的接口用于实现图模式下的框架适配，例如TensorFlow网络基于HCCL的Python API实现分布式优化；C语言接口用于实现单算子模式下的框架适配，例如HCCL单算子API嵌入到PyTorch后端代码中，PyTorch用户直接使用PyTorch原生集合通信API，即可实现分布式能力。

HCCL 软件架构

图 1-1 集合通信库软件架构图



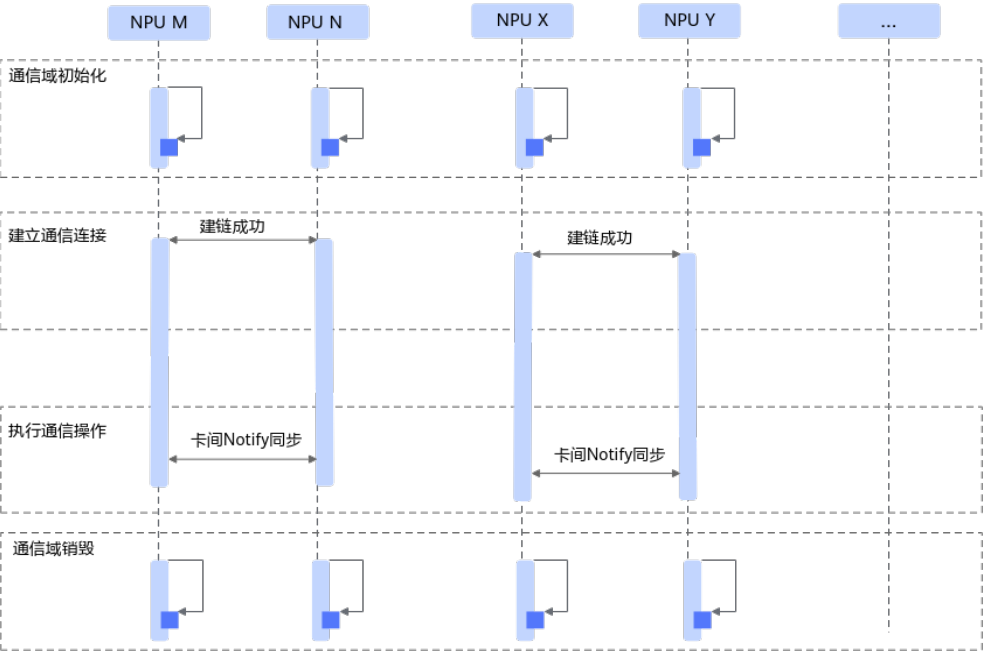
集合通信库软件架构分为三层：

- 适配层，图引擎与单算子适配，进行通信切分寻优等操作，提供通信域管理及通信算子接口。
- 集合通信业务层，包含通信框架与通信算法两个模块：
 - 通信框架：负责通信域管理，通信算子的业务串联，协同通信算法模块完成算法选择，协同通信平台模块完成资源申请并实现集合通信任务的下发。
 - 通信算法：作为集合通信算法的承载模块，提供特定集合通信操作的资源计算，并根据通信域信息完成通信任务编排。
- 集合通信平台层，提供NPU之上与集合通信关联的资源抽象，并提供集合通信的相关维护、测试能力。

集合通信流程

分布式场景中，HCCL提供了服务器间高性能集合通信功能，其通信流程如[图1-2](#)所示。

图 1-2 分布式场景集合通信流程



服务器间通信过程分为四个阶段：

1. 通信域初始化：获取必要的集合通信配置参数并初始化通信域。
通信初始化阶段不涉及NPU设备之间的交互。
2. 建立通信连接：建立socket连接并交换通信两端的通信参数和内存信息。
建立通信连接阶段，HCCL根据用户提供的集群信息并结合网络拓扑与其他NPU设备建链，交换用于通信的参数信息。如果在建链超时时间内未得到其他NPU设备的及时响应，会上报建链超时错误并退出业务进程。
3. 执行通信操作：通过“等待-通知”机制同步设备执行状态，传递内存数据。
通信操作执行阶段，HCCL会将通信算法编排、内存访问等任务通过Runtime下发给昇腾设备的任务调度器，设备根据编排信息调度并执行任务。
4. 通信域销毁：销毁通信域，释放通信资源。

2 术语与相关概念

为了您有更好的阅读体验，使用本文档前请先了解如下术语、缩略语与基本概念。

表 2-1 术语与相关概念

名称	说明
NPU	Neural Network Processing Unit，神经网络处理单元。 采用“数据驱动并行计算”的架构，特别擅长处理视频、图像类的海量多媒体业务数据，专门用于处理人工智能应用中的大量计算任务。
HCCL	Huawei Collective Communication Library，华为集合通信库。 提供单机多卡以及多机多卡间的数据并行、模型并行集合通信方案。
HCCS	Huawei Cache Coherence System，华为缓存一致性系统。 HCCS用于CPU/NPU之间的高速互联。
HCCP	Huawei Collective Communication adaptive Protocol，集合通信适配协议。 提供跨NPU设备通信能力，向上屏蔽具体通讯协议差异。
TOPO	拓扑、拓扑结构。 一个局域网内或者多个局域网之间的设备连接所构成的网络配置或者布置。
PCIe	Peripheral Component Interconnect Express，一种串行外设扩展总线标准，通常用于计算机系统中外设扩展使用。
PCIe-SW	PCIe Switch，符合PCIe总线扩展的交换设备。
SDMA	System Direct Memory Access，系统直接内存访问技术，简称DMA，允许外围设备直接访问系统内存，而不需要CPU的干预。
RDMA	Remote Direct Memory Access，远程直接内存访问技术，它将数据直接从一台机器的内存传输到另一台机器，无需双方操作系统的介入，一般指可以跨过网络的内存访问方式。
RoCE	RDMA over Converged Ethernet，承载在融合以太网上的RDMA技术，即跨越以太网的RDMA通信方式。

名称	说明
AI节点	昇腾AI节点，又称昇腾AI Server，通常是8卡或16卡的昇腾NPU设备组成的服务器形态的统称。
AI集群	多个AI节点通过交换机（ Switch ）互联后用于分布式训练或推理的系统。
通信域	通信域是集合通信执行的上下文，管理对应的通信实体（ 例如一个NPU就是一个通信实体 ）和通信所需的资源。
Rank	通信域中的每个通信实体称为一个Rank。

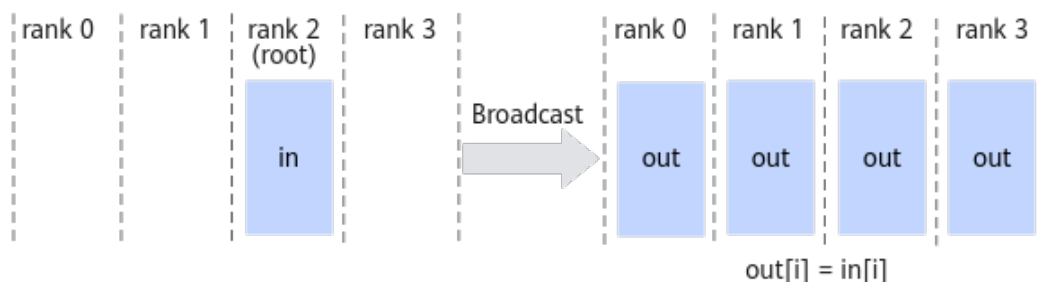
3 集合通信原语

集合通信是一个进程组的所有进程都参与的全局通信操作，其最为基础的操作有发送、接收、复制、节点间进程同步等，这些基本的操作经过组合构成了一组通信模板，也称为通信原语，这些通信原语通过相应的集合通信算子实现。

下面展开介绍HCCL提供的通信原语。

Broadcast

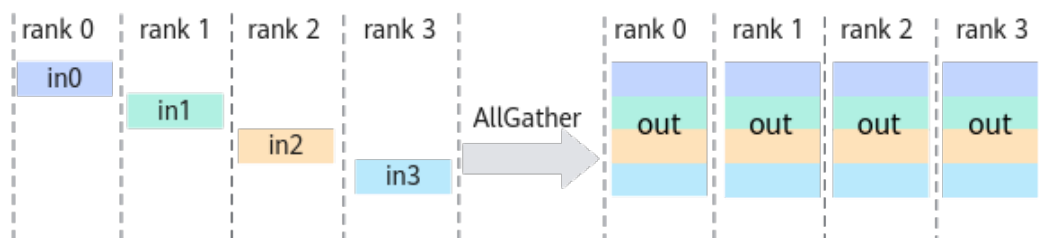
Broadcast操作是将通信域内root节点的数据广播到其他rank。



注意：全局只能有一个root节点。

AllGather

AllGather操作是将group内所有节点的输入按照rank id重新排序，然后拼接起来，再将结果发送到所有节点的输出。

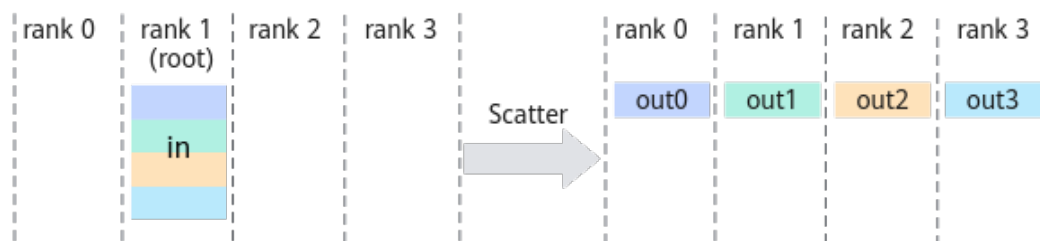


说明

针对AllGather操作，每个节点都接收按照rank id重新排序后的数据集，即每个节点的全AllGather输出都是一样的。

Scatter

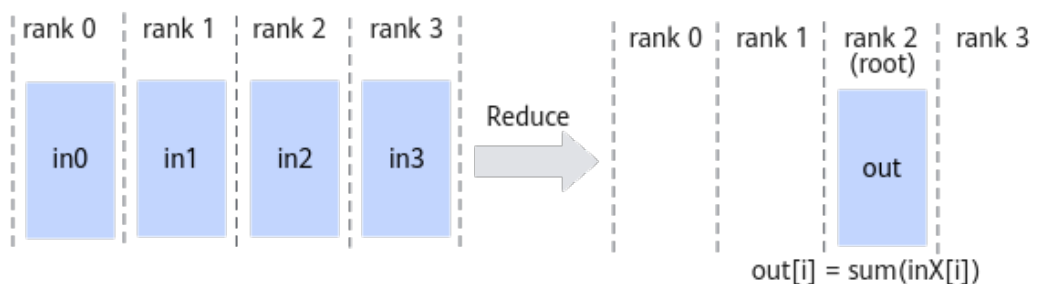
Scatter操作是将通信域内root节点的数据均分并散布至其他rank。



注意：全局只能有一个root节点。

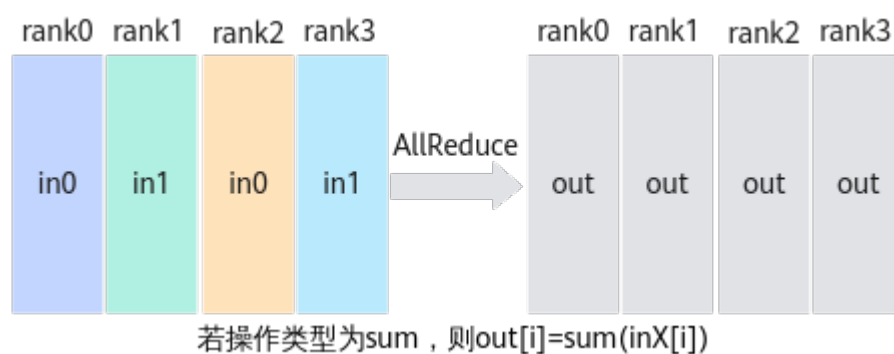
Reduce

Reduce操作是将所有rank的input相加（或prod/max/min）后，再把结果发送到root节点的输出buffer。



AllReduce

AllReduce操作是将通信域内所有节点的输入数据进行reduce操作后，再把结果发送到所有节点的输出buffer，其中reduce操作类型支持sum、prod、max、min。

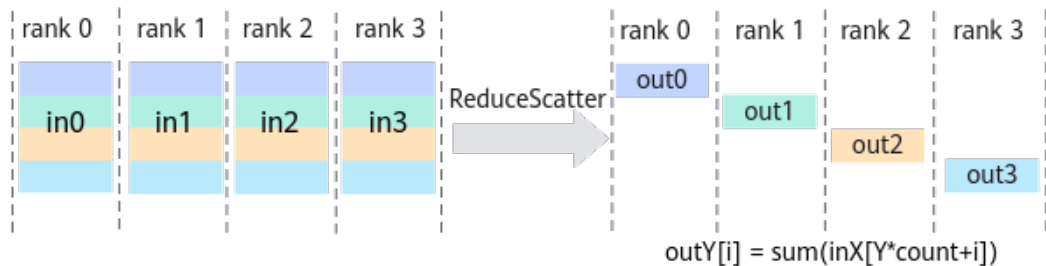


注意：每个rank只能有一个输入。

ReduceScatter

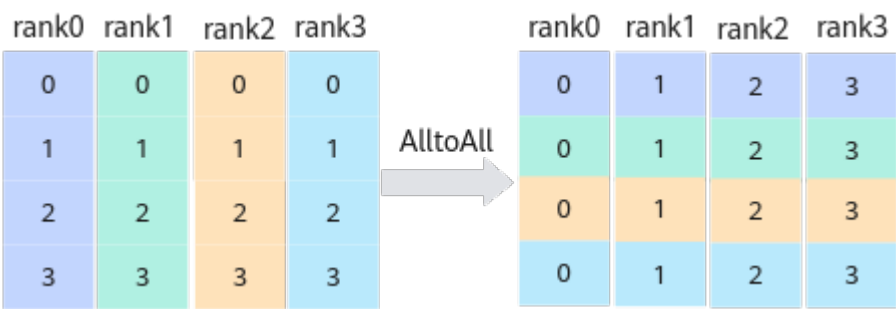
ReduceScatter操作是将所有rank的输入相加（或其他操作）后，再把结果按照rank编号均匀分散到各个rank的输出buffer，每个进程拿到其他进程 $1/ranksz$ 份的数据进行归约操作。

如下图所示，有rank0、rank1、rank2、rank3四个rank，每个rank的输入数据切分成4份，每个进程分别取每个rank的1/4数据进行sum操作（或其他操作），将结果发送到输出buffer。



AlltoAll

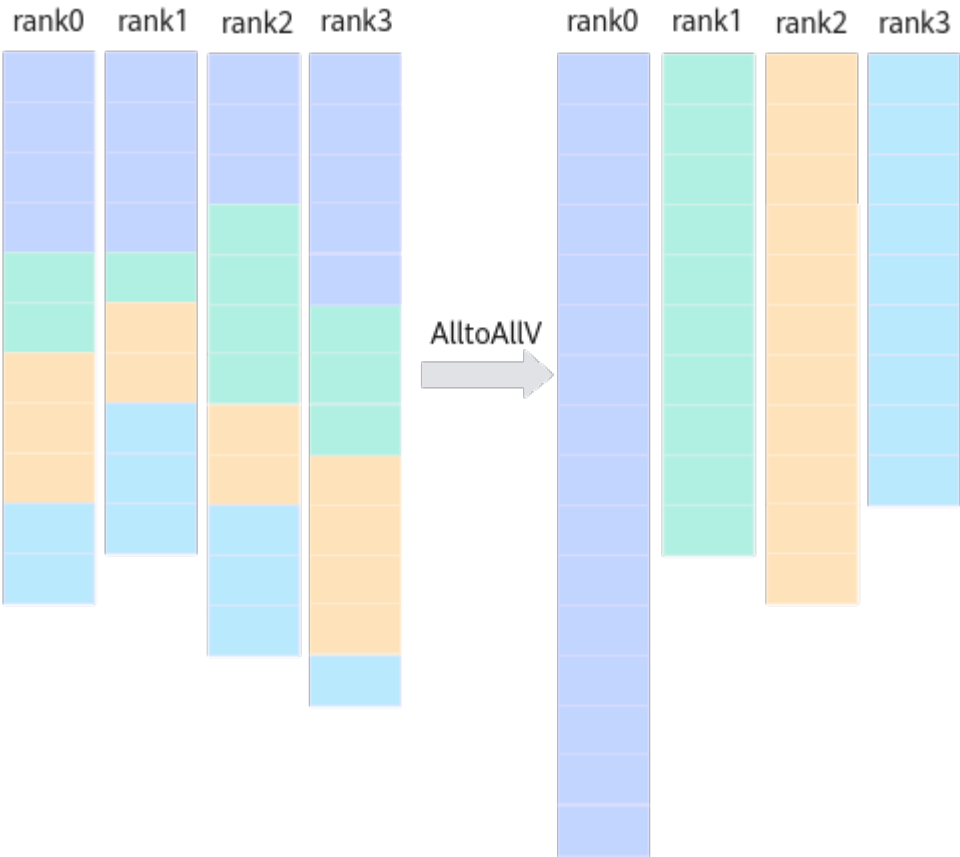
AlltoAll操作是向通信域内所有rank发送相同数据量的数据，并从所有rank接收相同数据量的数据。



AlltoAll操作将输入数据在特定的维度切分成特定的块数，并按顺序发送给其他rank，同时从其他rank接收输入，按顺序在特定的维度拼接数据。

AlltoAllV

AlltoAllV操作是向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。



4 分级通信原理

HCCL通常按Server内和Server间分为两级拓扑，分级执行集合通信，各集合通信算子的具体分级通信过程如下表所示：

表 4-1 集合通信算子分级通信过程

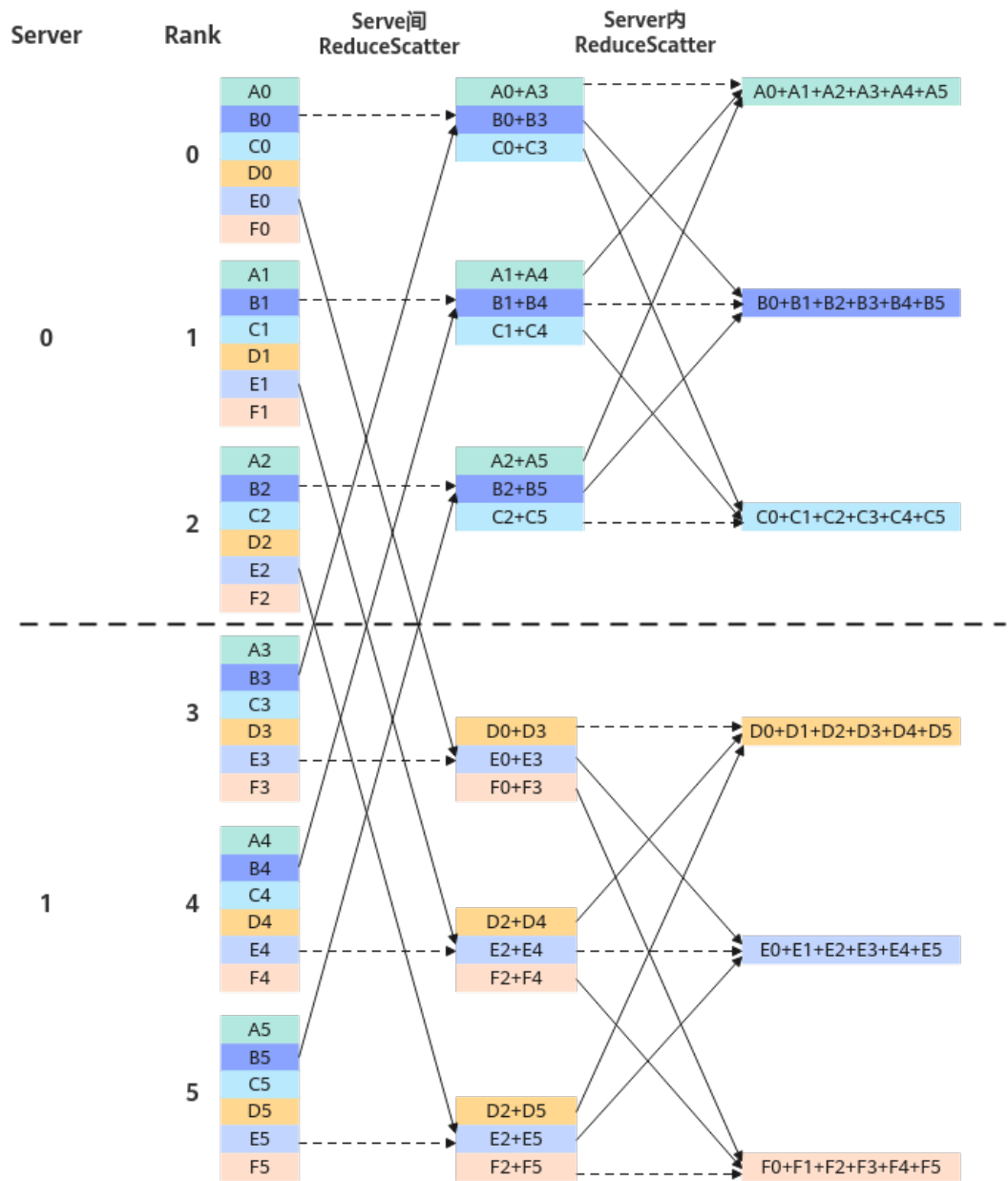
集合通信算子	阶段一	阶段二	阶段三
ReduceScatter	Server间 ReduceScatter	Server内 ReduceScatter	/
AllGather	Server内Allgather	Server间AllGather	/
AllReduce	Server内 ReduceScatter	Server间AllReduce	Server内AllGather
Scatter	Server间Scatter	Server内Scatter	/
Broadcast	Server内Scatter	Server间Broadcast	Server内AllGather
Reduce	Server内 ReduceScatter	Server间Reduce	Server内Gather
AlltoAll	Server内AlltoAll	Server间AlltoAll	/
AlltoAllV	Server内AlltoAllV	Server间AlltoAllV	/

下面以典型的通信算子ReduceScatter、AllGather、AllReduce为例介绍分级通信的流程。

ReduceScatter

如图4-1所示，ReduceScatter算子严格要求第i个rank最终得到第i份规约结果，为了保证Server间通信数据块的连续性，首先在Server间执行ReduceScatter操作，再在Server内执行ReduceScatter操作。

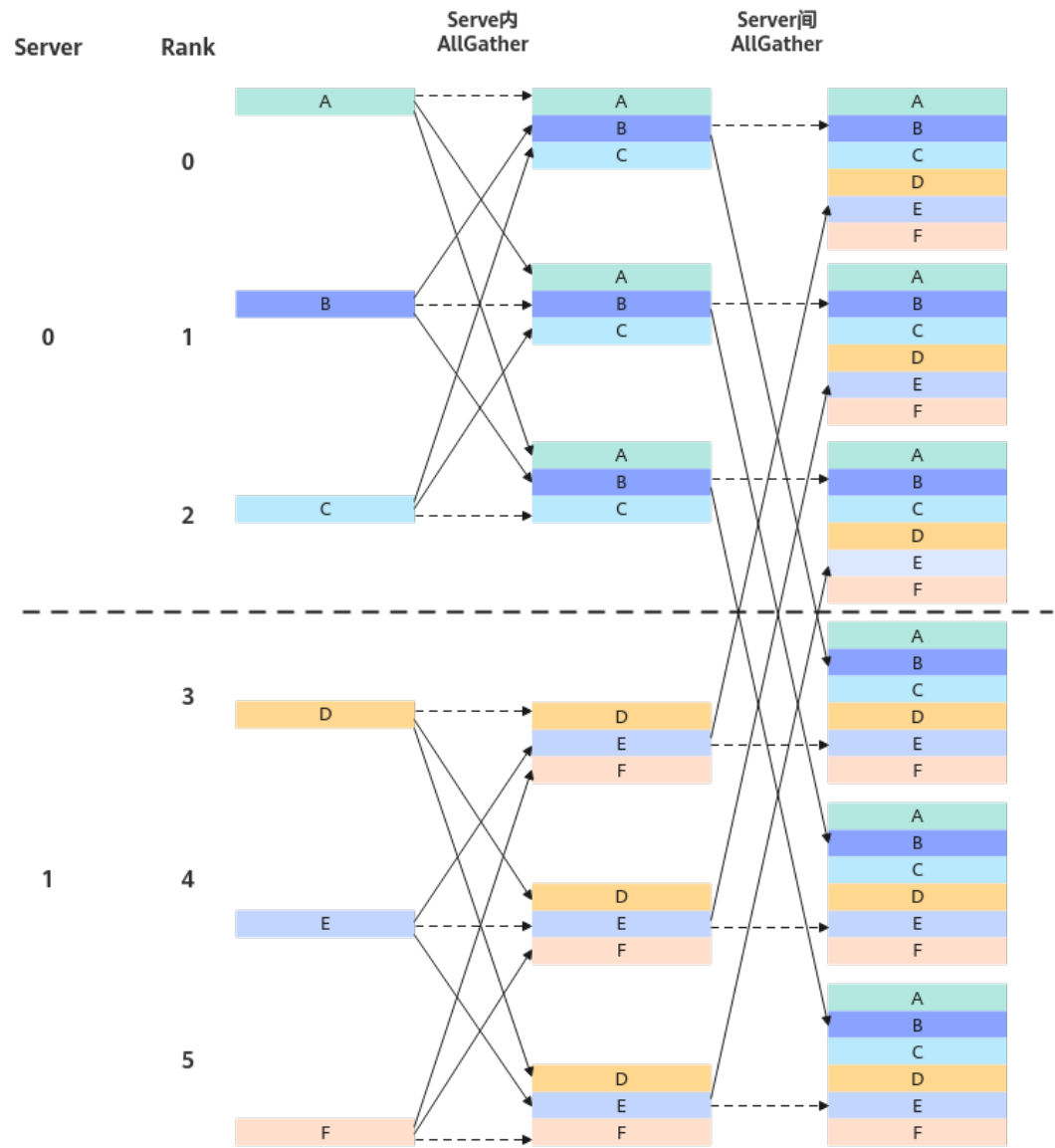
图 4-1 ReduceScatter 算子分级通信流程



AllGather

如图4-2所示，AllGather算子严格要求第i个rank的输入数据出现在结果的第i个位置上，为了保证Server间通信数据块的连续性，首先在Server内执行AllGather操作，然后在Server间执行AllGather操作。

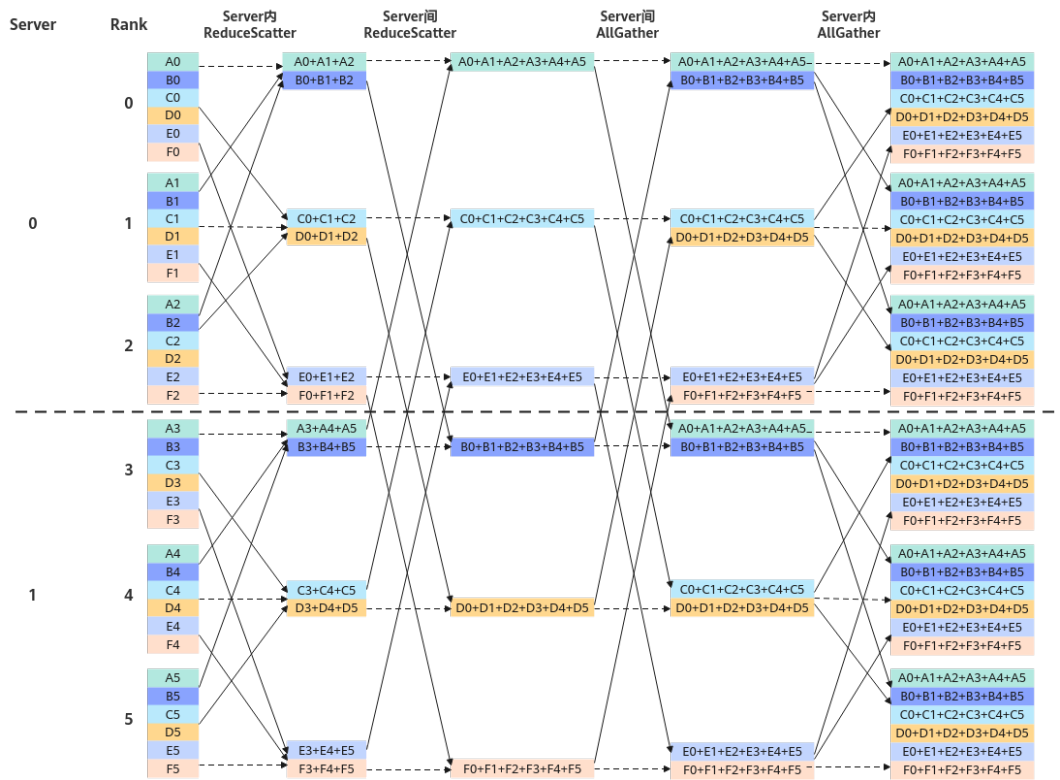
图 4-2 AllGather 算子分级通信流程



AllReduce

如图4-3所示，AllReduce算子的输出是完整的规约结果，因此虽然拆解为了ReduceScatter和AllGather两个阶段，但不需要严格遵循ReduceScatter和AllGather的语义，可以将较大数据量的通信过程放在带宽更高的Server内，即先在Server内执行ReduceScatter操作，然后在Server间执行AllReduce操作，最后在Server内执行AllGather操作。

图 4-3 AllReduce 算子分级通信流程



5 集合通信算法

针对同一个通信算子，随着网络拓扑、通信数据量、硬件资源等的不同，往往会采用不同的通信算法，从而最大化集群通信性能。HCCL提供了Mesh、Ring、Recursive Halving-Doubling (RHD)、NHR (Nonuniform Hierarchical Ring)、NB (Nonuniform Bruck)、Pipeline、Pairwise等拓扑算法用于Server内和Server间的集合通信。

Server 内通信算法

HCCL通信域Server内支持Mesh、Ring、Double-Ring和Star算法，具体使用的算法根据硬件拓扑自动选择，用户无须配置也不支持配置。

Server 间通信算法

HCCL通信域Server间支持如下算法的自适应选择，自适应算法会根据产品形态、数据量和Server个数进行选择，用户默认无需配置。

- Ring算法：基于环结构的通信算法，通信步数多（线性复杂度），时延相对较高，但通信关系简单，受网络拥塞影响较小。适合通信域内Server个数较少、通信数据量较小、网络存在明显拥塞、且pipeline算法不适用的场景。
- RHD (Recursive Halving-Doubling) 算法：递归二分和倍增算法，通信步数少（对数复杂度），时延相对较低，但在非2次幂节点规模下会引入额外的通信量。适合通信域内Server个数是2的整数次幂且pipeline算法不适用，或Server个数不是2的整数次幂但通信数据量较小的场景。
- NHR (Nonuniform Hierarchical Ring) 算法：非均衡的层次环算法，通信步数少（对数复杂度），时延相对较低。适合通信域内Server个数较多且pipeline算法不适用的场景。
- NB (Nonuniform Bruck) 算法：非均匀的数据块通信算法，通信步数少（对数复杂度），时延相对较低。适合通信域内Server个数较多且pipeline算法不适用的场景。
- Pipeline算法：流水线并行算法，可并发使用Server内与Server间的链路，适合通信数据量较大且通信域内每机包含多卡的场景。
- Pairwise算法：逐对通信算法，仅用于AllToAll、AlltoAllV与AlltoAllVC算子，通信步数较多（线性复杂度），时延相对较高，但可以避免网络中出现一打多现象（指一个rank通过同一个端口给多个rank发送数据），适合通信数据量较大、需要规避网络一打多的场景。

说明

- cann-hccl仓库当前开放的算法有Mesh、Ring、RHD、PairWise、Star，开发者可访问[Gitee-cann-hccl](#)详细了解对应实现。
- 开发者若想指定集合通信Server间跨机通信算法，可通过环境变量HCCL_ALGO进行设置。关于HCCL_ALGO如何设置及每种算法的算子支持度，可参见《[环境变量参考](#)》。需要注意，若通过环境变量HCCL_ALGO指定了Server间跨机通信算法，Server间的通信算法自适应选择功能不再生效，以用户指定的算法为准。

6 通信功能开发

[简介](#)

[通信域管理](#)

[集合通信](#)

[点对点通信](#)

[集群信息配置](#)

[样例代码](#)

6.1 简介

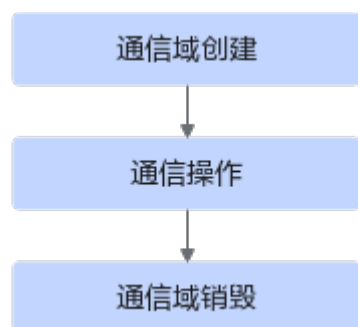
HCCL提供了C与Python两种语言的开发接口，用于实现分布式能力。

- C语言接口用于实现单算子模式下的框架适配，实现分布式能力。
针对PyTorch框架网络，HCCL单算子API已嵌入到Ascend Extension for PyTorch后端代码中，PyTorch用户直接使用PyTorch原生集合通信API，即可实现分布式能力。
- Python语言接口用于实现图模式下的框架适配，当前仅用于实现TensorFlow网络在昇腾AI处理器执行分布式优化。

本章节针对如何调用HCCL的C语言接口进行集合通信功能的开发进行介绍。

开发者调用HCCL C语言接口实现集合通信功能的主要开发流程如下所示。

图 6-1 集合通信操作流程



1. 首先进行集群信息配置，创建通信域句柄，并初始化HCCL通信域。
2. 实现通信操作，HCCL通信操作包含两大类：点对点通信与集合通信。
 - 点对点通信，指在多NPU环境下两个NPU之间直接传输数据的过程，常用于pipeline并行场景下对激活值的数据收发。HCCL提供了不同粒度的点对点通信，包括单rank到单rank的单收单发接口，以及多个rank之间的批量收发接口。
 - 集合通信，指多个NPU共同参与进行数据传输操作，例如AllReduce，AllGather，Broadcast等，常用于大规模集群中不同NPU之间的梯度同步和参数更新等操作。集合通信操作可让所有计算节点并行、高效、有序执行数据交换，提升数据传输效率。
3. 集合通信操作完成后，需要释放相关资源，销毁通信域。

6.2 通信域管理

通信域是集合通信算子执行的上下文，管理对应的通信对象（例如一个NPU就是一个通信对象）和通信所需的资源。通信域中的每个通信对象称为一个rank，每个rank都会分配一个介于0~n-1（n为npu的数量）的唯一标识。

通信域创建根据用户场景的不同主要有以下几种方式：

- 多机集合通信场景，如果有完整的描述集群信息的ranktable文件，可通过HcclCommInitClusterInfo接口创建通信域，或者通过HcclCommInitClusterInfoConfig接口创建具有特定配置的通信域。
- 多机集合通信场景，如果无完整的ranktable，可通过HcclGetRootInfo接口与HcclCommInitRootInfo/HcclCommInitRootInfoConfig接口配合使用，基于root节点广播方式创建通信域。
- 单机集合通信场景，可通过HcclCommInitAll接口在单机内批量创建通信域。
- 基于已有的通信域，可通过HcclCreateSubCommConfig接口切分具有特定配置的子通信域。

须知

- 针对同一个rank，通信算子需要one by one保序调用，不支持并发下发。
- 针对整个通信域，通信算子的调用在不同rank间要保证同一下发顺序。
- 同一个通信域内不支持图模式通信和单算子通信混合执行。

基于 ranktable 创建通信域

多机集合通信、基于集群信息配置ranktable文件创建通信域的场景，每张卡需要使用一个单独的进程参考如下流程创建通信域：

1. 构造ranktable文件（ranktable文件的配置可参见[6.5.2 ranktable文件配置资源信息](#)），如果已存在ranktable文件，也可以通过环境变量RANK_TABLE_FILE获取。
2. 每张卡使用HcclCommInitClusterInfo接口创建通信域，或者使用HcclCommInitClusterInfoConfig接口创建具有特定配置的通信域。

一个简单的代码示例片段如下：

```
// 获取ranktable路径
char* rankTableFile = getenv("RANK_TABLE_FILE");
```

```
// 定义通信域句柄
HcclComm hcclComm;
// 初始化HCCL通信域
HcclCommInitClusterInfo(rankTableFile, devId, &hcclComm);

/* 集合通信操作 */

// 销毁HCCL通信域
HcclCommDestroy(hcclComm);
```

完整的代码示例可参见[6.6.2 HcclCommInitClusterInfo初始化方式](#)。

基于 root 节点广播方式创建通信域

多机集合通信场景，若无完整的集群信息配置ranktable文件，HCCL提供了基于root节点广播的方式创建通信域，详细流程如下：

1. 配置环境变量HCCL_IF_IP，指定root通信网卡的IP地址。
需要配置为root节点Host网卡的IP地址，可以是IPv4或IPv6格式，仅支持配置一个IP地址，配置示例如下：
export HCCL_IF_IP=10.10.10.1
2. 在root节点调用[HcclGetRootInfo](#)接口，生成root节点rank标识信息“rootInfo”，包括device ip、device id等信息。
3. 将root节点的rank信息广播至集群中的所有rank。
4. 在所有节点调用[HcclCommInitRootInfo](#)或者[HcclCommInitRootInfoConfig](#)接口（创建具有特定配置的通信域），基于接收到的“rootInfo”，以及本rank的rank id等信息，进行HCCL初始化。

需要注意：每个卡需要使用一个单独的进程进行以上操作。

使用HcclCommInitRootInfo接口创建通信域的简单代码示例片段如下：

```
// 获取当前进程在所属进程组的编号
MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
int devId = procRank;
// 在root节点获取其rank信息
HcclRootInfo rootInfo;
int32_t rootRank = 0;
if(devId == rootRank) {
    HcclGetRootInfo(&rootInfo);
}
// 将root_info广播到通信域内的其他rank
MPI_Bcast(&rootInfo, HCCL_ROOT_INFO_BYTES, MPI_CHAR, rootRank, MPI_COMM_WORLD);
MPI_Barrier(MPI_COMM_WORLD);
// 定义通信域句柄
HcclComm hcclComm;
// 初始化HCCL通信域
HcclCommInitRootInfo(devCount, &rootInfo, devId, &hcclComm);

/* 集合通信操作 */

// 销毁HCCL通信域
HcclCommDestroy(hcclComm);
```

完整的代码示例可参见[6.6.4 HcclCommInitRootInfo初始化方式](#)。

单机内批量创建通信域

单机通信场景中，开发者通过一个进程统一创建多张卡的通信域，其中一张卡对应一个线程，创建流程如下：

1. 构造通信域中的Device列表，例如：{0, 1, 2, 3, 4, 5, 6, 7}，其中列表中的Device ID是逻辑ID。

2. 在进程中调用HcclCommInitAll接口创建通信域。

```
uint32_t ndev = 8;
// 构造Device的逻辑ID列表
int32_t devices[8] = {0, 1, 2, 3, 4, 5, 6, 7};
// 定义通信域句柄
HcclComm comms[ndev];
// 初始化HCCL通信域
HcclCommInitAll(ndev, devices, comms);

// 启动线程执行集合通信操作
std::vector<std::unique_ptr<std::thread>> threads(ndev);
struct ThreadContext args[ndev];
for (uint32_t i = 0; i < ndev; i++) {
    args[i].device = i;
    args[i].comm = comms[i];
    /* 集合通信操作 */
}

// 销毁HCCL通信域
for (uint32_t i = 0; i < ndev; i++) {
    HcclCommDestroy(comms[i]);
}
```

需要注意，多线程调用集合通信操作API时（例如HcclAllReduce时），需要确保不同线程中调用集合通信操作API的前后时间差不超过环境变量HCCL_CONNECT_TIMEOUT的时间，避免建链超时。

完整的代码示例可参见[6.6.6 HcclCommInitAll初始化](#)。

基于已有通信域切分子通信域

HCCL提供了[HcclCreateSubCommConfig](#)接口，实现基于已有通信域切分具有特性配置子通信域的功能。该子通信域创建方式无需进行socket建链与rank信息交换，可应用于业务故障下的快速通信域创建。

使用HcclCreateSubCommConfig接口切分子通信域的代码示例可参见[6.6.7 HcclCreateSubCommConfig方式创建子通信域](#)。

须知

该接口仅支持从全局通信域切分子通信域，不支持通信域的嵌套切分。

销毁通信域

集合通信操作完成后，需要调用运行时管理接口释放通信所用的内存、Stream、Device资源，并调用[HcclCommDestroy](#)接口销毁指定的通信域，完整的示例代码可参见[6.6 样例代码](#)。

6.3 集合通信

集合通信是指多个NPU共同参与进行数据传输，从而形成一次集体操作的通信模式，常用于大规模集群中不同NPU之间的梯度同步和参数更新等场景。

HCCL支持AllReduce、Broadcast、AllGather、Scatter、ReduceScatter、Reduce、AlltoAll和AlltoAllV等通信原语，关于这些通信原语的详细含义可参见[3 集合通信原语](#)。针对这些通信原语，HCCL提供了对应的API供开发者调用，用于快速实现集合通信能力。

例如，HCCL针对集合通信操作AllReduce提供了[HcclAllReduce](#)接口，原型定义如下：

```
HcclResult HcclAllReduce(void *sendBuf, void *recvBuf, uint64_t count, HcclDataType dataType,
HcclReduceOp op, HcclComm comm, aclrtStream stream)
```

HcclAllReduce用于将通信域内所有节点的输入进行Reduce操作，再把结果发送到所有节点的输出，其中op参数用于指定Reduce的操作类型，当前版本支持的操作类型有sum、prod、max、min。HcclAllReduce允许每个节点只有一个输入。

如下代码片段所示，将通信域内所有输入内存中的数据，按照float32的数据格式执行加法操作（示例中每个rank中只有一个数据参与），然后把相加结果发送到所有节点的输出内存。

```
void* host_buf = nullptr;
void* send_buff = nullptr;
void* recv_buff = nullptr;
uint64_t count = 1;
int malloc_kSize = count * sizeof(float);
aclrtStream stream;
aclrtCreateStream(&stream);

//申请集合通信操作的内存
aclrtMalloc((void**)&sendbuff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST);
aclrtMalloc((void**)&recvbuff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST);

//初始化输入内存
aclrtMallocHost((void**)&host_buf, malloc_kSize);
aclrtMemcpy((void*)send_buff, malloc_kSize, (void*)host_buf, malloc_kSize,
ACL_MEMCPY_HOST_TO_DEVICE);

//执行集合通信操作
HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32, HCCL_REDUCE_SUM,
hcclComm, stream);
```

HcclAllReduce接口调用的完整示例可参见[6.6 样例代码](#)章节不同通信域初始化方式中的“HcclAllReduce操作代码样例”，HCCL提供的更多集合通信接口可参见[8.2.3 集合通信](#)。

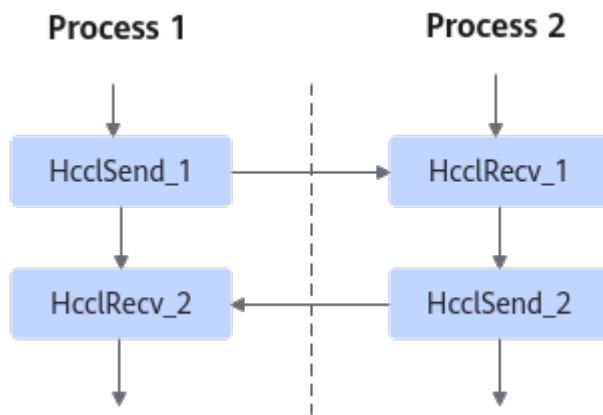
6.4 点对点通信

点对点通信是指多NPU环境下两个NPU之间直接传输数据的通信模式，常用于pipeline并行场景下对激活值的数据收发。

HCCL提供了不同粒度的点对点通信，包括单rank到单rank的单收单发接口，以及多个rank之间的批量收发接口。

HcclSend/HcclRecv

HcclSend/HcclRecv用于单收单发场景，需严格保序下发并配对使用，收发两端需完成同步后才能进行数据收发，数据收发完成后才能执行后续算子任务。



一个简单的代码示例片段如下：

```
if(rankId == 0){
    uint32_t destRank = 1;
    uint32_t srcRank = 1;
    HcclSend(sendBuf, count, dataType, destRank, hcclComm, stream);
    HcclRecv(recvBuf, count, dataType, srcRank, hcclComm, stream);
}
if(rankId == 1){
    uint32_t srcRank = 0;
    uint32_t destRank = 0;
    HcclRecv(recvBuf, count, dataType, srcRank, hcclComm, stream);
    HcclSend(sendBuf, count, dataType, destRank, hcclComm, stream);
}
```

完整的代码示例可参见[6.6 样例代码](#)章节不同通信域初始化方式中的“HcclSend/HcclRecv操作代码样例”。

HcclBatchSendRecv

HcclBatchSendRecv可用于通信域内多个rank之间的数据收发，该接口有两个特征：

- 接口内部会对批量数据收发顺序进行重排，所以不严格要求单次接口调用中批量收发的任务顺序，但需要确保一次接口调用中的数据发送与数据接收操作个数完全匹配。
- 收发过程独立调度执行，收发不相互阻塞，从而实现双工链路并发。

该接口使用时需要注意：单次接口调用下，两个rank之间单向数据流仅支持传递一块内存数据，避免收发过程中混淆多块内存数据的收发地址。

一个简单的代码示例片段如下：

```
HcclSendRecvItem sendRecvInfo[itemNum];
HcclSendRecvType currType;
for (size_t i = 0; i < op_type.size(); ++i) {
    if (op_type[i] == "isend") {
        currType = HcclSendRecvType::HCCL_SEND;
    } else if (op_type[i] == "irecv") {
        currType = HcclSendRecvType::HCCL_RECV;
    }
    sendRecvInfo[i] = HcclSendRecvItem{currType,
        tensor_ptr_list[i],
        count_list[i],
        type_list[i],
        remote_rank_list[i]
    };
}
HcclBatchSendRecv(sendRecvInfo, itemNum, hcclComm, stream);
```

6.5 集群信息配置

6.5.1 简介

HCCL提供了两种集群信息配置方式用于通信域初始化：ranktable配置文件的方式和环境变量的方式，开发者可根据使用场景选择其中一种方式，但两种方式不可以混用。

ranktable配置文件方式有以下使用场景：

- 通过C语言接口**HcclCommInitClusterInfo**或者**HcclCommInitClusterInfoConfig**初始化通信域时。
- TensorFlow分布式网络通信域初始化时。

通过环境变量的方式配置资源信息仅适用于TensorFlow框架网络的通信域初始化，仅支持如下型号的产品：

- Atlas 训练系列产品
- Atlas A2 训练系列产品

6.5.2 ranktable 文件配置资源信息

开发者可以通过ranktable文件配置参与集合通信的NPU资源信息，ranktable文件为json格式，开发者可以在此文件中配置全量NPU资源信息，后续进程启动时可使用其中指定的几个NPU资源。

配置文件说明（Atlas A2 训练系列产品）

针对Atlas A2 训练系列产品，ranktable文件配置说明如下：

```
{
  "status": "completed", // ranktable可用标识, completed为可用
  "version": "1.0",      // ranktable模板版本信息, 当前必须为"1.0"
  "server_count": "1",  // 参与训练的AI Server数目, 此例中, 只有一个AI Server
  "server_list":
  [
    {
      "device": [ // AI Server中的Device列表
        {
          "device_id": "0", // 处理器物理ID
          "device_ip": "192.168.1.8", // 处理器真实网卡IP
          "rank_id": "0" // rank的标识, rankID从0开始
        },
        {
          "device_id": "1",
          "device_ip": "192.168.1.9",
          "rank_id": "1"
        }
      ],
      "server_id": "node_0" // AI Server标识, String类型, 请确保全局唯一
    }
  ]
}
```

表 6-1 ranktable 文件说明

配置项	配置说明	可选/必选
status	ranktable可用标识。 - completed：表示ranktable可用，可执行训练。 - initializing：表示ranktable不可用，不可执行训练。	必选
version	ranktable模板版本信息。配置为1.0。	必选
server_count	本次参与训练的AI Server个数。	必选
server_list	本次参与训练的AI Server列表。	必选
server_id	AI Server标识，字符串类型，长度小于64，请确保全局唯一。 配置示例：node_0	必选
device_id	昇腾AI处理器的物理ID，即Device在AI Server上的序列号。 可通过执行“ls /dev/davinci*”命令获取昇腾AI处理器的物理ID。 例如：显示/dev/davinci0，表示昇腾AI处理器的物理ID为0。 取值范围：[0，实际Device数量-1]。 须知 “device_id”配置项的优先级高于环境变量“ASCEND_DEVICE_ID”。	必选
device_ip	昇腾AI处理器集成网卡IP，全局唯一，要求为常规IPv4或IPv6格式。 需要注意： - 多机场景下，device_ip必须配置。 - 单机场景下，device_ip可不配置。 可以在当前AI Server执行指令cat /etc/hccn.conf获取网卡IP，例如： address_0=xx.xx.xx.xx netmask_0=xx.xx.xx.xx netdetect_0=xx.xx.xx.xx address_1=xx.xx.xx.xx netmask_1=xx.xx.xx.xx netdetect_1=xx.xx.xx.xx ... 查询到的address_xx即为网卡IP，address后的序号为昇腾AI处理器物理ID，即device_id，后面的ip地址即为需要用户填入的该device对应的网卡IP。	可选

配置项	配置说明	可选/必选
rank_id	Rank唯一标识，请配置为整数，从0开始配置，且全局唯一，取值范围：[0, 总Device数量-1]。 为方便管理，建议rank_id按照Device物理连接顺序进行排序，即将物理连接上较近的device编排在一起。 例如，若device_ip按照物理连接从小到大设置，则rank_id也建议按照从小到大的顺序设置。	必选

配置文件说明（Atlas 300I Duo 推理卡）

针对Atlas 300I Duo 推理卡，ranktable文件配置说明如下：

```
{
  "status": "completed", // ranktable可用标识，completed为可用
  "version": "1.0",      // ranktable模板版本信息,当前必须为"1.0"
  "server_count": "1",  // 参与训练的AI Server数目，此例中，只有一个AI Server
  "server_list":
  [
    {
      "device": [ // AI Server中的Device列表
        {
          "device_id": "0", // 处理器物理ID
          "device_ip": "192.168.1.8", // 处理器真实网卡IP
          "rank_id": "0"    // rank的标识，rankID从0开始
        },
        {
          "device_id": "1",
          "device_ip": "192.168.1.9",
          "rank_id": "1"
        }
      ],
      "server_id": "node_0" // AI Server标识，String类型，请确保全局唯一
    }
  ]
}
```

表 6-2 ranktable 文件说明

配置项	配置说明	可选/必选
status	ranktable可用标识。 - completed：表示ranktable可用，可执行训练。 - initializing：表示ranktable不可用，不可执行训练。	必选
version	ranktable模板版本信息，当前仅支持配置为1.0。	必选
server_count	本次参与训练的AI Server个数。	必选
server_list	本次参与训练的AI Server列表。	必选

配置项	配置说明	可选/必选
server_id	AI Server标识，字符串类型，长度小于64，请确保全局唯一。 配置示例：node_0	必选
device_id	昇腾AI处理器的物理ID，即Device在AI Server上的序列号。 可通过执行“ls /dev/davinci*”命令获取昇腾AI处理器的物理ID。 例如：显示/dev/davinci0，表示昇腾AI处理器的物理ID为0。 取值范围：[0，实际Device数量-1]。 须知 “device_id”配置项的优先级高于环境变量“ASCEND_DEVICE_ID”。	必选
device_ip	昇腾AI处理器集成网卡IP，全局唯一，要求为常规IPv4或IPv6格式。 可以在当前AI Server执行指令“cat /etc/hccn.conf”获取网卡IP，例如： address_0=xx.xx.xx.xx netmask_0=xx.xx.xx.xx netdetect_0=xx.xx.xx.xx address_1=xx.xx.xx.xx netmask_1=xx.xx.xx.xx netdetect_1=xx.xx.xx.xx ... 查询到的address_xx即为网卡IP，address后的序号为昇腾AI处理器的物理ID，即device_id，后面的ip地址即为需要用户填入的该device对应的网卡IP。	必选
rank_id	Rank唯一标识，请配置为整数，从0开始配置，且全局唯一，取值范围：[0, 总Device数量-1] 为方便管理，建议rank_id按照Device物理连接顺序进行排序，即将物理连接上较近的device编排在一起。 例如，若device_ip按照物理连接从小到大设置，则rank_id也建议按照从小到大的顺序设置。	必选

配置文件说明（Atlas 训练系列产品）

针对Atlas 训练系列产品，在ranktable文件中配置参与训练的昇腾AI处理器信息支持两种配置模板，全新场景推荐使用模板一，模板二用于兼容部分已有场景。

- 模板一（推荐使用）

```
{  
  "status": "completed", // ranktable可用标识，completed为可用  
  "version": "1.0",      // ranktable模板版本信息,当前必须为"1.0"  
  "server_count": "1",  // 参与训练的AI Server数目，此例中，只有一个AI Server  
  "server_list":  
  [
```

```
{
  "device": [ // AI Server中的Device列表
    {
      "device_id": "0", // 处理器物理ID
      "device_ip": "192.168.1.8", // 处理器真实网卡IP
      "rank_id": "0" // rank的标识, rankID从0开始
    },
    {
      "device_id": "1",
      "device_ip": "192.168.1.9",
      "rank_id": "1"
    }
  ],
  "server_id": "node_0" //AI Server标识, String类型, 请确保全局唯一
}
```

表 6-3 ranktable 文件说明

配置项	配置说明	可选/必选
status	ranktable可用标识。 - completed：表示ranktable可用，可执行训练。 - initializing：表示ranktable不可用，不可执行训练。	必选
version	ranktable模板版本信息，当前仅支持配置为1.0。	必选
server_count	本次参与训练的AI Server个数。	必选
server_list	本次参与训练的AI Server列表。	必选
server_id	AI Server标识，字符串类型，长度小于64，请确保全局唯一。 配置示例：node_0	必选
device_id	昇腾AI处理器的物理ID，即Device在AI Server上的序列号。 可通过执行“ls /dev/davinci*”命令获取昇腾AI处理器的物理ID。 例如：显示/dev/davinci0，表示昇腾AI处理器的物理ID为0。 取值范围：[0，实际Device数量-1]。 须知 “device_id”配置项的优先级高于环境变量“ASCEND_DEVICE_ID”。	必选

配置项	配置说明	可选/必选
device_ip	昇腾AI处理器集成网卡IP，全局唯一，要求为常规IPv4或IPv6格式。 可以在当前AI Server执行指令“cat /etc/hccn.conf”获取网卡IP，例如： <pre>address_0=xx.xx.xx.xx netmask_0=xx.xx.xx.xx netdetect_0=xx.xx.xx.xx address_1=xx.xx.xx.xx netmask_1=xx.xx.xx.xx netdetect_1=xx.xx.xx.xx ...</pre> 查询到的address_xx即为网卡IP，address后的序号为昇腾AI处理器的物理ID，即device_id，后面的ip地址即为需要用户填入的该device对应的网卡IP。	必选
rank_id	Rank唯一标识，请配置为整数，从0开始配置，且全局唯一，取值范围：[0, 总Device数量-1] 为方便管理，建议rank_id按照Device物理连接顺序进行排序，即将物理连接上较近的device编排在一起。 例如，若device_ip按照物理连接从小到大设置，则rank_id也建议按照从小到大的顺序设置。	必选

- 模板二（兼容部分已有场景，新版本不推荐使用）

```
{
  "status": "completed", // ranktable可用标识, completed为可用
  "group_count": "1",    // group数量, 建议为1
  "group_list":          // group列表
  [
    {
      "group_name": "hccl_world_group", // group名称, 建议hccl_world_group
      "instance_count": "2",           // instance实例个数, 容器场景下可理解为容器个数
      "device_count": "2",             // group中的所有device数目
      "instance_list": [               // instance实例信息列表
        {
          "pod_name": "tf-bae41",      // instance实例名称, 一般为容器名称
          "server_id": "node_0",       // AI Server标识, String类型, 请确保全局唯一
          "devices": [                 // instance实例的device列表
            {
              "device_id": "0",        // 昇腾AI处理器的物理ID
              "device_ip": "192.168.1.8" // 昇腾AI处理器真实网卡IP
            }
          ]
        },
        {
          "pod_name": "tf-tbdf1",
          "server_id": "node_1",
          "devices": [
            {
              "device_id": "1",
              "device_ip": "192.168.1.9"
            }
          ]
        }
      ]
    }
  ]
}
```

```
    ]  
  }  
]  
}
```

表 6-4 ranktable 文件说明

配置项	配置说明	可选/必选
status	ranktable可用标识。 - completed：表示ranktable可用，可执行训练。 - initializing：表示ranktable不可用，不可执行训练。	必选
group_count	用户申请的group数量，建议配置为1。	必选
group_list	Group列表。	必选
group_name	Group名称，当group_count为1时，建议配置为hccl_world_group或者空。因为当前版本无论定义为任何值，都会创建名称为hccl_world_group的group。 如果通过该配置文件创建了多个group，则系统会自动将多个group合并为一个名称为“hccl_world_group”的group资源。	可选
instance_count	和instance_list中pod_name个数保持一致，例如：容器场景下为容器实际数量。	必选
device_count	group中设备数量。	必选
instance_list	instance实例信息列表。	必选
pod_name	用户自定义配置，保持instance_list内全局唯一。	必选
server_id	AI Server标识，字符串类型，长度小于64，请确保全局唯一。 配置示例：node_0	必选
devices	devices信息列表。	必选
device_id	昇腾AI处理器的物理ID，即Device在Server上的序列号。 可通过执行“ls /dev/davinci*”命令获取昇腾AI处理器的物理ID。 例如：显示/dev/davinci0，表示昇腾AI处理器的物理ID为0。 取值范围：[0，实际Device数量-1]。 须知 “device_id”配置项的优先级高于环境变量“ASCEND_DEVICE_ID”。	必选

配置项	配置说明	可选/必选
device_ip	昇腾AI处理器集成网卡IP，全局唯一，要求为常规IPv4或IPv6格式。 可以在当前Server执行指令cat /etc/hccn.conf获取网卡IP。	必选

6.5.3 环境变量配置资源信息

除了通过ranktable文件配置资源信息的方式外，开发者还可以通过本节所述的环境变量组合的方式配置资源信息。

该资源信息配置方式仅适用于TensorFlow框架网络的通信域初始化，仅支持如下产品型号：

Atlas 训练系列产品

Atlas A2 训练系列产品

配置说明

需要在执行训练的每个AI Server节点上分别配置如下环境变量，进行资源信息的配置，示例如下：

```
export CM_CHIEF_IP = 192.168.1.1
export CM_CHIEF_PORT = 6000
export CM_CHIEF_DEVICE = 0
export CM_WORKER_SIZE = 8
export CM_WORKER_IP = 192.168.0.1
export HCCL_SOCKET_FAMILY=AF_INET
```

- CM_CHIEF_IP、CM_CHIEF_PORT、CM_CHIEF_DEVICE用于配置Master节点的Host监听IP、监听端口与主Device ID。其中Master节点为集群管理主节点，负责集群内设备信息的管理、资源的分配和调度等。
 - 监听IP可以通过ifconfig命令进行查询，要求为常规IPv4或IPv6格式。
 - 指定的监听端口号需要确保在训练进程拉起时，无其他业务占用。
- CM_WORKER_SIZE：用于配置组网中参与集群训练的Device总数量。
- CM_WORKER_IP：用于配置当前节点与Master进行通信时所用的网卡IP，可通过ifconfig命令查询，要求为常规IPv4或IPv6格式。
需要确保指定的网卡IP能够与Master节点正常通信。
- HCCL_SOCKET_FAMILY：此环境变量可选，用于控制Device侧通信网卡使用的IP协议版本。AF_INET代表使用IPv4协议，AF_INET6代表使用IPv6协议，缺省时，优先使用IPv4协议。

说明

如果环境变量“HCCL_SOCKET_FAMILY”指定的IP协议与实际获取到的网卡信息不匹配，则以实际环境上的网卡信息为准。

例如，环境变量“HCCL_SOCKET_FAMILY”指定为“AF_INET6”，但Device侧只存在IPv4协议的网卡，则实际会使用IPv4协议的网卡。

配置示例

假设执行分布式训练的Server节点数量为2，Device数量为16为例，每个Server节点有8个Device。拉起每个Device上的训练进程前，在对应的shell窗口中配置如下环境变量，进行资源信息的配置。

- 节点0，此节点为Master节点，负责集群信息管理、资源分配与调度。

```
export CM_CHIEF_IP = 192.168.1.1
export CM_CHIEF_PORT = 6000
export CM_CHIEF_DEVICE = 0
export CM_WORKER_SIZE = 16
export CM_WORKER_IP = 192.168.1.1
```

- 节点1

```
export CM_CHIEF_IP = 192.168.1.1
export CM_CHIEF_PORT = 6000
export CM_CHIEF_DEVICE = 0
export CM_WORKER_SIZE = 16
export CM_WORKER_IP = 192.168.2.1
```

6.6 样例代码

6.6.1 安装配置 MPI

HCCL的通信域初始化依赖MPI拉起多个进程，所以进行HCCL的代码样例编写前，需要先安装配置MPI软件包。

MPI软件包的安装配置可参见《[HCCL性能测试工具使用指南](#)》中的“MPI安装与配置”章节。

6.6.2 HcclCommInitClusterInfo 初始化方式

准备 ranktable 文件

该样例通过获取ranktable的方式进行初始化，所以需准备一份ranktable文件配置集群信息，供后续调用接口时使用。

配置“RANK_TABLE_FILE”环境变量，指定ranktable文件所在路径，如下所示，文件名称为“ranktable.json”。

```
export RANK_TABLE_FILE=/home/test/ranktable.json
```

以Atlas A2 训练系列产品，组网为单机8卡为例，ranktable.json配置示例如下，不同产品形态ranktable文件的配置示例及详细参数说明可参见[6.5.2 ranktable文件配置资源信息](#)。

```
{
  "status": "completed", // ranktable可用标识，completed为可用
  "version": "1.0",
  "server_count": "1", // 参与训练的AI Server数目
  "server_list": [{
    "server_id": "SERVER_ID_SV1", // AI Server标识，String类型，请确保全局唯一
    "device": [{ // AI Server中的Device列表
      "device_id": "0",
      "device_ip": "192.168.1.8",
      "rank_id": "0"
    },
    {
      "device_id": "1",
      "device_ip": "192.168.1.9",
      "rank_id": "1"
    }
  ]
}
```

```
    },  
    {  
        "device_id": "2",  
        "device_ip": "192.168.1.10",  
        "rank_id": "2"  
    },  
    {  
        "device_id": "3",  
        "device_ip": "192.168.1.11",  
        "rank_id": "3"  
    },  
    {  
        "device_id": "4",  
        "device_ip": "192.168.1.12",  
        "rank_id": "4"  
    },  
    {  
        "device_id": "5",  
        "device_ip": "192.168.1.13",  
        "rank_id": "5"  
    },  
    {  
        "device_id": "6",  
        "device_ip": "192.168.1.14",  
        "rank_id": "6"  
    },  
    {  
        "device_id": "7",  
        "device_ip": "192.168.1.15",  
        "rank_id": "7"  
    }  
}  
}
```

HcclSend/HcclRecv 操作代码样例

该样例仅支持单机8卡的组网。

```
#include <iostream>  
#include <vector>  
#include <memory>  
#include <thread>  
#include <chrono>  
#include "hccl/hccl.h"  
#include "hccl/hccl_types.h"  
#include "mpi.h"  
  
#define ACLCHECK(ret) do { \  
    if(ret != ACL_SUCCESS) \  
    { \  
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \  
        return ret; \  
    } \  
} while(0)  
  
#define HCCLCHECK(ret) do { \  
    if(ret != HCCL_SUCCESS) \  
    { \  
        printf("hccl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \  
        return ret; \  
    } \  
} while(0)  
  
struct ThreadContext {  
    HcclComm comm;  
    int32_t device;  
};  
int Sample(void *arg)  
{
```

```
ThreadContext* ctx = (ThreadContext *)arg;
// 申请通信用device、sendBuf, recvBuf内存、stream等资源
ACLCHECK(aclrtSetDevice(ctx->device));
aclrtStream stream;
ACLCHECK(aclrtCreateStream(&stream));
void* sendBuff;
void* recvBuff;
void* hostBuff;
uint64_t count = 8;
int mallocSize = count * sizeof(float);
//初始化输入内存
ACLCHECK(aclrtMallocHost((void*)&hostBuff, mallocSize));
float* tmpHostBuff = static_cast<float*>(hostBuff);
for (uint32_t i = 0; i < count; ++i) {
    tmpHostBuff[i] = 2;
}
ACLCHECK(aclrtMalloc((void*)&sendBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
ACLCHECK(aclrtMemcpy((void*)&sendBuff, mallocSize, (void*)hostBuff, mallocSize,
ACL_MEMCPY_HOST_TO_DEVICE));
ACLCHECK(aclrtMalloc((void*)&recvBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
// 执行SendRecv操作
if (ctx->device / 4 == 0) {
    HCCHECK(HcclSend(sendBuff, count, HCCL_DATA_TYPE_FP32, ctx->device + 4, ctx->comm, stream));
} else {
    HCCHECK(HcclRecv(recvBuff, count, HCCL_DATA_TYPE_FP32, ctx->device - 4, ctx->comm, stream));
}
ACLCHECK(aclrtSynchronizeStream(stream));

if (ctx->device / 4 == 1) {
    void* resultBuff;
    ACLCHECK(aclrtMallocHost((void*)&resultBuff, mallocSize));
    ACLCHECK(aclrtMemcpy((void*)resultBuff, mallocSize, (void*)recvBuff, mallocSize,
ACL_MEMCPY_DEVICE_TO_HOST));
    float* tmpResBuff = static_cast<float*>(resultBuff);
    for (uint32_t i = 0; i < count; ++i) {
        std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
    }
    ACLCHECK(aclrtFreeHost(resultBuff));
}
// 释放通信用sendBuf、recvBuf内存, stream等资源
ACLCHECK(aclrtFreeHost(hostBuff));
ACLCHECK(aclrtFree(recvBuff));
ACLCHECK(aclrtFree(sendBuff));
ACLCHECK(aclrtDestroyStream(stream));
ACLCHECK(aclrtResetDevice(ctx->device));
return 0;
}
int main()
{
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 获取ranktable路径
    char* rankTableFile = getenv("RANK_TABLE_FILE");
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));
    HcclComm hcclComm;
    HcclCommInitClusterInfo(rankTableFile, devId, &hcclComm);
    struct ThreadContext args;
    args.comm = hcclComm;
    args.device = devId;
    Sample((void *)&args);
}
```



```
HCCLCHECK(HcclCommDestroy(hcclComm));  
// 设备资源去初始化  
ACLCHECK(aclFinalize());  
MPI_Finalize();  
return 0;  
}
```

HcclAllReduce 操作代码样例

该样例支持单机N卡的组网，N需要小于等于8。

```
#include <iostream>  
#include <vector>  
#include <memory>  
#include <thread>  
#include <chrono>  
#include "hccl/hccl.h"  
#include "hccl/hccl_types.h"  
#include "mpi.h"  
  
#define ACLCHECK(ret) do { \  
    if(ret != ACL_SUCCESS) \  
    { \  
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \  
        return ret; \  
    } \  
} while(0)  
  
#define HCCLCHECK(ret) do { \  
    if(ret != HCCL_SUCCESS) \  
    { \  
        printf("hccl interface return err return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \  
        return ret; \  
    } \  
} while(0)  
  
struct ThreadContext {  
    HcclComm comm;  
    int32_t device;  
};  
int Sample(void *arg)  
{  
    ThreadContext* ctx = (ThreadContext *)arg;  
    void* host_buf = nullptr;  
    void* send_buff = nullptr;  
    void* recv_buff = nullptr;  
    uint64_t count = 1;  
    int malloc_kSize = count * sizeof(float);  
    aclrtEvent start_event, end_event;  
    aclrtStream stream;  
    ACLCHECK(aclrtCreateStream(&stream));  
    ACLCHECK(aclrtCreateEvent(&start_event));  
    ACLCHECK(aclrtCreateEvent(&end_event));  
  
    //申请集合通信操作的内存  
    ACLCHECK(aclrtMalloc((void**)&send_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));  
    ACLCHECK(aclrtMalloc((void**)&recv_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));  
  
    //初始化输入内存  
    ACLCHECK(aclrtMallocHost((void**)&host_buf, malloc_kSize));  
    ACLCHECK(aclrtMemcpy((void*)send_buff, malloc_kSize, (void*)host_buf, malloc_kSize,  
        ACL_MEMCPY_HOST_TO_DEVICE));  
  
    //执行集合通信操作  
    HCCLCHECK(HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32,  
        HCCL_REDUCE_SUM, ctx->comm, stream));  
  
    //等待stream中集合通信任务执行完成  
    ACLCHECK(aclrtSynchronizeStream(stream));  
}
```

```
if (ctx->device < 8) {
    void* resultBuff;
    ACLCHECK(aclrtMallocHost((void**)&resultBuff, malloc_kSize));
    ACLCHECK(aclrtMemcpy((void*)resultBuff, malloc_kSize, (void*)recv_buff, malloc_kSize,
ACL_MEMCPY_DEVICE_TO_HOST));
    float* tmpResBuff = static_cast<float*>(resultBuff);
    for (uint32_t i = 0; i < count; ++i) {
        std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
    }
    ACLCHECK(aclrtFreeHost(resultBuff));
}

ACLCHECK(aclrtFree(send_buff));
ACLCHECK(aclrtFree(recv_buff));
ACLCHECK(aclrtFreeHost(host_buf));
//销毁任务流
ACLCHECK(aclrtDestroyStream(stream));
ACLCHECK(aclrtDestroyEvent(start_event));
ACLCHECK(aclrtDestroyEvent(end_event));
}

int main()
{
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 获取ranktable路径
    char* rankTableFile = getenv("RANK_TABLE_FILE");
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));
    HcclComm hcclComm;
    HcclCommInitClusterInfo(rankTableFile, devId, &hcclComm);
    struct ThreadContext args;
    args.comm = hcclComm;
    args.device = devId;
    Sample((void *)&args);
    HCCLCHECK(HcclCommDestroy(hcclComm));
    // 设备资源去初始化
    ACLCHECK(aclFinalize());
    MPI_Finalize();
    return 0;
}
```

6.6.3 HcclCommInitClusterInfoConfig 初始化方式

准备 ranktable 文件

该样例通过获取ranktable的方式进行初始化，所以需准备一份ranktable文件配置集群信息，供后续调用接口时使用。

配置“RANK_TABLE_FILE”环境变量，指定ranktable文件所在路径，如下所示，文件名称为“ranktable.json”。

```
export RANK_TABLE_FILE=/home/test/ranktable.json
```

以Atlas A2 训练系列产品，组网为单机8卡为例，ranktable.json配置示例如下，不同产品形态ranktable文件的配置示例及详细参数说明可参见[6.5.2 ranktable文件配置资源信息](#)。

```
{
  "status": "completed", // ranktable可用标识, completed为可用
  "version": "1.0",
  "server_count": "1", // 参与训练的AI Server数目
  "server_list": [{
    "server_id": "SERVER_ID_SV1", // AI Server标识, String类型, 请确保全局唯一
    "device": [{ // AI Server中的Device列表
      "device_id": "0",
      "device_ip": "192.168.1.8",
      "rank_id": "0"
    },
    {
      "device_id": "1",
      "device_ip": "192.168.1.9",
      "rank_id": "1"
    },
    {
      "device_id": "2",
      "device_ip": "192.168.1.10",
      "rank_id": "2"
    },
    {
      "device_id": "3",
      "device_ip": "192.168.1.11",
      "rank_id": "3"
    },
    {
      "device_id": "4",
      "device_ip": "192.168.1.12",
      "rank_id": "4"
    },
    {
      "device_id": "5",
      "device_ip": "192.168.1.13",
      "rank_id": "5"
    },
    {
      "device_id": "6",
      "device_ip": "192.168.1.14",
      "rank_id": "6"
    },
    {
      "device_id": "7",
      "device_ip": "192.168.1.15",
      "rank_id": "7"
    }
  ]
}]
}
```

HcclSend/HcclRecv 操作代码样例

该样例仅支持单机8卡的组网。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include <cstring>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
#include "mpi.h"

#define ACLCHECK(ret) do { \
  if(ret != ACL_SUCCESS)\
  {\
    printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
    return ret;\
  }\
}
```

```
} while(0)

#define HCCLCHECK(ret) do { \
    if(ret != HCCL_SUCCESS) \
    { \
        printf("hccl interface return errreturn err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \
        return ret;\
    } \
} while(0)

struct ThreadContext {
    HcclComm comm;
    int32_t device;
};
int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    // 申请通信用device、sendBuf, recvBuf内存、stream等资源
    ACLCHECK(aclrtSetDevice(ctx->device));
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    void* sendBuf;
    void* recvBuf;
    void* hostBuf;
    uint64_t count = 8;
    int mallocSize = count * sizeof(float);
    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void**)&hostBuf, mallocSize));
    float* tmpHostBuf = static_cast<float*>(hostBuf);
    for (uint32_t i = 0; i < count; ++i) {
        tmpHostBuf[i] = 2;
    }
    ACLCHECK(aclrtMalloc((void**)&sendBuf, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMemcpy((void*)sendBuf, mallocSize, (void*)hostBuf, mallocSize,
    ACL_MEMCPY_HOST_TO_DEVICE));
    ACLCHECK(aclrtMalloc((void**)&recvBuf, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    // 执行SendRecv操作
    if (ctx->device / 4 == 0) {
        HCCLCHECK(HcclSend(sendBuf, count, HCCL_DATA_TYPE_FP32, ctx->device + 4, ctx->comm, stream));
    } else {
        HCCLCHECK(HcclRecv(recvBuf, count, HCCL_DATA_TYPE_FP32, ctx->device - 4, ctx->comm, stream));
    }
    ACLCHECK(aclrtSynchronizeStream(stream));

    if (ctx->device / 4 == 1) {
        void* resultBuf;
        ACLCHECK(aclrtMallocHost((void**)&resultBuf, mallocSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuf, mallocSize, (void*)recvBuf, mallocSize,
    ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuf = static_cast<float*>(resultBuf);
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuf[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuf));
    }
    // 释放通信用sendBuf、recvBuf内存, stream等资源
    ACLCHECK(aclrtFreeHost(hostBuf));
    ACLCHECK(aclrtFree(recvBuf));
    ACLCHECK(aclrtFree(sendBuf));
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtResetDevice(ctx->device));
    return 0;
}
int main()
{
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
```

```
MPI_Comm_size(MPI_COMM_WORLD, &procSize);
MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
int devId = procRank;
int devCount = procSize;
// 设备资源初始化
ACL_CHECK(aclInit(NULL));
// 获取ranktable路径
char* rankTableFile = getenv("RANK_TABLE_FILE");
// 指定集合通信操作使用的设备
ACL_CHECK(aclrtSetDevice(devId));

// 创建并初始化通信域配置项
HcclCommConfig config;
HcclCommConfigInit(&config);
// 根据需要修改通信域配置
config.hcclBufferSize = 50;
strcpy(config.hcclCommName, "comm_1");

HcclComm hcclComm;
HcclCommInitClusterInfoConfig(rankTableFile, devId, &config, &hcclComm);
struct ThreadContext args;
args.comm = hcclComm;
args.device = devId;
Sample((void *)&args);
HCCL_CHECK(HcclCommDestroy(hcclComm));
// 设备资源去初始化
ACL_CHECK(aclFinalize());
MPI_Finalize();
return 0;
}
```

HcclAllReduce 操作代码样例

该样例支持单机N卡的组网，N需要小于等于8。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include <cstring>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
#include "mpi.h"

#define ACL_CHECK(ret) do { \
    if (ret != ACL_SUCCESS) \
    { \
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \
        return ret; \
    } \
} while(0)

#define HCCL_CHECK(ret) do { \
    if (ret != HCCL_SUCCESS) \
    { \
        printf("hccl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \
        return ret; \
    } \
} while(0)

struct ThreadContext {
    HcclComm comm;
    int32_t device;
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    void* host_buf = nullptr;
```

```
void* send_buff = nullptr;
void* recv_buff = nullptr;
uint64_t count = 1;
int malloc_kSize = count * sizeof(float);
aclrtEvent start_event, end_event;
aclrtStream stream;
ACLCHECK(aclrtCreateStream(&stream));
ACLCHECK(aclrtCreateEvent(&start_event));
ACLCHECK(aclrtCreateEvent(&end_event));

//申请集合通信操作的内存
ACLCHECK(aclrtMalloc((void**)&send_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));
ACLCHECK(aclrtMalloc((void**)&recv_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));

//初始化输入内存
ACLCHECK(aclrtMallocHost((void**)&host_buf, malloc_kSize));
ACLCHECK(aclrtMemcpy((void*)send_buff, malloc_kSize, (void*)host_buf, malloc_kSize,
ACL_MEMCPY_HOST_TO_DEVICE));

//执行集合通信操作
HCCHECK(HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32,
HCCL_REDUCE_SUM, ctx->comm, stream));
//等待stream中集合通信任务执行完成
ACLCHECK(aclrtSynchronizeStream(stream));

if (ctx->device < 8) {
    void* resultBuff;
    ACLCHECK(aclrtMallocHost((void**)&resultBuff, malloc_kSize));
    ACLCHECK(aclrtMemcpy((void*)resultBuff, malloc_kSize, (void*)recv_buff, malloc_kSize,
ACL_MEMCPY_DEVICE_TO_HOST));
    float* tmpResBuff = static_cast<float*>(resultBuff);
    for (uint32_t i = 0; i < count; ++i) {
        std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
    }
    ACLCHECK(aclrtFreeHost(resultBuff));
}

ACLCHECK(aclrtFree(send_buff));
ACLCHECK(aclrtFree(recv_buff));
ACLCHECK(aclrtFreeHost(host_buf));
//销毁任务流
ACLCHECK(aclrtDestroyStream(stream));
ACLCHECK(aclrtDestroyEvent(start_event));
ACLCHECK(aclrtDestroyEvent(end_event));
return 0;
}

int main()
{
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 获取ranktable路径
    char* rankTableFile = getenv("RANK_TABLE_FILE");
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));

    // 创建并初始化通信域配置项
    HcclCommConfig config;
    HcclCommConfigInit(&config);
    // 根据需要修改通信域配置
    config.hcclBufferSize = 50;
```

```
strcpy(config.hcclCommName, "comm_1");

HcclComm hcclComm;
HcclCommInitClusterInfoConfig(rankTableFile, devId, &config, &hcclComm);
struct ThreadContext args;
args.comm = hcclComm;
args.device = devId;
Sample((void *)&args);
HCCLCHECK(HcclCommDestroy(hcclComm));
// 设备资源去初始化
ACLCHECK(aclFinalize());
MPI_Finalize();
return 0;
}
```

6.6.4 HcclCommInitRootInfo 初始化方式

HcclSend/HcclRecv 操作代码样例

该样例仅支持单机8卡的组网。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
#include "mpi.h"

#define ACLCHECK(ret) do {\
    if(ret != ACL_SUCCESS)\
    {\
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)\

#define HCCLCHECK(ret) do {\
    if(ret != HCCL_SUCCESS)\
    {\
        printf("hccl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)

struct ThreadContext {\
    HcclComm comm;\
    int32_t device;\
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    // 申请通信用device、sendBuf, recvBuf内存、stream等资源
    ACLCHECK(aclrtSetDevice(ctx->device));
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    void* sendBuff;
    void* recvBuff;
    void* hostBuff;
    uint64_t count = 8;
    int mallocSize = count * sizeof(float);
    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void**)&hostBuff, mallocSize));
    float* tmpHostBuff = static_cast<float*>(hostBuff);
    for (uint32_t i = 0; i < count; ++i) {\
        tmpHostBuff[i] = 2;\
    }
}
```

```
    ACLCHECK(aclrtMalloc((void**)&sendBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMemcpy((void*)sendBuff, mallocSize, (void*)hostBuff, mallocSize,
ACL_MEMCPY_HOST_TO_DEVICE));
    ACLCHECK(aclrtMalloc((void**)&recvBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    //执行SendRecv操作
    if (ctx->device / 4 == 0) {
        HCCLCHECK(HcclSend(sendBuff, count, HCCL_DATA_TYPE_FP32, ctx->device + 4, ctx->comm, stream));
    } else {
        HCCLCHECK(HcclRecv(recvBuff, count, HCCL_DATA_TYPE_FP32, ctx->device - 4, ctx->comm, stream));
    }

    ACLCHECK(aclrtSynchronizeStream(stream));
    if (ctx->device / 4 == 1) {
        void* resultBuff;
        ACLCHECK(aclrtMallocHost((void**)&resultBuff, mallocSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuff, mallocSize, (void*)recvBuff, mallocSize,
ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuff = static_cast<float*>(resultBuff);
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuff));
    }
    // 释放通信用sendBuf、recvBuf内存，stream等资源
    ACLCHECK(aclrtFreeHost(hostBuff));
    ACLCHECK(aclrtFree(recvBuff));
    ACLCHECK(aclrtFree(sendBuff));
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtResetDevice(ctx->device));
    HCCLCHECK(HcclCommDestroy(ctx->comm));
    return 0;
}

int main() {
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    //设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));
    // 在 rootRank 获取 rootInfo
    HcclRootInfo rootInfo;
    int32_t rootRank = 0;
    if(devId == rootRank) {
        HCCLCHECK(HcclGetRootInfo(&rootInfo));
    }
    // 将root_info广播到通信域内的其他rank
    MPI_Bcast(&rootInfo, HCCL_ROOT_INFO_BYTES, MPI_CHAR, rootRank, MPI_COMM_WORLD);
    MPI_Barrier(MPI_COMM_WORLD);
    // 初始化集合通信域
    HcclComm hcclComm;
    HCCLCHECK(HcclCommInitRootInfo(devCount, &rootInfo, devId, &hcclComm));
    struct ThreadContext args;
    args.comm = hcclComm;
    args.device = devId;
    Sample((void *)&args);
    // 设备资源去初始化
    ACLCHECK(aclFinalize());
    MPI_Finalize();
    return 0;
}
```


HcclAllReduce 操作代码样例

该样例支持单机N卡的组网，N需要小于等于8。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
#include "mpi.h"

#define ACLCHECK(ret) do {\
    if(ret != ACL_SUCCESS)\
    {\
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)\

#define HCCLCHECK(ret) do {\
    if(ret != HCCL_SUCCESS)\
    {\
        printf("hccl interface return errreturn err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)

struct ThreadContext {
    HcclComm comm;
    int32_t device;
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    void* host_buf = nullptr;
    void* send_buff = nullptr;
    void* recv_buff = nullptr;
    uint64_t count = 1;
    int malloc_kSize = count * sizeof(float);
    aclrtEvent start_event, end_event;
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    ACLCHECK(aclrtCreateEvent(&start_event));
    ACLCHECK(aclrtCreateEvent(&end_event));

    //申请集合通信操作的内存
    ACLCHECK(aclrtMalloc((void**)&send_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMalloc((void**)&recv_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));

    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void**)&host_buf, malloc_kSize));
    ACLCHECK(aclrtMemcpy((void*)send_buff, malloc_kSize, (void*)host_buf, malloc_kSize,
    ACL_MEMCPY_HOST_TO_DEVICE));

    //执行集合通信操作
    HCCLCHECK(HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32,
    HCCL_REDUCE_SUM, ctx->comm, stream));

    //等待stream中集合通信任务执行完成
    ACLCHECK(aclrtSynchronizeStream(stream));

    if (ctx->device < 8) {
        void* resultBuff;
        ACLCHECK(aclrtMallocHost((void**)&resultBuff, malloc_kSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuff, malloc_kSize, (void*)recv_buff, malloc_kSize,
    ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuff = static_cast<float*>(resultBuff);
```

```
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuff));
    }

    ACLCHECK(aclrtFree(send_buff));
    ACLCHECK(aclrtFree(recv_buff));
    ACLCHECK(aclrtFreeHost(host_buf));
    //销毁任务流
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtDestroyEvent(start_event));
    ACLCHECK(aclrtDestroyEvent(end_event));
    return 0;
}

int main(int argc, char*argv[])
{
    MPI_Init(&argc, &argv);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    //设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));
    // 在 rootRank 获取 rootInfo
    HcclRootInfo rootInfo;
    int32_t rootRank = 0;
    if(devId == rootRank) {
        HCCLCHECK(HcclGetRootInfo(&rootInfo));
    }
    // 将root_info广播到通信域内的其他rank
    MPI_Bcast(&rootInfo, HCCL_ROOT_INFO_BYTES, MPI_CHAR, rootRank, MPI_COMM_WORLD);
    MPI_Barrier(MPI_COMM_WORLD);
    // 初始化集合通信域
    HcclComm hcclComm;
    HCCLCHECK(HcclCommInitRootInfo(devCount, &rootInfo, devId, &hcclComm));
    // 创建任务stream
    struct ThreadContext args;
    args.comm = hcclComm;
    args.device = devId;
    Sample((void *)&args);
    //销毁集合通信域
    HCCLCHECK(HcclCommDestroy(hcclComm));
    //重置设备
    ACLCHECK(aclrtResetDevice(devId));
    //设备去初始化
    ACLCHECK(aclFinalize());
    return 0;
}
```

6.6.5 HcclCommInitRootInfoConfig 初始化方式

HcclSend/HcclRecv 操作代码样例

该样例仅支持单机8卡的组网。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
```

```
#include "mpi.h"

#define ACLCHECK(ret) do {\
    if(ret != ACL_SUCCESS)\
    {\
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)\

#define HCCLCHECK(ret) do {\
    if(ret != HCCL_SUCCESS)\
    {\
        printf("hccl interface return errreturn err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)

struct ThreadContext {\
    HcclComm comm;\
    int32_t device;\
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    // 申请通信用device、sendBuf, recvBuf内存、stream等资源
    ACLCHECK(aclrtSetDevice(ctx->device));
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    void* sendBuff;
    void* recvBuff;
    void* hostBuff;
    uint64_t count = 8;
    int mallocSize = count * sizeof(float);
    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void**)&hostBuff, mallocSize));
    float* tmpHostBuff = static_cast<float*>(hostBuff);
    for (uint32_t i = 0; i < count; ++i) {
        tmpHostBuff[i] = 2;
    }
    ACLCHECK(aclrtMalloc((void**)&sendBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMemcpy((void*)sendBuff, mallocSize, (void*)hostBuff, mallocSize,
ACL_MEMCPY_HOST_TO_DEVICE));
    ACLCHECK(aclrtMalloc((void**)&recvBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    //执行SendRecv操作
    if (ctx->device / 4 == 0) {
        HCCLCHECK(HcclSend(sendBuff, count, HCCL_DATA_TYPE_FP32, ctx->device + 4, ctx->comm, stream));
    } else {
        HCCLCHECK(HcclRecv(recvBuff, count, HCCL_DATA_TYPE_FP32, ctx->device - 4, ctx->comm, stream));
    }

    ACLCHECK(aclrtSynchronizeStream(stream));
    if (ctx->device / 4 == 1) {
        void* resultBuff;
        ACLCHECK(aclrtMallocHost((void**)&resultBuff, mallocSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuff, mallocSize, (void*)recvBuff, mallocSize,
ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuff = static_cast<float*>(resultBuff);
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuff));
    }
    // 释放通信用sendBuf、recvBuf内存，stream等资源
    ACLCHECK(aclrtFreeHost(hostBuff));
    ACLCHECK(aclrtFree(recvBuff));
    ACLCHECK(aclrtFree(sendBuff));
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtResetDevice(ctx->device));
}
```

```
    HCCLCHECK(HcclCommDestroy(ctx->comm));  
    return 0;  
}  
int main() {  
    MPI_Init(NULL, NULL);  
    int procSize = 0;  
    int procRank = 0;  
    // 获取当前进程在所属进程组的编号  
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);  
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);  
    int devId = procRank;  
    int devCount = procSize;  
    //设备资源初始化  
    ACLCHECK(aclInit(NULL));  
    // 指定集合通信操作使用的设备  
    ACLCHECK(aclrtSetDevice(devId));  
    // 在 rootRank 获取 rootInfo  
    HcclRootInfo rootInfo;  
    int32_t rootRank = 0;  
    if(devId == rootRank) {  
        HCCLCHECK(HcclGetRootInfo(&rootInfo));  
    }  
    // 将root_info广播到通信域内的其他rank  
    MPI_Bcast(&rootInfo, HCCL_ROOT_INFO_BYTES, MPI_CHAR, rootRank, MPI_COMM_WORLD);  
    MPI_Barrier(MPI_COMM_WORLD);  
  
    // 创建并初始化通信域配置项  
    HcclCommConfig config;  
    HcclCommConfigInit(&config);  
  
    // 根据需要修改通信域配置  
    config.hcclBufferSize = 50;  
    strcpy(config.hcclCommName, "comm_1");  
  
    // 初始化集合通信域  
    HcclComm hcclComm;  
    HCCLCHECK(HcclCommInitRootInfoConfig(devCount, &rootInfo, devId, &config, &hcclComm));  
    struct ThreadContext args;  
    args.comm = hcclComm;  
    args.device = devId;  
    Sample((void *)&args);  
    // 设备资源去初始化  
    ACLCHECK(aclFinalize());  
    MPI_Finalize();  
    return 0;  
}
```

HcclAllReduce 操作代码样例

该样例支持单机N卡的组网，N需要小于等于8。

```
#include <iostream>  
#include <vector>  
#include <memory>  
#include <thread>  
#include <chrono>  
#include "hccl/hccl.h"  
#include "hccl/hccl_types.h"  
#include "mpi.h"  
  
#define ACLCHECK(ret) do {\br/>    if(ret != ACL_SUCCESS)\br/>    {\br/>        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\br/>        return ret;\br/>    }\br/>} while(0)\br/>  
#define HCCLCHECK(ret) do {\
```

```
if(ret != HCCL_SUCCESS)\
{\
    printf("hccl interface return errreturn err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
    return ret;\
}\
} while(0)

struct ThreadContext {
    HcclComm comm;
    int32_t device;
};
int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    void* host_buf = nullptr;
    void* send_buff = nullptr;
    void* recv_buff = nullptr;
    uint64_t count = 1;
    int malloc_kSize = count * sizeof(float);
    aclrtEvent start_event, end_event;
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    ACLCHECK(aclrtCreateEvent(&start_event));
    ACLCHECK(aclrtCreateEvent(&end_event));

    //申请集合通信操作的内存
    ACLCHECK(aclrtMalloc((void**)&send_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMalloc((void**)&recv_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));

    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void**)&host_buf, malloc_kSize));
    ACLCHECK(aclrtMemcpy((void*)send_buff, malloc_kSize, (void*)host_buf, malloc_kSize,
    ACL_MEMCPY_HOST_TO_DEVICE));

    //执行集合通信操作
    HCCLCHECK(HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32,
    HCCL_REDUCE_SUM, ctx->comm, stream));

    //等待stream中集合通信任务执行完成
    ACLCHECK(aclrtSynchronizeStream(stream));

    if (ctx->device < 8) {
        void* resultBuff;
        ACLCHECK(aclrtMallocHost((void**)&resultBuff, malloc_kSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuff, malloc_kSize, (void*)recv_buff, malloc_kSize,
    ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuff = static_cast<float*>(resultBuff);
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuff));
    }

    ACLCHECK(aclrtFree(send_buff));
    ACLCHECK(aclrtFree(recv_buff));
    ACLCHECK(aclrtFreeHost(host_buf));
    //销毁任务流
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtDestroyEvent(start_event));
    ACLCHECK(aclrtDestroyEvent(end_event));
    return 0;
}
int main(int argc, char*argv[])
{
    MPI_Init(&argc, &argv);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
int devId = procRank;
int devCount = procSize;
//设备资源初始化
ACL_CHECK(aclInit(NULL));
// 指定集合通信操作使用的设备
ACL_CHECK(aclrtSetDevice(devId));
// 在 rootRank 获取 rootInfo
HcclRootInfo rootInfo;
int32_t rootRank = 0;
if(devId == rootRank) {
    HCCL_CHECK(HcclGetRootInfo(&rootInfo));
}
// 将root_info广播到通信域内的其他rank
MPI_Bcast(&rootInfo, HCCL_ROOT_INFO_BYTES, MPI_CHAR, rootRank, MPI_COMM_WORLD);
MPI_Barrier(MPI_COMM_WORLD);
// 创建并初始化通信域配置项
HcclCommConfig config;
HcclCommConfigInit(&config);

// 根据需要修改通信域配置
config.hcclBufferSize = 1024;
config.hcclDeterministic = 1;
strcpy(config.hcclCommName, "comm_1");

// 初始化集合通信域
HcclComm hcclComm;
HCCL_CHECK(HcclCommInitRootInfoConfig(devCount, &rootInfo, devId, &config, &hcclComm));

// 创建任务stream
struct ThreadContext args;
args.comm = hcclComm;
args.device = devId;
Sample((void *)&args);
//销毁集合通信域
HCCL_CHECK(HcclCommDestroy(hcclComm));
//重置设备
ACL_CHECK(aclrtResetDevice(devId));
//设备去初始化
ACL_CHECK(aclFinalize());
return 0;
}
```

6.6.6 HcclCommInitAll 初始化

HcclSend/HcclRecv 操作代码样例

该样例仅支持单机8卡的组网，且仅支持单进程方式拉起。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
#include "mpi.h"

#define ACL_CHECK(ret) do { \
    if(ret != ACL_SUCCESS)\
    {\
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)

#define HCCL_CHECK(ret) do { \
    if(ret != HCCL_SUCCESS) \
```

```
{ \
    printf("hccl interface return errreturn err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \
    return ret;\
} \
} while(0)

struct ThreadContext {
    HcclComm comm;
    int32_t device;
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    // 申请通信用device、sendBuf、recvBuf内存、stream等资源
    ACLCHECK(aclrtSetDevice(ctx->device));
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    void* sendBuf;
    void* recvBuf;
    void* hostBuf;
    uint64_t count = 8;
    int mallocSize = count * sizeof(float);
    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void*)&hostBuf, mallocSize));
    float* tmpHostBuf = static_cast<float*>(hostBuf);
    for (uint32_t i = 0; i < count; ++i) {
        tmpHostBuf[i] = 2;
    }
    ACLCHECK(aclrtMalloc((void*)&sendBuf, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMemcpy((void*)sendBuf, mallocSize, (void*)hostBuf, mallocSize,
    ACL_MEMCPY_HOST_TO_DEVICE));
    ACLCHECK(aclrtMalloc((void*)&recvBuf, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
    // 执行SendRecv操作
    if (ctx->device / 4 == 0) {
        HCCLCHECK(HcclSend(sendBuf, count, HCCL_DATA_TYPE_FP32, ctx->device + 4, ctx->comm, stream));
    } else {
        HCCLCHECK(HcclRecv(recvBuf, count, HCCL_DATA_TYPE_FP32, ctx->device - 4, ctx->comm, stream));
    }

    ACLCHECK(aclrtSynchronizeStream(stream));

    if (ctx->device / 4 == 1) {
        void* resultBuf;
        ACLCHECK(aclrtMallocHost((void*)&resultBuf, mallocSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuf, mallocSize, (void*)recvBuf, mallocSize,
    ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuf = static_cast<float*>(resultBuf);
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuf[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuf));
    }

    // 释放通信用sendBuf、recvBuf内存、stream等资源
    ACLCHECK(aclrtFreeHost(hostBuf));
    ACLCHECK(aclrtFree(recvBuf));
    ACLCHECK(aclrtFree(sendBuf));
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtResetDevice(ctx->device));
    HCCLCHECK(HcclCommDestroy(ctx->comm));
    return 0;
}

int main() {
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    uint32_t ndev = 8;
    int32_t devices[8] = {0, 1, 2, 3, 4, 5, 6, 7};
    HcclComm comms[ndev];
    for (int32_t i = 0; i < ndev; i++) {
```

```
    ACLCHECK(aclrtSetDevice(devices[i]));
}
// 初始化通信域
HCCLCHECK(HcclCommInitAll(ndevid, devices, comms));

// 启动线程执行集合通信操作
std::vector<std::unique_ptr<std::thread>> threads(ndevid);
struct ThreadContext args[ndevid];
for (uint32_t i = 0; i < ndevid; i++) {
    args[i].device = i;
    args[i].comm = comms[i];
    threads[i].reset(new (std::nothrow) std::thread(&Sample, (void *)&args[i]));
}

for (uint32_t i = 0; i < ndevid; i++) {
    threads[i] -> join();
}

// 设备资源去初始化
ACLCHECK(aclFinalize());
return 0;
}
```

HcclAllReduce 操作代码样例

该样例仅支持单机8卡的组网，且仅支持单进程方式拉起。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"

#define ACLCHECK(ret) do { \
    if(ret != ACL_SUCCESS)\
    {\
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)

#define HCCLCHECK(ret) do { \
    if(ret != HCCL_SUCCESS) \
    { \
        printf("hccl interface return errreturn err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \
        return ret;\
    }\
} while(0)

struct ThreadContext {
    HcclComm comm;
    int32_t device;
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    //std::cout << "-----" << "begin rankId:" << ctx->device << "-----" <<
    std::endl;
    // 申请通信device、sendBuf, recvBuf内存、stream等资源
    ACLCHECK(aclrtSetDevice(ctx->device));
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    void* sendBuf;
    void* recvBuf;
    void* hostBuf;
```



```
uint64_t count = 8;
int mallocSize = count * sizeof(float);
//初始化输入内存
ACLCHECK(aclrtMallocHost((void**)&hostBuff, mallocSize));
float* tmpHostBuff = static_cast<float*>(hostBuff);
for (uint32_t i = 0; i < count; ++i) {
    tmpHostBuff[i] = 2;
}
ACLCHECK(aclrtMalloc((void**)&sendBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));
ACLCHECK(aclrtMemcpy((void*)sendBuff, mallocSize, (void*)hostBuff, mallocSize,
ACL_MEMCPY_HOST_TO_DEVICE));
ACLCHECK(aclrtMalloc((void**)&recvBuff, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));

// 执行AllReduce操作
HCCLCHECK(HcclAllReduce((void *)sendBuff, (void*)recvBuff, count, HCCL_DATA_TYPE_FP32,
HCCL_REDUCE_SUM, ctx->comm, stream));
ACLCHECK(aclrtSynchronizeStream(stream));

if (ctx->device < 8) {
    void* resultBuff;
    ACLCHECK(aclrtMallocHost((void**)&resultBuff, mallocSize));
    ACLCHECK(aclrtMemcpy((void*)resultBuff, mallocSize, (void*)recvBuff, mallocSize,
ACL_MEMCPY_DEVICE_TO_HOST));
    float* tmpResBuff = static_cast<float*>(resultBuff);
    for (uint32_t i = 0; i < count; ++i) {
        std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
    }
    ACLCHECK(aclrtFreeHost(resultBuff));
}
// 释放通信用sendBuf、recvBuf内存, stream等资源
ACLCHECK(aclrtFree(recvBuff));
ACLCHECK(aclrtFree(sendBuff));
ACLCHECK(aclrtFreeHost(hostBuff));
ACLCHECK(aclrtDestroyStream(stream));
ACLCHECK(aclrtResetDevice(ctx->device));
//std::cout << "-----" << "end rankId:" << ctx->device << "-----" <<
std::endl;
return 0;
}

int main() {
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    uint32_t ndev = 8;
    int32_t devices[8] = {0, 1, 2, 3, 4, 5, 6, 7};
    HcclComm comms[ndev];
    for (int32_t i = 0; i < ndev; i++) {
        ACLCHECK(aclrtSetDevice(devices[i]));
    }
    // 初始化通信域
    HCCLCHECK(HcclCommInitAll(ndev, devices, comms));
    // 启动线程执行集合通信操作
    std::vector<std::unique_ptr<std::thread>> threads(ndev);
    struct ThreadContext args[ndev];
    for (uint32_t i = 0; i < ndev; i++) {
        args[i].device = i;
        args[i].comm = comms[i];
        threads[i].reset(new (std::nothrow) std::thread(&Sample, (void *)&args[i]));
        std::chrono::seconds duration(6);
        std::this_thread::sleep_for(duration);
    }
    for (uint32_t i = 0; i < ndev; i++) {
        threads[i]->join();
    }
    // 释放通信域等相关资源
    for (uint32_t i = 0; i < ndev; i++) {
        HCCLCHECK(HcclCommDestroy(comms[i]));
    }
}
```

```
std::cout << "end end end" << std::endl;
// 设备资源去初始化
ACL_CHECK(aclFinalize());
return 0;
}
```

6.6.7 HcclCreateSubCommConfig 方式创建子通信域

HcclCreateSubCommConfig接口可以基于全局通信域创建子通信域，全局通信域可以基于ranktable文件或者root info协商方式创建，该样例以基于ranktable文件创建的全局通信域为例，给出如何创建子通信域的样例代码。

准备 ranktable 文件

该样例中全局通信域通过获取ranktable的方式进行初始化，所以需准备一份ranktable文件配置集群信息，供后续调用接口时使用。

配置“RANK_TABLE_FILE”环境变量，指定ranktable文件所在路径，如下所示，文件名称为“ranktable.json”。

```
export RANK_TABLE_FILE=/home/test/ranktable.json
```

以Atlas A2 训练系列产品，组网为单机8卡为例，ranktable.json配置示例如下，不同产品形态ranktable文件的配置示例及详细参数说明可参见[6.5.2 ranktable文件配置资源信息](#)。

```
{
  "status": "completed", // ranktable可用标识，completed为可用
  "version": "1.0",
  "server_count": "1", // 参与训练的AI Server数目
  "server_list": [{
    "server_id": "SERVER_ID_SV1", // AI Server标识，String类型，请确保全局唯一
    "device": [{ // AI Server中的Device列表
      "device_id": "0",
      "device_ip": "192.168.1.8",
      "rank_id": "0"
    },
    {
      "device_id": "1",
      "device_ip": "192.168.1.9",
      "rank_id": "1"
    },
    {
      "device_id": "2",
      "device_ip": "192.168.1.10",
      "rank_id": "2"
    },
    {
      "device_id": "3",
      "device_ip": "192.168.1.11",
      "rank_id": "3"
    },
    {
      "device_id": "4",
      "device_ip": "192.168.1.12",
      "rank_id": "4"
    },
    {
      "device_id": "5",
      "device_ip": "192.168.1.13",
      "rank_id": "5"
    },
    {
      "device_id": "6",
      "device_ip": "192.168.1.14",
      "rank_id": "6"
    }
  ]
}
```

```
    },  
    {  
        "device_id": "7",  
        "device_ip": "192.168.1.15",  
        "rank_id": "7"  
    }  
}  
}
```

HcclSend/HcclRecv 操作代码样例

该样例仅支持从单机N卡的组网中切分出1个4卡子通信域，N需要大于等于4且小于等于8。实际执行节点仅限于属于子通信域的4张卡，属于组网但不属于子通信域的节点不可执行该用例。

```
#include <iostream>  
#include <vector>  
#include <memory>  
#include <thread>  
#include <chrono>  
#include <cstring>  
#include "hccl/hccl.h"  
#include "hccl/hccl_types.h"  
#include "mpi.h"  
  
#define ACLCHECK(ret) do { \  
    if(ret != ACL_SUCCESS) \  
    { \  
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \  
        return ret; \  
    } \  
} while(0)  
  
#define HCCLCHECK(ret) do { \  
    if(ret != HCCL_SUCCESS) \  
    { \  
        printf("hccl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \  
        return ret; \  
    } \  
} while(0)  
  
struct ThreadContext {  
    HcclComm comm;  
    int32_t device;  
};  
int Sample(void *arg)  
{  
    ThreadContext* ctx = (ThreadContext *)arg;  
    // 申请通信device、sendBuf, recvBuf内存、stream等资源  
    ACLCHECK(aclrtSetDevice(ctx->device));  
    aclrtStream stream;  
    ACLCHECK(aclrtCreateStream(&stream));  
    void* sendBuf;  
    void* recvBuf;  
    void* hostBuf;  
    uint64_t count = 4;  
    int mallocSize = count * sizeof(float);  
    //初始化输入内存  
    ACLCHECK(aclrtMallocHost((void**)&hostBuf, mallocSize));  
    float* tmpHostBuf = static_cast<float*>(hostBuf);  
    for (uint32_t i = 0; i < count; ++i) {  
        tmpHostBuf[i] = 2;  
    }  
    ACLCHECK(aclrtMalloc((void**)&sendBuf, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));  
    ACLCHECK(aclrtMemcpy((void*)sendBuf, mallocSize, (void*)hostBuf, mallocSize,  
        ACL_MEMCPY_HOST_TO_DEVICE));  
    ACLCHECK(aclrtMalloc((void**)&recvBuf, mallocSize, ACL_MEM_MALLOC_HUGE_FIRST));  
    // 执行SendRecv操作
```

```
if (ctx->device / 2 == 0) {
    HCCLCHECK(HcclSend(sendBuff, count, HCCL_DATA_TYPE_FP32, ctx->device + 2, ctx->comm, stream));
} else {
    HCCLCHECK(HcclRecv(recvBuff, count, HCCL_DATA_TYPE_FP32, ctx->device - 2, ctx->comm, stream));
}
ACLCHECK(aclrtSynchronizeStream(stream));

if (ctx->device / 2 == 1) {
    void* resultBuff;
    ACLCHECK(aclrtMallocHost((void**)&resultBuff, mallocSize));
    ACLCHECK(aclrtMemcpy((void*)resultBuff, mallocSize, (void*)recvBuff, mallocSize,
ACL_MEMCPY_DEVICE_TO_HOST));
    float* tmpResBuff = static_cast<float*>(resultBuff);
    for (uint32_t i = 0; i < count; ++i) {
        std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
    }
    ACLCHECK(aclrtFreeHost(resultBuff));
}
// 释放通信用sendBuf、recvBuf内存，stream等资源
ACLCHECK(aclrtFreeHost(hostBuff));
ACLCHECK(aclrtFree(recvBuff));
ACLCHECK(aclrtFree(sendBuff));
ACLCHECK(aclrtDestroyStream(stream));
ACLCHECK(aclrtResetDevice(ctx->device));
return 0;
}
int main()
{
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 获取ranktable路径
    char* rankTableFile = getenv("RANK_TABLE_FILE");
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));

    // 创建并初始化通信域配置项
    HcclCommConfig config;
    HcclCommConfigInit(&config);
    // 根据需要修改通信域配置
    config.hcclBufferSize = 50;
    strcpy(config.hcclCommName, "comm_1");

    HcclComm globalHcclComm;
    HcclCommInitClusterInfoConfig(rankTableFile, devId, &config, &globalHcclComm);
    HcclComm hcclComm;
    strcpy(config.hcclCommName, "comm_2");
    uint32_t rankIds[4] = {0, 1, 2, 3};
    HCCLCHECK(HcclCreateSubCommConfig(&globalHcclComm, 4, rankIds, 1, devId, &config, &hcclComm));
    struct ThreadContext args;
    args.comm = hcclComm;
    args.device = devId;
    Sample((void *)&args);
    HCCLCHECK(HcclCommDestroy(hcclComm));
    // 设备资源去初始化
    ACLCHECK(aclFinalize());
    MPI_Finalize();
    return 0;
}
```

HcclAllReduce 操作代码样例

该样例仅支持从单机8卡的组网中切分出2个4卡子通信域。

```
#include <iostream>
#include <vector>
#include <memory>
#include <thread>
#include <chrono>
#include <cstring>
#include "hccl/hccl.h"
#include "hccl/hccl_types.h"
#include "mpi.h"

#define ACLCHECK(ret) do { \
    if(ret != ACL_SUCCESS)\
    {\
        printf("acl interface return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret);\
        return ret;\
    }\
} while(0)

#define HCCLCHECK(ret) do { \
    if(ret != HCCL_SUCCESS) \
    { \
        printf("hccl interface return err return err %s:%d, retcode: %d \n", __FILE__, __LINE__, ret); \
        return ret;\
    } \
} while(0)

struct ThreadContext {
    HcclComm commFront;
    HcclComm commBack;
    int32_t device;
};

int Sample(void *arg)
{
    ThreadContext* ctx = (ThreadContext *)arg;
    void* host_buf = nullptr;
    void* send_buff = nullptr;
    void* recv_buff = nullptr;
    uint64_t count = 1;
    int malloc_kSize = count * sizeof(float);
    aclrtEvent start_event, end_event;
    aclrtStream stream;
    ACLCHECK(aclrtCreateStream(&stream));
    ACLCHECK(aclrtCreateEvent(&start_event));
    ACLCHECK(aclrtCreateEvent(&end_event));

    //申请集合通信操作的内存
    ACLCHECK(aclrtMalloc((void**)&send_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));
    ACLCHECK(aclrtMalloc((void**)&recv_buff, malloc_kSize, ACL_MEM_MALLOC_HUGE_FIRST));

    //初始化输入内存
    ACLCHECK(aclrtMallocHost((void**)&host_buf, malloc_kSize));
    ACLCHECK(aclrtMemcpy((void*)send_buff, malloc_kSize, (void*)host_buf, malloc_kSize,
        ACL_MEMCPY_HOST_TO_DEVICE));

    //执行集合通信操作
    if (ctx->device < 4) {
        HCCLCHECK(HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32,
            HCCL_REDUCE_SUM, ctx->commFront, stream));
    } else {
        HCCLCHECK(HcclAllReduce((void *)send_buff, (void*)recv_buff, count, HCCL_DATA_TYPE_FP32,
            HCCL_REDUCE_SUM, ctx->commBack, stream));
    }
    //等待stream中集合通信任务执行完成
    ACLCHECK(aclrtSynchronizeStream(stream));
}
```

```
    if (ctx->device < 8) {
        void* resultBuff;
        ACLCHECK(aclrtMallocHost((void*)&resultBuff, malloc_kSize));
        ACLCHECK(aclrtMemcpy((void*)resultBuff, malloc_kSize, (void*)recv_buff, malloc_kSize,
ACL_MEMCPY_DEVICE_TO_HOST));
        float* tmpResBuff = static_cast<float*>(resultBuff);
        for (uint32_t i = 0; i < count; ++i) {
            std::cout << "rankId:" << ctx->device << ",i" << i << " " << tmpResBuff[i] << std::endl;
        }
        ACLCHECK(aclrtFreeHost(resultBuff));
    }

    ACLCHECK(aclrtFree(send_buff));
    ACLCHECK(aclrtFree(recv_buff));
    ACLCHECK(aclrtFreeHost(host_buf));
    //销毁任务流
    ACLCHECK(aclrtDestroyStream(stream));
    ACLCHECK(aclrtDestroyEvent(start_event));
    ACLCHECK(aclrtDestroyEvent(end_event));
    return 0;
}

int main()
{
    MPI_Init(NULL, NULL);
    int procSize = 0;
    int procRank = 0;
    // 获取当前进程在所属进程组的编号
    MPI_Comm_size(MPI_COMM_WORLD, &procSize);
    MPI_Comm_rank(MPI_COMM_WORLD, &procRank);
    int devId = procRank;
    int devCount = procSize;
    // 设备资源初始化
    ACLCHECK(aclInit(NULL));
    // 获取ranktable路径
    char* rankTableFile = getenv("RANK_TABLE_FILE");
    // 指定集合通信操作使用的设备
    ACLCHECK(aclrtSetDevice(devId));

    // 创建并初始化通信域配置项
    HcclCommConfig config;
    HcclCommConfigInit(&config);
    // 根据需要修改通信域配置
    config.hcclBufferSize = 50;
    strcpy(config.hcclCommName, "comm_1");

    HcclComm globalHcclComm;
    HcclCommInitClusterInfoConfig(rankTableFile, devId, &config, &globalHcclComm);
    struct ThreadContext args;
    if (devId < 4) {
        HcclComm hcclCommFront;
        strcpy(config.hcclCommName, "comm_2");
        uint32_t rankIdsFront[4] = {0, 1, 2, 3};
        HCCLCHECK(HcclCreateSubCommConfig(&globalHcclComm, 4, rankIdsFront, 1, devId, &config,
&hcclCommFront));
        args.commFront = hcclCommFront;
        args.device = devId;
        Sample((void*)&args);
        HCCLCHECK(HcclCommDestroy(hcclCommFront));
    } else {
        HcclComm hcclCommBack;
        strcpy(config.hcclCommName, "comm_3");
        uint32_t rankIdsBack[4] = {4, 5, 6, 7};
        HCCLCHECK(HcclCreateSubCommConfig(&globalHcclComm, 4, rankIdsBack, 2, devId - 4, &config,
&hcclCommBack));
        args.commBack = hcclCommBack;
        args.device = devId;
        Sample((void*)&args);
        HCCLCHECK(HcclCommDestroy(hcclCommBack));
    }
}
```

```
}  
// 设备资源去初始化  
ACLCHECK(aclFinalize());  
MPI_Finalize();  
return 0;  
}
```

6.6.8 样例编译运行

介绍目录结构

以HcclComminitRootInfo初始化方式为例，样例代码目录结构如下所示：

```
CommInitRootInfo  
├── bin  
│   └── CommInitTest  
├── Makefile  
├── src  
│   └── main.cc
```

- Makefile文件内容可参见[配置Makefile文件](#)。
- main.cc的样例代码可参见[6.6.4 HcclCommInitRootInfo初始化方式](#)。
- bin目录下的CommInitTest为编译后生成的可执行文件。

配置环境变量

配置样例编译时依赖的环境变量：

```
source /usr/local/Ascend/ascend-toolkit/set_env.sh  
export INSTALL_DIR=/usr/local/Ascend/ascend-toolkit/latest  
export PATH=/usr/local/mpich-3.2.1/bin:$PATH  
export LD_LIBRARY_PATH=/usr/local/mpich-3.2.1/lib:${INSTALL_DIR}:${LD_LIBRARY_PATH}
```

- “INSTALL_DIR”是CANN软件安装后文件存储路径，其中“/usr/local/Ascend”为root用户的默认安装路径，如果使用普通用户安装，或指定路径安装，请自行替换。
- “/usr/local/mpich-3.2.1/lib”为安装后MPI的库文件所在路径，请根据实际情况替换。

配置 Makefile 文件

Makefile文件示例如下：

```
#  
#loading path  
#-----  
-----  
CXXFLAGS := -std=c++11\  
            -Werror\  
            -fstack-protector-strong\  
            -fPIE -pie\  
            -O2\  
            -s\  
            -Wl,-z,relro\  
            -Wl,-z,now\  
            -Wl,-z,noexecstack\  
            -Wl,--copy-dt-needed-entries  
Common_DIR = ./src  
Common_SRC = $(wildcard ${Common_DIR}/*.cc)  
HCCL_INC_DIR = ${ASCEND_DIR}/include  
HCCL_LIB_DIR = ${ASCEND_DIR}/lib64  
ACL_INC_DIR = ${ASCEND_DIR}/include
```

```
ACL_LIB_DIR = ${ASCEND_DIR}/lib64
MPI_INC_DIR = ${MPI_HOME}/include
MPI_LIB_DIR = ${MPI_HOME}/lib
LIST = CommInitTest
#
#library flags
#-----
LIBS = -L$(HCCL_LIB_DIR) -lhcc\
        -L$(ACL_LIB_DIR) -lascendcl\
        -L$(MPI_LIB_DIR) -lmpi
INCLUDEDIRS = -I$(HCCL_INC_DIR)\
               -I$(ACL_INC_DIR)\
               -I$(MPI_INC_DIR)\
               -I$(Common_DIR)
#
#make
#-----
all:
    @mkdir -p bin
    g++ $(CXXFLAGS) $(Common_SRC) $(INCLUDEDIRS) -o CommInitTest $(LIBS)
    @printf "\nCommInitTest compile completed\n"
    mv $(LIST) ./bin
.PHONY: clean
clean:
    rm -rf ./bin/*Test
```

编译样例

执行如下命令编译样例：

```
make MPI_HOME=/usr/local/mpich-3.2.1/ ASCEND_DIR=/usr/local/Ascend/ascend-toolkit/latest
```

编译成功后，会在执行编译命令的bin/目录下生成CommInitTest可执行文件。

执行样例

编译完成后，通过以下指令启动：

```
mpirun -n <npu_num> ./bin/CommInitTest
```

其中“-n”是需要启动的NPU总数。

须知

HcclCommInitAll初始化方式为单进程多线程的拉起方式，所以该初始化方式的样例执行命令为mpirun -1 ./bin/CommInitTest。

7 常用工具与配置

HCCL 业务配置

HCCL提供了系列环境变量用于进行集合通信业务侧配置，可参见《[环境变量参考](#)》的“执行相关 > 集合通信与分布式训练”章节。

HCCL 性能测试

分布式训练场景下，开发者可以通过HCCL Test工具测试集合通信的功能以及性能，此工具支持测试的集合通信算子包含all_gather、all_reduce、alltoallv、alltoall、broadcast、reduce_scatter、reduce、scatter，当前集群场景的集合通信常用的是all_reduce和alltoallv算子，因此测试集合通信性能当前也建议基于all_reduce和alltoallv算子。

HCCL Test工具的详细使用可参见《[HCCL性能测试工具使用指南](#)》。

8 接口参考

[HCCL接口简介](#)

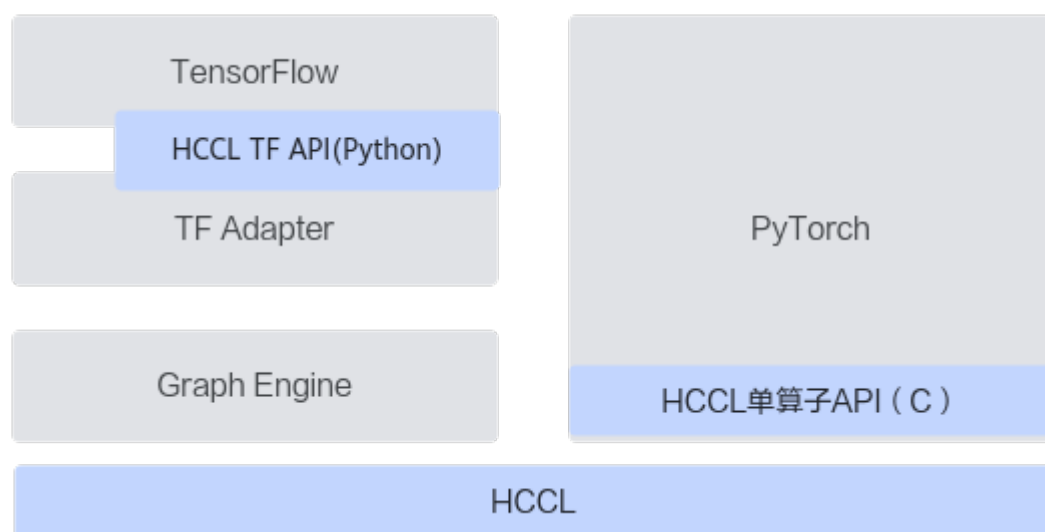
[HCCL API \(C \)](#)

[HCCL API \(Python \)](#)

8.1 HCCL 接口简介

HCCL提供了Python与C两种语言的接口，其中Python语言的接口用于实现TensorFlow网络在昇腾AI处理器执行分布式优化；C语言接口用于实现单算子模式下的框架适配，实现分布式能力，例如HCCL单算子API嵌入到PyTorch后端代码中，PyTorch用户直接使用PyTorch原生集合通信API，即可实现分布式能力。

图 8-1 集合通信 API 概览



8.2 HCCL API (C)

8.2.1 接口列表

框架开发者可以通过HCCL提供的C语言接口进行单算子模式下的框架适配，实现分布式能力。

HCCL提供的C语言的单算子API如表8-1所示。

- 开发者可在“CANN软件安装目录/include/hccl/”目录中查看接口定义。
- “hccl.h”：通信域管理与通信算子相关接口定义文件，对应的库文件为libhccl.so。
- 相关的数据类型定义在“CANN软件安装目录/include/hccl/hccl_types.h”文件中。

表 8-1 HCCL（C 语言）接口列表

分类	接口	简介
通信域管理	HcclCommInitClusterInfo	基于ranktable初始化HCCL通信域。
	HcclGetRootInfo	此接口需要在HCCL初始化接口 HcclCommInitRootInfo 或 HcclCommInitRootInfoConfig 前调用，仅需在root节点调用，用于生成root节点的rank的标识信息（HcclRootInfo）。
	HcclCommInitRootInfo	根据rootInfo初始化HCCL，创建HCCL通信域。
	HcclCommInitRootInfoConfig	根据rootInfo初始化HCCL，创建具有特定配置的HCCL通信域。
	HcclCommConfigInit	初始化通信域配置项，并将其中的可配置参数设为默认值（hcclBufferSize的默认值为200，hcclDeterministic的默认值为0）。
	HcclCommInitAll	单机通信场景中，通过一个进程统一创建多张卡的通信域（其中一张卡对应一个线程）。在初始化通信域的过程中，devices[0]作为root rank自动收集集群信息。
	HcclCommDestroy	销毁指定的HCCL通信域。
	HcclGetRankSize	查询当前集合通信域的rank总数。
	HcclGetRankId	获取device在集合通信域中对应的rank序号。
	HcclSetConfig	进行集合通信相关配置，当前仅支持配置是否支持确定性计算。
	HcclGetConfig	获取集合通信相关配置。
	HcclGetCommName	获取当前集合通信操作所在的通信域的名称。

分类	接口	简介
	HcclGetCommConfigCapability	该接口用于判断当前版本软件是否支持某项通信域初始化配置。
	HcclCreateSubCommConfig	基于既有的全局通信域，切分具有特定配置的子通信域。
集合通信	HcclAllReduce	集合通信算子AllReduce的操作接口，将通信域内所有节点的输入数据进行reduce操作后，再把结果发送到所有节点的输出buffer，其中reduce操作类型由op参数指定。
	HcclBroadcast	集合通信算子Broadcast的操作接口，将通信域内root节点的数据广播到其他rank。
	HcclAllGather	集合通信算子AllGather的操作接口，将通信域内所有节点的输入按照rank id重新排序，然后拼接起来，再将结果发送到所有节点的输出。
	HcclReduceScatter	集合通信算子ReduceScatter的操作接口。将所有rank的输入相加（或其他操作）后，再把结果按照rank编号均匀分散的到各个rank的输出buffer，每个进程拿到其他进程1/ranksize份的数据进行归约操作。
	HcclReduce	集合通信算子Reduce的操作接口，将所有rank的input相加（或其他操作）后，再把结果发送到root节点的输出buffer。
	HcclAlltoAllV	集合通信域alltoallv操作接口。向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。
	HcclAlltoAll	集合通信alltoall操作接口。向通信域内所有rank发送相同数据量的数据，并从所有rank接收相同数据量的数据。
	HcclBarrier	将指定通信域内所有rank的stream阻塞，直到所有rank都下发执行该操作为止。
	HcclScatter	集合通信域Scatter操作接口，将root节点的数据均分并散布至其他rank。
点对点通信	HcclSend	集合通信域Send操作接口。将当前rank的sendBuf数据发送至destRank。
	HcclRecv	集合通信域Recv操作接口。从srcRank接收数据到当前rank的recvBuf。
	HcclBatchSendRecv	集合通信域异步批量点对点通信操作接口。调用一次接口可以完成本rank上的多个收发任务，本rank发送和接收之间是异步的，发送和接收任务之间不会相互阻塞。

分类	接口	简介
异常处理	HcclGetCommAsyncError	当集群信息中存在Device网口通信链路不稳定、出现网络拥塞的情况时，Device日志中会存在“error cqe”的打印，我们称这种错误为“RDMA ERROR CQE”错误。 当前版本，此接口仅支持查询通信域内是否存在“RDMA ERROR CQE”的错误。
	HcclGetErrorString	解析HCCL_RESULT类型的错误码。

8.2.2 通信域管理

8.2.2.1 HcclCommInitClusterInfo

功能说明

基于ranktable初始化HCCL通信域。

ranktable文件是一个json格式的文件，配置了参与集合通信的NPU资源信息，关于ranktable文件的配置可参见[6.5.2 ranktable文件配置资源信息](#)。

函数原型

HcclResult HcclCommInitClusterInfo(const char *clusterInfo, uint32_t rank, HcclComm *comm)

参数说明

参数名	输入/输出	描述
clusterInfo	输入	rank table的文件路径（含文件名），作为字符串最大长度为4096字节，含结束符。
rank	输入	本rank的rank id。 需要注意，此参数取值需要与ranktable中对应的“rank_id”字段取值一致。
comm	输出	将初始化后的通信域以指针的信息回传给调用者。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

HcclResult：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

重复初始化会报错。

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

调用示例

可参见[6.6.2 HcclCommInitClusterInfo初始化方式](#)。

8.2.2.2 HcclCommInitClusterInfoConfig

功能说明

基于ranktable，初始化具有特定配置的HCCL通信域。

函数原型

HcclResult HcclCommInitClusterInfoConfig(const char *clusterInfo, uint32_t rank, HcclCommConfig *config, HcclComm *comm)

参数说明

参数名	输入/输出	描述
clusterInfo	输入	rank table的文件路径（含文件名），作为字符串最大长度为4096字节，含结束符。
rank	输入	本rank的rank id。 需要注意，此参数取值需要与ranktable中对应的“rank_id”字段取值一致。

参数名	输入/输出	描述
config	输入	通信域配置项，目前的可配置项为buffer大小、确定性计算开关、通信域名称。 需要注意： - 传入的config必须先调用HcclCommConfigInit对其进行初始化。 - config中的配置参数需确保在合法值域内，详见7.10 HcclCommConfig的定义。 - 若通过config指定通信域名称，需确保与其他通信域中的名称不重复。 - config中的配置项hcclBufferSize优先级高于环境变量HCCL_BUFFSIZE，配置项hcclDeterministic优先级高于环境变量HCCL_DETERMINISTIC。关于环境变量的详细说明，可参见《环境变量参考》。
comm	输出	将初始化后的通信域以指针的信息回传给调用者。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

HcclResult：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

同一通信域不支持重复初始化。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

可参见[6.6.3 HcclCommInitClusterInfoConfig初始化方式](#)。

8.2.2.3 HcclGetRootInfo

功能说明

此接口需要在HCCL初始化接口[HcclCommInitRootInfo](#)或[HcclCommInitRootInfoConfig](#)前调用，仅需在root节点调用，用于生成root节点的rank的标识信息（HcclRootInfo）。

函数原型

HcclResult HcclGetRootInfo(HcclRootInfo *rootInfo)

参数说明

参数名	输入/输出	描述
rootInfo	输出	本rank的标识信息，主要包含device ip、device id 等信息。此信息需广播至集群内所有rank用来进行 HCCL初始化。 HcclRootInfo类型的定义可参见 8.2.6.6 HcclRootInfo 。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 多机集合通信场景，调用HcclGetRootInfo前，可以进行如下操作（非必选）：
 - 配置环境变量HCCL_IF_IP或HCCL_SOCKET_IFNAME，指定HCCL的初始化 root网卡IP（环境变量网HCCL_IF_IP的优先级高于HCCL_SOCKET_IFNAME，若二者都不配置，默认使用网卡名字典序升序选择root网卡）。
 - 配置环境变量HCCL_WHITELIST_DISABLE开启白名单校验，并通过 HCCL_WHITELIST_FILE指定通信白名单配置文件（若不配置，默认关闭通信白名单校验）。
- 调用此接口后，需要以此rank为root节点，将此rank标识信息广播至集群中的所有rank，然后调用[HcclCommInitRootInfo](#)接口使用接收到的rootInfo进行HCCL初始化。
- 该接口和初始化接口[HcclCommInitRootInfo](#)配对使用，不能单独使用。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

可参见[6.6.4 HcclCommInitRootInfo初始化方式](#)。

8.2.2.4 HcclCommInitRootInfo

功能说明

根据rootInfo初始化HCCL，创建HCCL通信域。

函数原型

```
HcclResult HcclCommInitRootInfo(uint32_t nRanks, const HcclRootInfo
*rootInfo, uint32_t rank, HcclComm *comm)
```

参数说明

参数名	输入/输出	描述
nRanks	输入	集群中的rank数量。
rootInfo	输入	root rank信息，主要包含root rank的ip、id等信息，由 HcclGetRootInfo 接口生成。
rank	输入	本rank的rank id。
comm	输出	初始化后的通信域指针。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 同一通信域中所有rank的nranks、rootInfo均应相同。
- 该接口只能串行调用，不支持并发调用。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

可参见[6.6.4 HcclCommInitRootInfo初始化方式](#)。

8.2.2.5 HcclCommInitRootInfoConfig

功能说明

根据rootInfo初始化HCCL，创建具有特定配置的HCCL通信域。

函数原型

```
HcclResult HcclCommInitRootInfoConfig(uint32_t nRanks, const HcclRootInfo
*rootInfo, uint32_t rank, const HcclCommConfig *config, HcclComm *comm)
```

参数说明

参数名	输入/输出	描述
nRanks	输入	集群中的rank数量。
rootInfo	输入	root rank信息，主要包含root rank的ip、id等信息，由 HcclGetRootInfo 接口生成。
rank	输入	本rank的rank id。
config	输入	通信域配置项，目前的可配置项为buffer大小、确定性计算开关、通信域名称。 需要注意： - 传入的config必须先调用HcclCommConfigInit对其进行初始化。 - config中的配置参数需确保在合法值域内，详见7.10 HcclCommConfig的定义。 - 若通过config指定通信域名称，需确保与其他通信域中的名称不重复。 - config中的配置项hcclBufferSize优先级高于环境变量HCCL_BUFFSIZE，配置项hcclDeterministic优先级高于环境变量HCCL_DETERMINISTIC。关于环境变量的详细说明，可参见《环境变量参考》。
comm	输出	初始化后的通信域指针。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 同一通信域中所有rank的nranks、rootInfo、config均应相同。
- 该接口只能串行调用，不支持并发调用。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

可参见[6.6.5 HcclCommInitRootInfoConfig初始化方式](#)。

8.2.2.6 HcclCommConfigInit

功能说明

初始化通信域配置项，并将其中的可配置参数设为默认值（hcclBufferSize的默认值为200，hcclDeterministic的默认值为0）。

函数原型

```
inline void HcclCommConfigInit(HcclCommConfig *config)
```

参数说明

参数名	输入/输出	描述
config	输出	需要初始化的通信域配置项。 HcclCommConfig类型的定义可参见 7.10 HcclCommConfig 。

返回值

无

约束说明

无

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
HcclCommConfig config;  
HcclCommConfigInit(&config);
```

8.2.2.7 HcclCommInitAll

功能说明

单机通信场景中，通过一个进程统一创建多张卡的通信域（其中一张卡对应一个线程）。在初始化通信域的过程中，devices[0]作为root rank自动收集集群信息。

函数原型

```
HcclResult HcclCommInitAll(uint32_t ndev, int32_t* devices, HcclComm*  
comms)
```

参数说明

参数名	输入/输出	描述
ndev	输入	通信域内的device个数。
devices	输入	通信域中的device列表，其值为device的逻辑ID，通信库按照devices中设置的顺序创建通信域。 需要注意，输入的devices列表中不能包含重复的device ID。
comms	输出	生成的通信域句柄数组，其大小为：ndev * sizeof(HcclComm)。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 该接口仅支持单机通信场景使用，不支持多机通信场景。
- 多线程调用通信操作API时（例如HcclAllReduce等），用户需确保不同线程中调用通信操作API的前后时间差不超过环境变量HCCL_CONNECT_TIMEOUT的时间，避免建链超时。
- 不支持一张卡同时调用多个通信操作API。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

可参见[6.6.6 HcclCommInitAll初始化](#)。

8.2.2.8 HcclCommDestroy

功能说明

销毁指定的HCCL通信域。

函数原型

HcclResult HcclCommDestroy(HcclComm comm)

参数说明

参数名	输入/输出	描述
comm	输入	指向需要销毁的通信域的指针。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

无

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

8.2.2.9 HcclGetRankSize

功能说明

查询当前集合通信域的rank总数。

函数原型

HcclResult HcclGetRankSize(HcclComm comm, uint32_t *rankSize)

参数说明

参数名	输入/输出	描述
comm	输入	集合通信操作所在的通信域。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。
rankSize	输出	指定集合通信域的rank总数输出地址指针。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

无

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
uint32_t rankSize;  
HcclComm comm;  
HcclGetRankSize(comm, &rankSize);
```

8.2.2.10 HcclGetRankId

功能说明

获取device在集合通信域中对应的rank序号。

函数原型

HcclResult HcclGetRankId(HcclComm comm, uint32_t *rank)

参数说明

参数名	输入/输出	描述
comm	输入	集合通信操作所在的通信域。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。
rank	输出	指定集合通信域中rank序号的输出地址指针。

返回值

HcclResult：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

无

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
uint32_t rank;  
HcclComm comm;  
HcclGetRankId(comm, &rank);
```

8.2.2.11 HcclSetConfig

功能说明

进行集合通信相关配置，当前仅支持配置是否支持确定性计算。

在不开启确定性计算的场景下，多次执行的结果可能不同。这个差异的来源，一般是因为在算子实现中，存在异步的多线程执行，会导致浮点数累加的顺序变化。当开启确定性计算后，算子在相同的硬件和输入下，多次执行将产生相同的输出。

默认情况下，无需开启确定性计算，但当发现模型执行多次结果不同或者精度调优时，可以开启确定性计算辅助进行调试调优，但开启确定性计算后，算子执行时间会变慢，导致性能下降。

函数原型

HcclResult HcclSetConfig(HcclConfig config, HcclConfigValue configValue)

参数说明

参数名	输入/输出	描述
config	输入	config中可配置的参数。 HcclConfig类型，当前版本仅支持配置为“HCCL_DETERMINISTIC”。
configValue	输入	config中所配置参数的取值。 HcclConfigValue类型，针对“HCCL_DETERMINISTIC”参数，“0”代表不支持确定性计算，“1”代表支持确定性计算。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

无

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

8.2.2.12 HcclGetConfig

功能说明

获取集合通信相关配置。

函数原型

```
HcclResult HcclGetConfig(HcclConfig config, HcclConfigValue *configValue)
```

参数说明

参数名	输入/输出	描述
config	输入	集合通信配置参数。 HcclConfig类型，当前版本仅支持配置为“HCCL_DETERMINISTIC”。
configValue	输出	获取config中可配置的参数的值。 HcclConfigValue类型，针对“HCCL_DETERMINISTIC”参数，“0”代表不支持确定性计算，“1”代表支持确定性计算。

返回值

HcclResult：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

无

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

8.2.2.13 HcclGetCommName

功能说明

获取当前集合通信操作所在的通信域的名称。

函数原型

```
HcclResult HcclGetCommName(HcclComm comm, char* commName)
```

参数说明

参数名	输入/输出	描述
comm	输入	集合通信操作所在的通信域。 HcclComm类型的定义可参见8.2.6.7 HcclComm。

参数名	输入/输出	描述
commName	输出	获取到的通信域名称。 char*类型，最大长度支持128。

返回值

HcclResult: 接口成功返回HcclSuccess，其他失败。

约束说明

无

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

调用示例

```
string commName;  
HcclComm comm;  
HcclGetCommNam(comm, &commName);
```

8.2.2.14 HcclGetCommConfigCapability

功能说明

该接口用于判断当前版本软件是否支持某项通信域初始化配置。

通信域初始化时支持的完整配置项可参见[8.2.6.11 HcclCommConfigCapability](#)，包括共享数据的缓存区大小、确定性计算开关、通信域名称等。

使用HcclGetCommConfigCapability接口判断当前软件是否支持某项配置的流程为：

- 调用HcclGetCommConfigCapability接口，获取一个代表当前软件通信域初始化配置能力的数值。
- 比较该数值与[8.2.6.11 HcclCommConfigCapability](#)中某项配置枚举值的大小，若该数值大于枚举值，则代表当前软件支持[8.2.6.11 HcclCommConfigCapability](#)中对应枚举值的配置能力；若该数值小于等于枚举值，则代表不支持。

例如，若想判断当前软件是否支持配置通信域名称，则使用HcclGetCommConfigCapability接口的返回值与枚举值“HCCL_COMM_CONFIG_COMM_NAME”做比较，若返回值大于“HCCL_COMM_CONFIG_COMM_NAME”，则代表当前软件支持配置通信域名称；若返回值小于等于“HCCL_COMM_CONFIG_COMM_NAME”，否代表当前软件不支持配置通信域名称。

函数原型

uint32_t HcclGetCommConfigCapability()

参数说明

无

返回值

uint32_t: 表示通信域初始化配置能力的数值。

约束说明

无

支持的型号

Atlas 训练系列产品

Atlas 300I Duo 推理卡

Atlas A2 训练系列产品

调用示例

```
uint32_t configCapability = HcclGetCommConfigCapability();  
bool isSupportCommName = configCapability > HCCL_COMM_CONFIG_COMM_NAME; // 判断是否支持配置  
通信域名称，需使用严格大于
```

8.2.2.15 HcclCreateSubCommConfig

功能说明

基于既有的全局通信域，切分具有特定配置的子通信域。

该子通信域创建方式无需进行socket建链与rank信息交换，可应用于业务故障下的快速通信域创建。

说明

如果组网中卡间存在负载不均衡的情况，使用该接口创建的子通信域可能会由于卡间不同步发生建链超时。此时可通过环境变量HCCL_CONNECT_TIMEOUT增加设备间的建链超时时间。示例：

```
export HCCL_CONNECT_TIMEOUT=600
```

关于环境变量的详细说明，可参见《[环境变量参考](#)》。

函数原型

**HcclResult HcclCreateSubCommConfig(HcclComm *comm, uint32_t rankNum,
uint32_t *rankIds, uint64_t subCommId, uint32_t subCommRankId,
HcclCommConfig *config, HcclComm *comm)**

参数说明

参数名	输入/输出	描述
comm	输入	被切分的全局通信域。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。
rankNum	输入	需要切分的子通信域中的rank数量。
rankIds	输入	子通信域中rank在全局通信域中的rank id组成的数组。 需要注意： 该数组应当是有序的，数组中每个rank的下标将映射为其在子通信域的rank id。
subCommId	输入	当前子通信域标识。 用户自定义，在全局通信域中唯一即可。
subCommRankId	输入	本rank在子通信域中的rank id。 请配置为当前rank在rankIds数组中的下标索引。
config	输入	通信域配置项，目前的可配置项为buffer大小、确定性计算开关、通信域名称。 需要注意： - 传入的config必须先调用HcclCommConfigInit对其进行初始化。 - config中的配置参数需确保在合法值域内，详见7.10 HcclCommConfig的定义。 - 若通过config指定通信域名称，需确保与其他通信域中的名称不重复。 - config中的配置项hcclBufferSize优先级高于环境变量HCCL_BUFFSIZE，配置项hcclDeterministic优先级高于环境变量HCCL_DETERMINISTIC。关于环境变量的详细说明，可参见《环境变量参考》。
comm	输出	将初始化后的子通信域以指针的信息回传给调用者。 HcclComm类型的定义可参见 8.2.6.7 HcclComm 。

返回值

HcclResult： 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 属于同一子通信域的rank调用该接口时传入的rankNum、rankIds、subCommId、config均应相同。
- 不需要创建子通信域的rank应当传入rankIds==nullptr和subCommId=0xFFFFFFFF。

- 对于同一个全局通信域，不可使用重复的subCommId创建子通信域。
- 只支持从全局通信域切分子通信域，不支持通信域嵌套切分。

支持的型号

Atlas 训练系列产品

Atlas A2 训练系列产品

调用示例

一个简单的代码示例片段如下：

```
HcclComm globalHcclComm;  
HcclCommInitClusterInfo(rankTableFile, devId, &globalHcclComm);  
HcclCommConfig config;  
HcclCommConfigInit(&config);  
config.hcclBufferSize = 50;  
strcpy(config.hcclCommName, "comm_1");  
HcclComm hcclComm;  
uint32_t rankIds[4] = {0, 1, 2, 3};  
HCCLCHECK(HcclCreateSubCommConfig(&globalHcclComm, 4, rankIds, 1, devId, &config, &hcclComm));
```

完整的代码示例可参见[6.6.7 HcclCreateSubCommConfig方式创建子通信域](#)。

8.2.3 集合通信

8.2.3.1 HcclAllReduce

功能说明

集合通信算子AllReduce的操作接口，将通信域内所有节点的输入数据进行reduce操作后，再把结果发送到所有节点的输出buffer，其中reduce操作类型由op参数指定。

函数原型

```
HcclResult HcclAllReduce(void *sendBuf, void *recvBuf, uint64_t count,  
HcclDataType dataType, HcclReduceOp op, HcclComm comm, aclrtStream  
stream)
```

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
count	输入	参与allreduce操作的数据个数，比如只有一个int32数据参与，则count=1。

参数名	输入/输出	描述
dataType	输入	allreduce操作的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、int32、int64、float16、float32。 针对Atlas 300I Duo 推理卡，支持数据类型：int8、int16、int32、float16、float32。 针对Atlas A2 训练系列产品，支持数据类型：int8、int16、int32、int64、float16、float32、bfp16。
op	输入	reduce的操作类型，目前支持操作类型为sum、prod、max、min。 说明 针对Atlas 300I Duo 推理卡，当前版本“prod”、“max”、“min”操作不支持int16数据类型。 针对Atlas A2 训练系列产品，当前版本“prod”操作不支持int16、bfp16数据类型。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 所有rank的count、dataType、op均应相同。
- 每个rank只能有一个输入。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

可参见[6.6 样例代码](#)中每种通信域初始化方式中的“HcclAllReduce操作代码样例”。

8.2.3.2 HcclBroadcast

功能说明

集合通信算子Broadcast的操作接口，将通信域内root节点的数据广播到其他rank。

函数原型

HcclResult HcclBroadcast(void *buf, uint64_t count, HcclDataType dataType, uint32_t root, HcclComm comm, aclrtStream stream)

参数说明

参数名	输入/输出	描述
buf	输入/输出	数据buffer，对于root节点，是数据源；对于非root节点，是数据接收buffer。
count	输入	参与broadcast操作的数据个数，比如只有一个int32数据参与，则count=1。
dataType	输入	Broadcast操作的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
root	输入	作为broadcast root的rank id。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

HcclResult：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 所有rank的count、dataType、root均应相同。
- 全局只能有1个root节点。

支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

8.2.3.3 HcclAllGather

功能说明

集合通信算子AllGather的操作接口，将通信域内所有节点的输入按照rank id重新排序，然后拼接起来，再将结果发送到所有节点的输出。

 说明

针对AllGather操作，每个节点都接收按照rank id重新排序后的数据集，即每个节点的全局输出都是一样的。

函数原型

```
HcclResult HcclAllGather(void *sendBuf, void *recvBuf, uint64_t sendCount,
HcclDataType dataType, HcclComm comm, aclrtStream stream)
```

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
sendCount	输入	参与allgather操作的sendBuf的数据size，recvBuf的数据size则等于count * rank size。
dataType	输入	allgather操作的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas 300I Duo 推理卡，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

所有rank的sendCount、dataType均应相同。

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

8.2.3.4 HcclReduceScatter

功能说明

集合通信算子ReduceScatter的操作接口。将所有rank的输入相加（或其他操作）后，再把结果按照rank编号均匀分散的到各个rank的输出buffer，每个进程拿到其他进程1/ranksize份的数据进行归约操作。

函数原型

```
HcclResult HcclReduceScatter(void *sendBuf, void *recvBuf, uint64_t
recvCount, HcclDataType dataType, HcclReduceOp op, HcclComm comm,
aclrtStream stream)
```

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
recvCount	输入	参与reducescatter操作的recvBuf的数据size，sendBuf的数据size则等于recvCount * rank size。
dataType	输入	reduce-scatter操作的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、int32、int64、float16、float32。 针对Atlas A2 训练系列产品，支持数据类型：int8、int16、int32、int64、float16、float32、bfp16。
op	输入	reduce的操作类型，目前支持操作类型为sum、prod、max、min。 说明 针对Atlas A2 训练系列产品，当前版本“prod”操作不支持int16、bfp16数据类型。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

所有rank的recvCount、dataType、op均应相同。

支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

8.2.3.5 HcclReduce

功能说明

集合通信算子Reduce的操作接口，将所有rank的input相加（或其他操作）后，再把结果发送到root节点的输出buffer。

函数原型

HcclResult HcclReduce(void *sendBuf, void *recvBuf, uint64_t count, HcclDataType dataType, HcclReduceOp op, uint32_t root, HcclComm comm, aclrtStream stream)

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
count	输入	参与reduce操作的数据个数，比如只有一个int32数据参与，则count=1。
dataType	输入	reduce操作的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、int32、int64、float16、float32。 针对Atlas A2 训练系列产品，支持数据类型：int8、int16、int32、int64、float16、float32、bfp16。
op	输入	reduce的操作类型，目前支持操作类型为sum、prod、max、min。 说明 针对Atlas A2 训练系列产品，当前版本“prod”操作不支持int16、bfp16数据类型。
root	输入	作为reduce root的rankid
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

所有rank的count、dataType、op均应相同。

支持的型号

- Atlas 训练系列产品
- Atlas A2 训练系列产品

8.2.3.6 HcclAlltoAll

功能说明

集合通信alltoall操作接口。向通信域内所有rank发送相同数据量的数据，并从所有rank接收相同数据量的数据。

函数原型

```
HcclResult HcclAlltoAll(const void *sendBuf, uint64_t sendCount,
HcclDataType sendType, const void *recvBuf, uint64_t recvCount,
HcclDataType recvType, HcclComm comm, aclrtStream stream)
```

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
sendCount	输入	表示向每个rank发送的数据量。
sendType	输入	发送数据的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
recvCount	输入	表示从每个rank接收的数据量。
recvType	输入	接收数据的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。

参数名	输入/输出	描述
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 所有rank的sendCount、sendType、recvCount、recvType均应相同。
- alltoall操作的性能与NPU之间共享数据的缓存区大小有关，当通信数据量超过缓存区大小时性能将出现明显下降。若业务中alltoall通信数据量较大，建议通过配置环境变量HCCL_BUFFSIZE适当增大缓存区大小以提升通信性能。
- 针对Atlas 训练系列产品，alltoall的通信域需要满足如下约束：
单server 1p、2p通信域要在同一个cluster内（server内0-3卡和4-7卡各为一个cluster），单server4p、8p和多server通信域中rank要以cluster为基本单位，并且server间cluster选取要一致。

支持的型号

Atlas 训练系列产品

Atlas A2 训练系列产品

8.2.3.7 HcclAlltoAllV

功能说明

集合通信域alltoallv操作接口。向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。

函数原型

HcclResult HcclAlltoAllV(const void *sendBuf, const void *sendCounts, const void *sdispls, HcclDataType sendType, const void *recvBuf, const void *recvCounts, const void *rdispls, HcclDataType recvType, HcclComm comm, aclrtStream stream)

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。

参数名	输入/输出	描述
sendCounts	输入	表示发送数据量的uint64数组，“sendCounts[i] = n”表示本rank发给rank i的数据量为n。 例如，若“sendType”为float32，“sendCounts[i] = n”表示本rank发给rank i n个float32数据。
sdispls	输入	表示发送偏移量的uint64数组，“sdispls[i] = n”表示本rank发给rank i的数据在sendBuf的起始位置相对sendBuf的偏移量，以sendType为基本单位。
sendType	输入	发送数据的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
recvCounts	输入	表示接收数据量的uint64数组，“recvCounts[i] = n”表示本rank从rank i收到的数据量为n。 例如，若“recvType”为float32，“recvCounts[i] = n”表示本rank从rank i收到n个float32数据。
rdispls	输入	表示接收偏移量的uint64数组，“rdispls[i] = n”表示本rank从rank i收到的数据存放在recvBuf的起始位置相对recvBuf的偏移量，以recvType为基本单位。
recvType	输入	接收数据的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- alltoallv操作的性能与NPU之间共享数据的缓存区大小有关，当通信数据量超过缓存区大小时性能将出现明显下降。若业务中alltoallv通信数据量较大，建议通过配置环境变量HCCL_BUFFSIZE适当增大缓存区大小以提升通信性能。
- 针对Atlas 训练系列产品，alltoallv的通信域需要满足如下约束：
集群组网下，单server 1p、2p通信域要在同一个cluster内（server内0-3卡和4-7卡各为一个cluster），单server4p、8p和多server通信域中rank要以cluster为基本单位，并且server间cluster选取要一致。
- 该接口不支持非集群场景下使用。

支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

8.2.3.8 HcclBarrier

功能说明

将指定通信域内所有rank的stream阻塞，直到所有rank都下发执行该操作为止。

函数原型

HcclResult HcclBarrier(HcclComm comm, aclrtStream stream)

参数说明

参数名	输入/输出	描述
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

无

支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

8.2.3.9 HcclScatter

功能说明

集合通信域Scatter操作接口，将root节点的数据均分并散布至其他rank。

函数原型

```
HcclResult HcclScatter(void *sendBuf, void *recvBuf, uint64_t recvCount,
HcclDataType dataType, uint32_t root, HcclComm comm, aclrtStream stream)
```

参数说明

参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
recvBuf	输出	目的数据buffer地址，集合通信结果输出至此buffer中。
recvCount	输入	参与scatter操作的recvBuf的数据个数，比如只有一个int32数据参与，则count=1。
dataType	输入	scatter操作的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
root	输入	作为scatter root的rank id。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- 所有rank的recvCount、dataType、root均应相同。
- 全局只能有1个root节点。
- 非root节点的sendBuf可以为空。root节点的sendBuf不能为空。

支持的型号

Atlas 训练系列产品

Atlas A2 训练系列产品

8.2.4 点对点通信

8.2.4.1 HcclSend

功能说明

集合通信域Send操作接口。将当前rank的sendBuf数据发送至destRank。

函数原型

```
HcclResult HcclSend(void* sendBuf, uint64_t count, HcclDataType dataType,
uint32_t destRank,HcclComm comm, aclrtStream stream)
```

参数说明

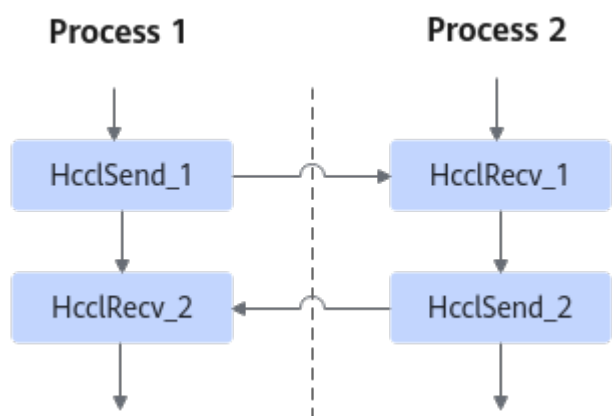
参数名	输入/输出	描述
sendBuf	输入	源数据buffer地址。
count	输入	发送数据的个数。
dataType	输入	发送数据的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64，bfp16。
destRank	输入	通信域内数据接收端的rank编号
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

[HcclResult](#)：接口成功返回HCCL_SUCCESS，其他失败。

约束说明

HcclSend与HcclRecv接口采用同步调用方式，且必须配对使用。即一个进程调用HcclSend接口后，需要等到与之配对的HcclRecv接口接收数据后，才可以进行下一个接口调用，如下图所示。



支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

调用示例

可参见[6.6 样例代码](#)中每种通信域初始化方式中的“HcclSend/HcclRecv操作代码样例”。

8.2.4.2 HcclRecv

功能说明

集合通信域Recv操作接口。从srcRank接收数据到当前rank的recvBuf。

函数原型

HcclResult HcclRecv(void* recvBuf, uint64_t count, HcclDataType dataType, uint32_t srcRank,HcclComm comm, aclrtStream stream)

参数说明

参数名	输入/输出	描述
recvBuf	输入	数据接收buffer地址。
count	输入	接收数据的个数。
dataType	输入	接收数据的数据类型， HcclDataType 类型。 针对Atlas 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持数据类型：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64，bfp16。

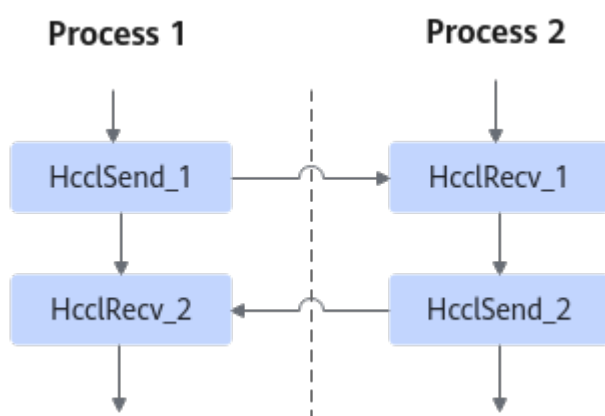
参数名	输入/输出	描述
srcRank	输入	通信域内数据发送端的rank编号
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

HcclSend与HcclRecv接口采用同步调用方式，且必须配对使用。即一个进程调用HcclSend接口后，需要等到与之配对的HcclRecv接口接收数据后，才可以进行下一个接口调用，如下图所示。



支持的型号

Atlas 训练系列产品

Atlas A2 训练系列产品

调用示例

可参见[6.6 样例代码](#)中每种通信域初始化方式中的“HcclSend/HcclRecv操作代码样例”。

8.2.4.3 HcclBatchSendRecv

功能说明

集合通信域异步批量点对点通信操作接口。调用一次接口可以完成本rank上的多个收发任务，本rank发送和接收之间是异步的，发送和接收任务之间不会相互阻塞。

函数原型

```
HcclResult HcclBatchSendRecv(HcclSendRecvItem* sendRecvInfo, uint32_t itemNum, HcclComm comm, aclrtStream stream);
```

参数说明

参数名	输入/输出	描述
sendRecvInfo	输入	本rank需要下发的收发任务列表的首地址。 HcclSendRecvItem类型，详细可参见 8.2.6.9 HcclSendRecvItem 。
itemNum	输入	本rank需要接收和发送的任务个数。
comm	输入	集合通信操作所在的通信域。
stream	输入	本rank所使用的stream。

返回值

HcclResult: 接口成功返回HCCL_SUCCESS，其他失败。

约束说明

- “异步”是指同一张卡上的接收和发送任务是异步的，不会相互阻塞。但是在卡间，收发任务依旧是同步的，因此，卡间的收发任务也同HcclSend、HcclRecv一样，必须是一一对应的。
- 任务列表中不能有重复的send/recv任务，重复指向（从）同一rank发送（接收）的两个任务。
- 当前版本此接口不支持Virtual Pipeline（VPP）开启的场景。
- 针对[Atlas 200T A2 Box16 异构子框](#)，若Server内卡间出现建链失败的情况（错误码：EI0010），需要将环境变量HCCL_INTRA_ROCE_ENABLE配置为1，HCCL_INTRA_PCIE_ENABLE配置为0，让Server内采用RoCE环路进行多卡间的通信（请确保Server上存在RoCE网卡，且具有send/recv收发关系的设备之间RDMA链路互通），环境变量配置示例如下：

```
export HCCL_INTRA_ROCE_ENABLE=1
export HCCL_INTRA_PCIE_ENABLE=0
```

支持的型号

- Atlas 训练系列产品
- Atlas A2 训练系列产品

8.2.5 异常处理

8.2.5.1 HcclGetCommAsyncError

功能说明

当集群信息中存在Device网口通信链路不稳定、出现网络拥塞的情况时，Device日志中会存在“error cqe”的打印，我们称这种错误为“RDMA ERROR CQE”错误。

当前版本，此接口仅支持查询通信域内是否存在“RDMA ERROR CQE”的错误。

 说明

此接口为同步接口，即接口调用后需要等待返回结果。

函数原型

HcclResult HcclGetCommAsyncError(HcclComm comm, HcclResult *asyncError)

参数说明

参数名	输入/输出	描述
comm	输入	通信域。
asyncError	输出	结果为0，表示该通信域内无错误发生；其他返回值，请参见HcclResult类型。

返回值

参见[HcclResult](#)类型，当前版本仅返回HCCL_E_REMOTE错误类型。

约束说明

- 建立通信域后，才可调用此接口。
- 通信域销毁后，不可调用此接口。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

8.2.5.2 HcclGetErrorString

功能说明

解析HCCL_RESULT类型的错误码。

函数原型

const char* HcclGetErrorString (HcclResult code)

参数说明

参数名	输入/输出	描述
code	输入	HCCL_RESULT类型错误码。

返回值

HcclResult类型错误码的解析结果。

约束说明

无。

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

8.2.6 数据类型

8.2.6.1 HcclResult

功能

定义集合通信相关操作的返回值。

定义原型

```
typedef enum {
    HCCL_SUCCESS = 0,           /* success */
    HCCL_E_PARA = 1,           /* parameter error */
    HCCL_E_PTR = 2,            /* empty pointer */
    HCCL_E_MEMORY = 3,         /* memory error */
    HCCL_E_INTERNAL = 4,       /* internal error */
    HCCL_E_NOT_SUPPORT = 5,     /* not support feature */
    HCCL_E_NOT_FOUND = 6,      /* not found specific resource */
    HCCL_E_UNAVAIL = 7,        /* resource unavailable */
    HCCL_E_SYSCALL = 8,        /* call system interface error */
    HCCL_E_TIMEOUT = 9,        /* timeout */
    HCCL_E_OPEN_FILE_FAILURE = 10, /* open file fail */
    HCCL_E_TCP_CONNECT = 11,    /* tcp connect fail */
    HCCL_E_ROCE_CONNECT = 12,   /* roce connect fail */
    HCCL_E_TCP_TRANSFER = 13,   /* tcp transfer fail */
    HCCL_E_ROCE_TRANSFER = 14,  /* roce transfer fail */
    HCCL_E_RUNTIME = 15,        /* call runtime api fail */
    HCCL_E_DRV = 16,            /* call driver api fail */
    HCCL_E_PROFILING = 17,      /* call profiling api fail */
    HCCL_E_CCE = 18,            /* call cce api fail */
    HCCL_E_NETWORK = 19,        /* call network api fail */
    HCCL_E_AGAIN = 20,          /* try again */
    HCCL_E_REMOTE = 21,         /* error cq */
    HCCL_E_SUSPENDING = 22,     /* error communicator suspending */
}
```

```
    HCCL_E_RESERVED          /* reserved */  
} HcclResult;
```

8.2.6.2 HcclReduceOp

功能

定义集合通信reduce操作的类型。

定义原型

```
typedef enum {  
    HCCL_REDUCE_SUM = 0,    /* sum */  
    HCCL_REDUCE_PROD = 1,   /* prod */  
    HCCL_REDUCE_MAX = 2,    /* max */  
    HCCL_REDUCE_MIN = 3,    /* min */  
    HCCL_REDUCE_RESERVED   /* reserved */  
} HcclReduceOp;
```

8.2.6.3 HcclDataType

功能

定义集合通信相关操作的数据类型。

定义原型

```
typedef enum {  
    HCCL_DATA_TYPE_INT8 = 0, /* int8 */  
    HCCL_DATA_TYPE_INT16 = 1, /* int16 */  
    HCCL_DATA_TYPE_INT32 = 2, /* int32 */  
    HCCL_DATA_TYPE_FP16 = 3, /* float16 */  
    HCCL_DATA_TYPE_FP32 = 4, /* float32 */  
    HCCL_DATA_TYPE_INT64 = 5, /* int64 */  
    HCCL_DATA_TYPE_UINT64 = 6, /* uint64 */  
    HCCL_DATA_TYPE_UINT8 = 7, /* uint8 */  
    HCCL_DATA_TYPE_UINT16 = 8, /* uint16 */  
    HCCL_DATA_TYPE_UINT32 = 9, /* uint32 */  
    HCCL_DATA_TYPE_FP64 = 10, /* fp64 */  
    HCCL_DATA_TYPE_BFP16 = 11, /* bfp16 */  
    HCCL_DATA_TYPE_INT128 = 12, /* int128 */  
    HCCL_DATA_TYPE_RESERVED /* reserved */  
} HcclDataType;
```

8.2.6.4 HcclConfig

功能

定义集合通信相关配置。

当前版本仅支持配置集合通信是否支持确定性计算。

定义原型

```
typedef enum {  
    HCCL_DETERMINISTIC = 0,  
    HCCL_CONFIG_RESERVED  
} HcclConfig;
```

8.2.6.5 HcclConfigValue

功能

定义**HcclConfig**中可配置的参数的值。

定义原型

当前版本仅支持配置HcclConfig中“HCCL_DETERMINISTIC”参数的值。

```
union HcclConfigValue {  
    int32_t value;    /* 0: 不支持确定性计算, 1: 支持确定性计算 */  
};
```

8.2.6.6 HcclRootInfo

功能

root节点的rank信息，主要包含root节点的host ip、host port以及root节点唯一标识（由device id和时间戳等信息拼接获得）。

定义原型

```
const uint32_t HCCL_ROOT_INFO_BYTES = 4108; // 4108: root info length  
typedef struct HcclRootInfoDef {  
    char internal[HCCL_ROOT_INFO_BYTES];  
} HcclRootInfo;
```

8.2.6.7 HcclComm

功能

指向当前通信域的句柄。

定义原型

```
typedef void *HcclComm;
```

8.2.6.8 HcclSendRecvType

功能

用于批量点对点通信操作，用来标识当前的任务类型是发送还是接收。

定义原型

```
typedef enum {  
    HCCL_SEND = 0,    /* 当前任务是发送任务 */  
    HCCL_RECV = 1,    /* 当前任务是接收任务 */  
    HCCL_SEND_RECV_RESERVED    /* 保留字段 */  
} HcclSendRecvType;
```

8.2.6.9 HcclSendRecvItem

功能

用于批量点对点通信操作，定义每一个通信任务的基本信息，包含任务类型、数据地址、数据个数、数据类型、对端rank编号等。

定义原型

```
typedef struct HcclSendRecvItemDef {  
    HcclSendRecvType sendRecvType; /* 标识当前任务类型是发送还是接收 */  
    void *buf; /* 数据发送/接收的buffer地址 */  
    uint64_t count; /* 发送/接收数据的个数 */  
    HcclDataType dataType; /* 发送/接收数据的数据类型 */  
    uint32_t remoteRank; /* 通信域内数据接收/发送端的rank编号 */  
} HcclSendRecvItem;
```

8.2.6.10 HcclCommConfig

功能

用于在使用HcclCommInitRootInfoConfig进行通信域初始化时，定义通信域配置信息，包含缓存区大小、确定性计算开关和通信域名称。

定义原型

```
const uint32_t HCCL_COMM_CONFIG_INFO_BYTES = 24;  
const uint32_t COMM_NAME_MAX_LENGTH = 128;  
typedef struct HcclCommConfigDef {  
    char reserved[HCCL_COMM_CONFIG_INFO_BYTES]; /* 保留字段，不可修改 */  
    uint32_t hcclBufferSize; /* 共享数据的缓存区大小，取值需大于等于1，单位为MByte，默认值为200 */  
    uint32_t hcclDeterministic; /* 确定性计算开关：0 关闭、1 开启，默认值为0 */  
    char hcclCommName[COMM_NAME_MAX_LENGTH]; /* 通信域名称，最大长度为128，不指定时由HCCL自动生成 */  
} HcclCommConfig;
```

8.2.6.11 HcclCommConfigCapability

功能

定义通信域初始化时支持的相关配置信息。

定义原型

```
typedef enum {  
    HCCL_COMM_CONFIG_BUFFER_SIZE = 0, /* 共享数据的缓存区大小 */  
    HCCL_COMM_CONFIG_DETERMINISTIC = 1, /* 确定性计算开关 */  
    HCCL_COMM_CONFIG_COMM_NAME = 2, /* 通信域名称 */  
    HCCL_COMM_CONFIG_RESERVED /* 预留字段 */  
} HcclCommConfigCapability;
```

8.2.7 预留接口

本章节列出的接口均为预留接口，后续有可能变更，不支持开发者使用，开发者无需关注。

- HcclResult HcclCommSuspend(HcclComm comm)

- HcclResult HcclCommResume(HcclComm comm)

8.3 HCCL API (Python)

8.3.1 接口列表

TF Adapter提供的分布式优化器NPUDistributedOptimizer和npudistributed_optimizer_wrapper可以让用户在不需要感知allreduce的情况下自动完成梯度聚合功能，实现数据并行训练方式。但为了能够同时满足用户灵活的使用方式，集合通信库HCCL提供了常用的rank管理、梯度切分功能、集合通信原型等接口。HCCL为TensorFlow网络提供的Python API如表8-2所示。

表 8-2 HCCL (Python) 接口列表

分类	接口	简介	定义文件
rank 管理	create_group	创建集合通信group。	\$ {install_path }/python/ site- packages/ hccl/ manage/ api.py
	destroy_group	销毁集合通信group。	
	get_rank_size	获取group内rank数量（即Device数量）。	
	get_local_rank_size	获取group内device所在服务器内的local rank数量。	
	get_rank_id	获取device在group中对应的rank序号。	
	get_local_rank_id	获取device在group中对应的local rank序号。	
	get_world_rank_from_group_rank	从group rank id，获取该进程对应的world rank id。	
	get_group_rank_from_world_rank	从world rank id，获取该进程在group中的group rank id。	

分类	接口	简介	定义文件
梯度切分	set_split_strategy_by_idx	基于梯度的索引id，在集合通信group内设置反向梯度切分策略。	\$ {install_path} }/python/ site- packages/ hccl/split/ api.py
	set_split_strategy_by_size	基于梯度数据量百分比，在集合通信group内设置反向梯度切分策略。	
集合通信	allreduce	提供group内的集合通信allreduce功能，对所有节点的同名张量进行约减。	\$ {install_path} }/python/ site- packages/ npu_bridge/ hccl/ hccl_ops.py
	allgather	提供group内的集合通信allgather功能，将所有节点的输入Tensor合并起来。	
	broadcast	提供group内的集合通信broadcast功能，将root节点的数据广播到其他rank。	
	reduce_scatter	提供group内的集合通信reducescatter功能。	
	send	提供group内点对点通信发送数据的send功能。	
	receive	提供group内点对点通信发送数据的receive功能。	
	reduce	提供group内的集合通信reduce功能，对所有节点的同名张量进行规约，并将数据输出至root节点。	
	alltoallv	集合通信域alltoallv操作接口。向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。	

分类	接口	简介	定义文件
	alltoallvc	集合通信域 alltoallvc操作接口。向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。 alltoallvc通过输入参数 send_count_matrix传入所有rank的收发参数，与alltoallv相比，性能更优。	

8.3.2 hccl.manage.api

8.3.2.1 create_group

功能说明

创建集合通信用户自定义group。

如果开发者不调用此接口创建用户自定义group，则默认将所有参与集群训练的设备创建为全局的hccl_world_group。

函数原型

```
def create_group(group, rank_num, rank_ids)
```

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，集合通信group的标识，不能为hccl_world_group。 hccl_world_group是默认group，由ranktable文件创建，不能通过该接口创建，如果用户传入的group如果是hccl_world_group，创建group失败。
rank_num	输入	int类型。 组成该group的rank数量。 最大值为32768。

参数名	输入/输出	描述
rank_ids	输入	list类型。 组成该group的world_rank_id列表。 在不同单板类型上，有不同的限制。 针对Atlas 训练系列产品： - 对于Server单机场景，rank_ids需满足如下条件： rank数量必须为1/2/4/8，0-3卡与4-7卡各为一个组网，rank数量为2/4时要求选取的昇腾AI处理器同属一个cluster。 - 对于Server集群场景，rank_ids满足如下条件： - 各Server要选取相同数量的rank（且数量要求为1/2/4/8）。 - 各Server选取rank数量为2/4时要求选取的昇腾AI处理器同属一个cluster（即rank id按8取模余数都小于4或都大于等于4）。 举例： 假设对三台Server创建group，三台Server的rank id分别为： {0,1,2,3,4,5,6,7} {8,9,10,11,12,13,14,15} {16,17,18,19,20,21,22,23} 则满足要求的rank_ids列表可以是： rank_ids=[1,9,17] rank_ids=[1,2,9,10,17,18] rank_ids=[4,5,6,7,12,13,14,15,20,21,22,23] 针对Atlas 300I Duo 推理卡： - 对于Server单机场景，rank_ids无限制条件。 - 对于Server集群场景，rank_ids需满足如下条件： 建议各Server要选取相同数量的rank（数量大小无要求），且各Server选取的rank对应位置要相等（即rank id按8取模相等）。若各Server选取的rank数量不同，会造成性能裂化。 举例： 假设对三台Server创建group，三台Server的rank id分别为： {0,1,2,3,4,5,6,7} {8,9,10,11,12,13,14,15} {16,17,18,19,20,21,22,23} 则满足要求的rank_ids列表可以是： rank_ids=[1,9,17] rank_ids=[1,2,9,10,17,18] rank_ids=[4,5,6,7,12,13,14,15,20,21,22,23]

参数名	输入/输出	描述
		针对Atlas A2 训练系列产品： - 对于Server单机场景，rank_ids无限制条件。 - 对于Server集群场景，rank_ids需满足如下条件： 建议各Server要选取相同数量的rank（数量大小无要求），且各Server选取的rank对应位置要相等（即rank id按8取模相等）。若各Server选取的rank数量不同，会造成性能裂化。 举例： 假设对三台Server创建group，三台Server的rank id分别为： {0,1,2,3,4,5,6,7} {8,9,10,11,12,13,14,15} {16,17,18,19,20,21,22,23} 则满足要求的rank_ids列表可以是： rank_ids=[1,9,17] rank_ids=[1,2,9,10,17,18] rank_ids=[4,5,6,7,12,13,14,15,20,21,22,23] 补充说明： 建议rank_ids按照Device物理连接顺序进行排序，即将物理连接上较近的device编排在一起。例如，若device_ip按照物理连接从小到大设置，则rank_ids也建议按照从小到大的顺序设置。

返回值

无。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 重复创建名称相同的group，会创建失败。

支持的型号

Atlas 训练系列产品

Atlas 300I Duo 推理卡

Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
create_group("myGroup", 4, [0, 1, 2, 3])
```

8.3.2.2 destroy_group

功能说明

销毁用户自定义group。

函数原型

def destroy_group(group)

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，集合通信group的标识。

返回值

无。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 相同名称的group，[8.3.2.2 destroy_group](#)和[8.3.2.1 create_group](#)配套使用，必须在create_group完成之后调用；
- 如果用户传入的group恰好是hccl_world_group（默认group），销毁group失败；

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
create_group("myGroup", 4, [0, 1, 2, 3])  
destroy_group("myGroup")
```

8.3.2.3 get_rank_size

功能说明

获取group内的rank数量（即Device数量）。

函数原型

```
def get_rank_size(group="hccl_world_group")
```

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group，如果用户不传，默认为"hccl_world_group"。

返回值

int类型，返回group内rank数量。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- [8.3.2.1 create_group](#)完成之后，调用此API获取本group的rank数量；
- 如果传入"hccl_world_group"，返回world_group的rank数量。

支持的型号

Atlas 训练系列产品

Atlas 300I Duo 推理卡

Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
create_group("myGroup", 4, [0, 1, 2, 3])  
rankSize = get_rank_size("myGroup")  
#rankSize = 4
```

8.3.2.4 get_local_rank_size

功能说明

获取group内device所在服务器内的local rank数量。

函数原型

```
def get_local_rank_size(group="hccl_world_group")
```

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group，如果用户不传，默认为"hccl_world_group"。

返回值

int类型，返回device所在服务器内的local rank数量。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- [8.3.2.1 create_group](#)完成之后，调用此API获取本group的local rank数量。
- 如果传入"hccl_world_group"，返回world_group的local rank数量。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
create_group("myGroup", 4, [0, 1, 2, 3])
localRankSize = get_local_rank_size("myGroup")
#localRankSize = 1
```

8.3.2.5 get_rank_id

功能说明

获取device在group中对应的rank序号。

函数原型

def get_rank_id(group="hccl_world_group")

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group，如果用户不传，默认为"hccl_world_group"。

返回值

int类型，返回device所在group的rank id。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- [8.3.2.1 create_group](#)完成之后，调用此API获取进程在group中的rank id。
- 如果传入"hccl_world_group"，返回进程在world_group的rank id。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
create_group("myGroup", 4, [0, 1, 2, 3])
rankId = get_rank_id("myGroup")
#rankId = 0/1/2/3
```

8.3.2.6 get_local_rank_id

功能说明

获取device在group中对应的local rank序号。

函数原型

def get_local_rank_id(group="hccl_world_group")

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group，如果用户不传，默认为"hccl_world_group"。

返回值

int类型，返回device所在服务器内的local rank id序号。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- [8.3.2.1 create_group](#)完成之后，调用此API获取进程在group中的local rank id。
- 如果传入"hccl_world_group"，返回进程在world_group的local rank id。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
create_group("myGroup", 4, [0, 1, 2, 3])
localRankId = get_local_rank_id("myGroup")
#rankId = 0
```

8.3.2.7 get_world_rank_from_group_rank

功能说明

从group rank id，获取该进程对应的world rank id。

函数原型

def get_world_rank_from_group_rank(group, group_rank_id)

参数说明

参数名	输入/输出	描述
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。
group_rank_id	输入	int类型。 进程在group中的rank id。

返回值

int类型，进程在"hccl_world_group"中的rank id。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- [8.3.2.1 create_group](#)完成之后，调用此API转换group rank id到world rank id。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
create_group("myGroup", 4, [0, 1, 2, 3])
worldRankId = get_world_rank_from_group_rank("myGroup", 1)
#worldRankId = 8
```

8.3.2.8 get_group_rank_from_world_rank

功能说明

从world rank id，获取该进程在group中的group rank id。

函数原型

```
def get_group_rank_from_world_rank(world_rank_id, group)
```

参数说明

参数名	输入/输出	描述
world_rank_id	输入	int类型。 进程在"hccl_world_group"中的rank id。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者 "hccl_world_group"。

返回值

int类型，正常返回进程在group中的rank id。

约束说明

- 必须在集合通信初始化完成之后调用。
- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- [8.3.2.1 create_group](#)完成之后，调用此API转换world rank id到group rank id。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
create_group("myGroup", 4, [0, 1, 2, 3])
groupRankId = get_group_rank_from_world_rank(8, "myGroup")
#groupRankId = 1
```

8.3.3 hccl.split.api

8.3.3.1 set_split_strategy_by_idx

功能说明

基于梯度的索引id，在集合通信group内设置反向梯度切分策略，实现allreduce的融合，用于进行集合通信的性能调优。

函数原型

def set_split_strategy_by_idx(idxList, group="hccl_world_group")

参数说明

参数名	输入/输出	描述
idxList	输入	list类型。 梯度的索引id列表。 - 梯度的索引id列表需为非负，升序序列。 - 梯度的索引id必须基于模型的总梯度参数个数去设置。索引id从0开始，最大值可通过以下方法获得： - 不调用梯度切分接口设置梯度切分策略进行训练，此时脚本会使用8.3.3.2 set_split_strategy_by_size中的默认梯度切分方式进行训练。 - 训练结束后，在INFO级别的host训练日志中搜索"segment result"关键字，可以得到梯度切分的分段的情况如: segment index list: [0,107] [108,159]。此分段序列中最大的数字（例如159）即总梯度参数索引最大的值。 说明 完整的训练过程可能出现日志覆盖情况，此时用户可以修改“/var/log/npu/conf/slog/slog.conf”中的配置项LogAgentMaxFileNum，提高Host侧保存日志文件的数量。或可以只进行一次迭代训练。 - 梯度的切分最多支持8段。 - 比如模型总共有160个参数会产生梯度，需要切分[0,20]、[21,100]和[101,159]三段，则可以设置为idxList=[20,100,159]。
group	输入	String类型。 group名称，可以为"hccl_world_group"或自定义group，默认为"hccl_world_group"。

返回值

无。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 若用户不调用梯度切分接口设置切分策略，则会按默认反向梯度切分策略切分。
默认切分策略：按梯度数据量切分为2段，第一段数据量为96.54%，第二段数据量为3.46%（部分情况可能出现为一段情况）。

支持的型号

Atlas 训练系列产品

Atlas 300I Duo 推理卡

Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
set_split_strategy_by_idx([20, 100, 159], "group")
```

8.3.3.2 set_split_strategy_by_size

功能说明

基于梯度数据量百分比，在集合通信group内设置反向梯度切分策略，实现allreduce的融合，用于进行集合通信的性能调优。

函数原型

```
def set_split_strategy_by_size(dataSizeList, group="hccl_world_group")
```

参数说明

参数名	输入/输出	描述
dataSizeList	输入	list类型。 梯度参数数据量百分比列表。 - 梯度的索引id列表需为非负，且梯度数据量序列总百分比之和必须为100。 - 梯度的切分最多支持8段。 - 比如模型总共有150M梯度数据量，需要切分90M，30M，30M三段，则可以设置dataSizeList =[60,20,20]。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为"hccl_world_group"或自定义group，默认为"hccl_world_group"。

返回值

无。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 在同时基于梯度数据量百分比及梯度的索引id设置反向梯度切分策略时，以基于梯度数据量百分比设置结果优先。
- 若用户不调用梯度切分接口设置切分策略，则会按默认反向梯度切分策略切分。
默认切分策略：ResNet50的最优切分位置，即按梯度数据量切分为2段，第一段数据量为96.54%，第二段数据量为3.46%。

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
set_split_strategy_by_size([60, 20, 20], "group")
```

8.3.4 npu_bridge.hccl.hccl_ops

8.3.4.1 allreduce

功能说明

集合通信算子AllReduce的操作接口，将group内所有节点的输入数据进行reduce操作后，再把结果发送到所有节点的输出buf，其中reduce操作类型由reduction参数指定。

函数原型

```
def allreduce(tensor, reduction, fusion=1, fusion_id=-1, group =  
"hccl_world_group")
```

参数说明

参数名	输入/输出	描述
tensor	输入	TensorFlow的tensor类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、int32、int64、float16、float32。 针对Atlas 300I Duo 推理卡，支持的数据类型为：int8、int16、int32、float16、float32。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、int16、int32、int64、float16、float32、bfp16。
reduction	输入	String类型。 reduce的op类型，可以为max、min、prod和sum。 说明 针对Atlas 300I Duo 推理卡，当前版本“prod”、“max”、“min”操作不支持int16数据类型。 针对Atlas A2 训练系列产品，当前版本“prod”操作不支持int16、bfp16数据类型。

参数名	输入/输出	描述
fusion	输入	int类型。 allreduce算子融合标识。 0：不融合，该allreduce算子不和其他allreduce算子融合。 1：按照梯度切分策略进行融合，默认为1。 2：按照相同fusion_id进行融合。
fusion_id	输入	allreduce算子的融合id。 对相同fusion_id的allreduce算子进行融合。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。

返回值

tensor：对输入tensor执行完allreduce操作之后的结果tensor。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 每个rank只能有一个输入。
- allreduce上游节点暂不支持variable算子。
- 该接口要求输入tensor的数据量不超过8GB。
- allreduce算子融合只支持reduction为sum类型的算子。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
tensor = tf.random_uniform((1, 3), minval=1, maxval=10, dtype=tf.float32)  
result = hccl_ops.allreduce(tensor, "sum")
```

8.3.4.2 allgather

功能说明

集合通信算子AllGather的操作接口，将通信域内所有节点的输入按照rank id重新排序，然后拼接起来，再将结果发送到所有节点的输出。

 说明

针对AllGather操作，每个节点都接收按照rank id重新排序后的数据集，即每个节点的全AllGather输出都是一样的。

函数原型

```
def allgather(tensor, rank_size, group = "hccl_world_group", fusion=0, fusion_id=-1)
```

参数说明

参数名	输入/输出	描述
tensor	输入	TensorFlow的tensor类型。 针对Atlas 训练系列产品，支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas 300I Duo 推理卡，支持数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
rank_size	输入	int类型。 group内device的数量。 最大值为32768。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。
fusion	输入	int类型。 allgather算子融合标识，支持以下取值： 0：标识网络编译时不会对该算子进行融合，即该allgather算子不和其他allgather算子融合。 2：网络编译时，会对allgather算子按照相同的fusion_id进行融合，即“fusion_id”相同的allgather算子之间会进行融合。 默认值为“0”。
fusion_id	输入	allgather算子的融合id。 对相同fusion_id的allgather算子进行融合。

返回值

tensor：对输入tensor执行完allgather操作之后的结果tensor。

约束说明

调用该接口的rank必须在当前接口入参group定义的范围
内，不在此范围内的rank调用该接口会失败。

支持的型号

- Atlas 训练系列产品
- Atlas 300I Duo 推理卡
- Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
rank_size = 2  
result = hccl_ops.allgather(tensor, rank_size)
```

8.3.4.3 broadcast

功能说明

集合通信算子Broadcast的操作接口，将通信域内root节点的数据广播到其他rank。

函数原型

```
def broadcast(tensor, root_rank, fusion=2,fusion_id=0,group =  
"hccl_world_group")
```

参数说明

参数名	输入/输出	描述
tensor	输入	TensorFlow的tensor类型，为list。 针对Atlas 训练系列产品，支持的数据类型为： int8、uint8、int16、uint16、int32、uint32、 int64、uint64、float16、float32、float64。 针对Atlas 300I Duo 推理卡，支持数据类型为： int8、uint8、int16、uint16、int32、uint32、 int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，支持的数据类型为： int8、uint8、int16、uint16、int32、uint32、 int64、uint64、float16、float32、float64、 bfp16。
root_rank	输入	int类型。 作为root节点的rank_id，该id是group内的rank id。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者 "hccl_world_group"。

参数名	输入/输出	描述
fusion	输入	int类型。 broadcast算子融合标识。 0：不融合，该broadcast算子不和其他broadcast算子融合。 2：按照相同fusion_id进行融合。默认为2。 - 其他值非法。
fusion_id	输入	broadcast算子的融合id。 对相同fusion_id的broadcast算子进行融合。

返回值

tensor：对输入tensor执行完broadcast操作之后的结果tensor。

约束说明

调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
root = 0
inputs = [tensor]
result = hccl_ops.broadcast(inputs, root)
```

8.3.4.4 reduce_scatter

功能说明

集合通信算子ReduceScatter的操作接口。将所有rank的输入相加（或其他操作）后，再把结果按照rank编号均匀分散的到各个rank的输出buffer，每个进程拿到其他进程1/ranksize份的数据进行归约操作。

函数原型

```
def reduce_scatter(tensor, reduction, rank_size, group = "hccl_world_group",
fusion=0, fusion_id=-1)
```

参数说明

参数名	输入/输出	描述
tensor	输入	TensorFlow的tensor类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、int32、int64、float16、float32。 针对Atlas 300I Duo 推理卡，支持的数据类型为：int8、int16、int32、float16、float32。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、int16、int32、int64、float16、float32、bfp16。 需要注意tensor的第一个维度的元素个数必需是rank size的整数倍。
reduction	输入	String类型。 reduce的op类型，可以为max、min、prod和sum。 说明 针对Atlas 300I Duo 推理卡，当前版本“prod”、“max”、“min”操作不支持int16数据类型。 针对Atlas A2 训练系列产品，当前版本“prod”操作不支持int16、bfp16数据类型。
rank_size	输入	int类型。 group内device的数量。 最大值：32768。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。
fusion	输入	int类型。 reducescatter算子融合标识，支持以下取值： 0：标识网络编译时不会对该算子进行融合，即该reducescatter算子不和其他reducescatter算子融合。 2：网络编译时，会对reducescatter算子按照相同的fusion_id进行融合，即“fusion_id”相同的reducescatter算子之间会进行融合。 默认值为“0”。
fusion_id	输入	reducescatter算子的融合id。 对相同fusion_id的reducescatter算子进行融合。

返回值

tensor：对输入tensor执行完reducescatter操作之后的结果tensor。建议返回tensor的数据量为32Byte对齐，否则会导致性能下降。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 该接口要求输入tensor的数据量不超过8GB。

支持的型号

Atlas 训练系列产品

Atlas 300I Duo 推理卡

Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
rank_size = 2
result = hccl_ops.reduce_scatter(tensor, "sum", rank_size)
```

8.3.4.5 reduce

功能说明

集合通信算子Reduce的操作接口，将所有rank的input相加（或其他操作）后，再把结果发送到root节点的输出buffer。

函数原型

def reduce(tensor, reduction, root_rank, fusion=0, fusion_id=-1, group="hccl_world_group")

参数说明

参数名	输入/输出	描述
tensor	输入	TensorFlow的tensor类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、int32、int64、float16、float32。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、int16、int32、int64、float16、float32、bfp16。
reduction	输入	String类型。 reduce的op类型，可以为max、min、prod和sum。 说明 针对Atlas A2 训练系列产品，当前版本“prod”操作不支持int16、bfp16数据类型。
root_rank	输入	int类型。 作为root节点的rank_id，该id是group内的rank id。

参数名	输入/输出	描述
fusion	输入	int类型。 reduce算子融合标识。 0：不融合，该reduce算子不和其他reduce算子融合。 2：按照相同fusion_id进行融合。
fusion_id	输入	reduce算子的融合id。 对相同fusion_id的reduce算子进行融合。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。

返回值

tensor：对输入tensor执行完reduce操作之后的结果tensor。

调用示例

```
from npu_bridge.npu_init import *  
result = hccl_ops.reduce(tensor, "sum", 0)
```

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 该接口要求输入tensor的数据量不超过8GB。
- reduce算子融合只支持reduction为sum类型的算子。

支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

8.3.4.6 send

功能说明

提供group内点对点通信发送数据的send功能。

函数原型

```
def send(tensor, sr_tag, dest_rank, group = "hccl_world_group")
```

参数说明

参数名	输入/输出	描述
tensor	输入	TensorFlow的tensor类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas 300I Duo 推理卡，支持数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
sr_tag	输入	int类型。 消息标签，相同sr_tag的send/recv对可以收发数据。
dest_rank	输入	int类型。 数据的目标节点，该rank是group中的rank id。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。

返回值

无。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 如果点对点收发数据的rank属于不同的AI Server，要保证昇腾AI处理器的物理id相同
- 要求send和receive必须配对使用，且和图中其他算子要有依赖关系。

支持的型号

Atlas 训练系列产品
Atlas 300I Duo 推理卡
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
sr_tag = 0
```

```
dest_rank = 1
hccl_ops.send(tensor, sr_tag, dest_rank)
```

8.3.4.7 receive

功能说明

提供group内点对点通信发送数据的receive功能。

函数原型

```
def receive(shape, data_type, sr_tag, src_rank, group = "hccl_world_group")
```

参数说明

参数名	输入/输出	描述
shape	输入	接收tensor的shape。
data_type	输入	接收数据的数据类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas 300I Duo 推理卡，支持数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。
sr_tag	输入	int类型。 消息标签，相同sr_tag的send/recv对可以收发数据。
src_rank	输入	int类型。 接收数据的源节点，该rank是group中的rank id。
group	输入	String类型，最大长度为128字节，含结束符。 group名称，可以为用户自定义group或者"hccl_world_group"。

返回值

tensor：进行receive操作之后接收到对端的结果tensor。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围，不在此范围内的rank调用该接口会失败。

- 点对点收发数据的rank要属于不同的Server，且昇腾AI处理器物理ID必须相同。
- 要求send和receive必须配对使用，且和图中其他算子要有依赖关系。

支持的型号

Atlas 训练系列产品

Atlas 300I Duo 推理卡

Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *
sr_tag = 0
src_rank = 0
tensor = hccl_ops.receive(tensor.shape, tensor.dtype, sr_tag, src_rank)
```

8.3.4.8 alltoallv

功能说明

集合通信域alltoallv操作接口。向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。

两个NPU之间共享数据的缓存区大小默认为200M，可通过环境变量HCCL_BUFFIZE进行调整，单位为M，取值需要大于等于1，默认值是200M。集合通信网络中，每一个集合通信域都会占用HCCL_BUFFSIZE大小的缓冲区，用户可以根据通信数据量与业务模型数据量的大小，适当调整HCCL_BUFFSIZE的大小，提升网络执行性能，例如：

```
export HCCL_BUFFSIZE=2048
```

函数原型

```
def all_to_all_v(send_data, send_counts, send_displacements, rcv_counts, rcv_displacements, group="hccl_world_group")
```

参数说明

参数名	输入/输出	描述
send_data	输入	待发送的数据。 TensorFlow的tensor类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64 。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。

参数名	输入/输出	描述
send_counts	输入	发送的数据量，send_counts[i]表示本rank发给rank i的数据个数，基本单位是send_data数据类型对应的字节数。 例：send_data的数据类型为int32，send_counts[0]=1,send_count[1]=2，表示本rank给rank0发送1个int32类型的数据，给rank1发送2个int32类型的数据。 TensorFlow的tensor类型，tensor支持的数据类型为int64。
send_displacements	输入	发送数据的偏移量，send_displacements[i] 表示本rank发送给rank i的数据块相对于send_data的偏移量，基本单位是send_data数据类型对应字节数。 例： - send_data的数据类型为int32。 - send_counts[0]=1,send_counts[1]=2 - send_displacements[0]=0,send_displacements[1]=1 则表示本rank给rank0发送send_data上的第1个int32类型的数据，给rank1发送send_data上第2个与第3个int32类型的数据。 TensorFlow的tensor类型，tensor支持的数据类型为int64。
recv_counts	输入	接收的数据量，recv_counts[i]表示本rank从rank i收到的数据量。使用方法与send_counts类似。 TensorFlow的tensor类型。tensor支持的数据类型为int64。
recv_displacements	输入	接收数据的偏移量，recv_displacements[i] 表示本rank发送给rank i数据块相对于recv_data的偏移量，基本单位是recv_data_type的字节数。使用方法与send_displacements类似。 TensorFlow的tensor类型。tensor支持的数据类型为int64。
group	输入	group名称，可以为用户自定义group或者"hccl_world_group"。 String类型，最大长度为128字节，含结束符。

返回值

recv_data：对输入tensor执行完all_to_all_v操作之后的结果tensor。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，不在此范围内的rank调用该接口会失败。
- 针对Atlas 训练系列产品，alltoallv的通信域需要满足如下约束：
集群组网下，单server 1p、2p通信域要在同一个cluster内（server内0-3卡和4-7卡各为一个cluster），单server4p、8p和多server通信域中rank要以cluster为基本单位，并且server间cluster选取要一致。

- alltoallv操作的性能与NPU之间共享数据的缓存区大小有关，当通信数据量超过缓存区大小时性能将出现明显下降。若业务中alltoallv通信数据量较大，建议通过配置环境变量HCCL_BUFFSIZE适当增大缓存区大小以提升通信性能。
- 该接口不支持非集群场景下使用。

支持的型号

Atlas 训练系列产品
Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
result = hccl_ops.all_to_all_v(send_data_tensor, send_counts_tensor, send_displacements_tensor,  
recv_counts_tensor, recv_displacements_tensor)
```

8.3.4.9 alltoallvc

功能说明

集合通信域alltoallvc操作接口。向通信域内所有rank发送数据（数据量可以定制），并从所有rank接收数据。

alltoallvc通过输入参数send_count_matrix传入所有rank的收发参数，与[8.3.4.8 alltoallv](#)相比，性能更优。

两个NPU之间共享数据的缓存区大小默认为200M，可通过环境变量HCCL_BUFFSIZE进行调整，单位为M，取值需要大于等于1，默认值是200M。集合通信网络中，每一个集合通信域都会占用HCCL_BUFFSIZE大小的缓冲区，用户可以根据通信数据量与业务模型数据量的大小，适当调整HCCL_BUFFSIZE的大小，提升网络执行性能，例如：
export HCCL_BUFFSIZE=2048

函数原型

```
def all_to_all_v_c(send_data, send_count_matrix, rank, fusion=0, fusion_id=-1,  
group="hccl_world_group")
```

参数说明

参数名	输入/输出	描述
send_data	输入	待发送的数据。 TensorFlow的tensor类型。 针对Atlas 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64。 针对Atlas A2 训练系列产品，tensor支持的数据类型为：int8、uint8、int16、uint16、int32、uint32、int64、uint64、float16、float32、float64、bfp16。

参数名	输入/输出	描述
send_count_matrix	输入	所有rank的收发参数，send_count_matrix[i][j]表示rank i发给rank j的数据量，基本单位是send_data_type的字节数。 例：send_data_type为int32，send_count_matrix[0][1]=1，表示rank0给rank1发送1个int32。 TensorFlow的tensor类型。tensor支持的数据类型为int64。
rank	输入	int类型。 本节点的rank id，该id是group内的rank id。
fusion	输入	int类型。 alltoallvc算子融合标识，支持以下取值： 0：标识网络编译时不会对该算子进行融合，即该alltoallvc算子不和其他alltoallvc算子融合。 2：网络编译时，会对alltoallvc算子按照相同的fusion_id进行融合，即“fusion_id”相同的alltoallvc算子之间会进行融合。 说明： “fusion_id”相同的alltoallvc算子之间融合有一定的前提，算子需要在相同的通信域内，并且算子发送数据的数据类型需要相同。 默认值为“0”。
fusion_id	输入	标识alltoallvc算子的融合id。 int类型。 开启alltoallvc算子融合功能的场景下，需要配置该参数，取值范围[0, 0x7fffffff]。
group	输入	group名称，可以为用户自定义group或者“hccl_world_group”。 String类型，最大长度为128字节，含结束符。

返回值

recv_data：对输入tensor执行完all_to_all_v_c操作之后的结果tensor。

约束说明

- 调用该接口的rank必须在当前接口入参group定义的范围内，输入的rank id有效且不重复，否则调用该接口会失败。
- 针对Atlas 训练系列产品，alltoallvc的通信域需要满足如下约束：
单server 1p、2p通信域要在同一个cluster内（server内0-3卡和4-7卡各为一个cluster），单server4p、8p和多server通信域中rank要以cluster为基本单位，并且server间cluster选取要一致。
- 各节点输入的send_count_matrix要保持一致。

4. alltoallvc操作的性能与NPU之间共享数据的缓存区大小有关，当通信数据量超过缓存区大小时性能将出现明显下降。若业务中alltoallvc通信数据量较大，建议通过配置环境变量HCCL_BUFFSIZE适当增大缓存区大小以提升通信性能。

支持的型号

Atlas 训练系列产品

Atlas A2 训练系列产品

调用示例

```
from npu_bridge.npu_init import *  
result = hccl_ops.all_to_all_v_c(send_data_tensor, send_count_matrix_tensor, rank_tensor)
```

9 FAQ

- HCCL常见问题总结
- HCCP常见问题总结
- HCCL Test常见问题总结

9.1 HCCL 常见问题总结

9.1.1 环境变量配置错误（EI0001）

问题现象

执行日志报错：**EI0001 "Environment variable [***] is invalid"**，其中“***”是报错的环境变量名称，报错信息示例如下图所示。

```
W ProcessGroupHCCL.cpp:630] Warning: The HCCL execution timeout 8000000ms is bigger than watchdog timeout 1800000ms which is set by init_process_group! The plot may not be recorded. (function ProcessGroupHCCL)
2024-04-20 19:42:34.028 [ERROR] [LLM] (llm_inference.py:28): gRPCClient: torch.mpu/csrc/distributed/ProcessGroupHCCL.cpp:954 HCCL function error: HcclGetRootInfo(hcclID), error code is 1
[ERROR] 2024-04-20 19:42:34 (PID:2090824, Device:0, RankID:0) ENRM2280 DIST call hccl api failed.
EI0001: 2024-04-20 19:42:34.029:677 Environment variable [HCCL_EXEC_TIMEOUT] is invalid. Reason: it should be a number greater than or equal to 1s and less than or equal to 17340s.
Possible Causes: The environment variable configuration is invalid.
Solution: Try again with valid environment variable configuration.
run network: llama2 failed
2024-04-20 19:42:34.028 [ERROR] [LLM] (llm_inference.py:68): Run model: llama2 failed
/home/anaconda3/envs/llama76_dynamo/lib/python3.8/tempfile.py:818: ResourceWarning: Implicitly cleaning up <TemporaryDirectory '/tmp/tmps3dtl_ah'>
warnings.warn(ResourceWarning)
/home/anaconda3/envs/llama76_dynamo/lib/python3.8/tempfile.py:818: ResourceWarning: Implicitly cleaning up <TemporaryDirectory '/tmp/tmp16du0049'>
warnings.warn(ResourceWarning)
/home/anaconda3/envs/llama76_dynamo/lib/python3.8/tempfile.py:818: ResourceWarning: Implicitly cleaning up <TemporaryDirectory '/tmp/tmp1vmb0b0e'>
warnings.warn(ResourceWarning)
/home/anaconda3/envs/llama76_dynamo/lib/python3.8/tempfile.py:818: ResourceWarning: Implicitly cleaning up <TemporaryDirectory '/tmp/tmpjfehbhh'>
```

原因分析

环境变量配置有问题，通常是参数超过可配置范围或在识别范围以外，报错信息关键字及含义如表9-1所示。

表 9-1 EI0001 报错信息关键字汇总

报错信息关键字	含义
RankIpFamily rank[] device ip family[] is not same with others[].	IPv4和IPv6混用。
HCCL_CONNECT_TIMEOUT it should be a number greater than or equal to 120s and less than or equal to 7200s	HCCL_CONNECT_TIMEOUT的取值不在允许范围内。

报错信息关键字	含义
HCCL_INTRA_PCIE_ENABLE or HCCL_INTRA_ROCE_ENABLE HCCL_INTRA_PCIE_ENABLE and HCCL_INTRA_ROCE_ENABLE cannot be both configured to 1	HCCL_INTRA_PCIE_ENABLE和 HCCL_INTRA_ROCE_ENABLE两配置互 斥，不能同时配置为1。
HCCL_WHITELIST_DISABLE It must be 0 or 1.	HCCL_WHITELIST_DISABLE的取值不在 允许范围内。
HCCL_WHITELIST_FILE Please check env config	HCCL_WHITELIST_FILE配置有问题，通 常为HCCL通信白名单配置文件内容异 常，或者文件不存在。
HCCL_IF_IP it should be ip[]	HCCL_IF_IP配置的IP格式不正确。
HCCL_SOCKET_IFNAME Please check env config	HCCL_SOCKET_IFNAME配置格式不正 确，请确认“，”位置。
HCCL_SOCKET_FAMILY it should be AF_INET or AF_INET6	HCCL_SOCKET_FAMILY配置参数不正 确，需要是AF_INET或者AF_INET6。
HCCL_IF_BASE_PORT Value range[0,65520]	HCCL_IF_BASE_PORT的取值不在允许范 围内。
HCCL_ALGO expect: levelX:algo1;levelY:algo2	HCCL_ALGO配置错误，通常为格式不 符合要求、长度异常或内容不符合预期 （重复配置、字段不正确）。
HCCL_RDMA_TC Value range[0, 255], Must be a multiple of 4	HCCL_RDMA_TC配置错误，通常为数值 超范围、非数字、长度过长等。
HCCL_RDMA_SL Value range[0, 7]	HCCL_RDMA_SL的取值不在允许范围 内。
HCCL_RDMA_TIMEOUT Value range[5, 24]	HCCL_RDMA_TIMEOUT的取值不在允许 范围内。
HCCL_RDMA_RETRY_CNT Value range[1, 7]	HCCL_RDMA_RETRY_CNT的取值不在允 许范围内。
HCCL_BUFFSIZE Value should be equal to or greater than 1(MB).	HCCL_BUFFSIZE的取值不在允许范围 内。
HCCL_DETERMINISTIC Value should be true or false.	HCCL_DETERMINISTIC的取值不在允许 范围内。
HCCL_ENTRY_LOG_ENABLE It must be 0 or 1.	HCCL_ENTRY_LOG_ENABLE的取值不在 允许范围内。
HCCL_INTER_HCCS_DISABLE Value should be true or false.	HCCL_INTER_HCCS_DISABLE的取值不 在允许范围内。
HCCL_OP_EXPANSION_MODE it should be "AI_CPU"	HCCL_OP_EXPANSION_MODE配置只能 填AI_CPU

报错信息关键字	含义
HCCL_EXEC_TIMEOUT it should be a number greater than or equal to 0s and less than or equal to	HCCL_EXEC_TIMEOUT配置不在取值范围内
CM_CHIEF_IP it should be an available ip.	CM_CHIEF_IP配置的IP不可用。
CM_CHIEF_PORT it should be a unsigned number less than the max port num	CM_CHIEF_PORT的取值不在允许范围内。
CM_CHIEF_DEVICE it should be a unsigned number less than the max device num	CM_CHIEF_DEVICE的取值不在允许范围内。
CM_WORKER_IP it should be an available ip.	CM_WORKER_IP配置的IP不可用。
HCCL_WHITELIST_FILE HCCL_WHITELIST_DISABLE is [0] but HCCL_WHITELIST_FILE is not set	HCCL_WHITELIST_DISABLE配置为0， HCCL_WHITELIST_FILE却没有设置。
HCCL_WHITELIST_FILE hccl whitelist load config file[] failed.	HCCL_WHITELIST_FILE参数指定的文件 打开失败，请确认路径是否正确。

解决方法

确认报错提示的“环境变量”配置是否正确，并参见表9-1的报错信息进行修改。

9.1.2 执行通信操作超时（EI0002）

问题现象

常见于算子执行阶段，屏显日志报错信息“EI0002: The wait execution of the Notify register times out.”

```
Error Message is :
EI0002: The wait execution of the Notify register times out. Reason: The Notify register has not received the Notify record from remote rank [4].base information: [streamID:[14], taskID:[6], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_0].] task information: [notify_id:[0x0000000070000053], stage:[FFFFFFF], remote_rank:[4]
there are(is) 1 abnormal device(s):
serverId[192.168.100.111], deviceId[4], Heartbeat Lost Occurred, Possible Reason: 1. Process has exited, 2. Network Disconnected
}
Possible Cause: 1. An exception occurs during the execution on some NPUs in the cluster. As a result, collective communication operation failed.2. The execution speed on some NPU in the cluster is too slow to complete a communication operation within the timeout interval. (default 600s, You can set the interval by using HCCL_EXEC_TIMEOUT.).3. The number of training samples of each NPU is inconsistent.4. Packet loss or other connectivity problems occur on the communication link.
Solution: 1. If this error is reported on part of these ranks, check other ranks to see whether other errors have been reported earlier.2. If this error is reported for all ranks, check whether the HCCL reporting time is consistent (the maximum difference must not exceed 600s). If not, locate the cause or adjust the HCCL_EXEC_TIMEOUT environment variable to a larger value.3. Check whether the completion queue element (CQE) of the error exists in the plog(grep -m "error cqe"). If so, check the network connection status. (For details, see the TLS command and HCN connectivity check examples.)4. Ensure that the number of training samples of each NPU is consistent.
[drv api hal@qcom@thevcs failed: deviceId=7, taskId=0, type=2, drvRetCode=16{FUNC:logic@ReportV2}][FILE:npd_driver.cc][LINE:803]
Notify wait execute failed, device_id=7, stream_id=16, flip_num=0, notify_id=11{FUNC:Gdetrail}[FILE:stream.cc][LINE:838]
rtStreamSynchronize execute failed, reason=[the model stream execute failed][FUNC:func@rtStreamSynchronize][FILE:error_message_manage.cc][LINE:45]
```

plog日志中查询Notify，有如下类似信息：

```
[root@localhost ~]# grep -m 10000 | grep Notify
log-218074_20220215165540039.log:44901:[ERROR] RUNTIME(2138674.pythons):2022-02-15 17:08:59.589.252 [task.cc:2904][2168866] PrintErrorInfo:[EXEC][EXEC]Notify wait execute failed, device_id=0, stream_id=0, task_id=0, flip_num=0, notify_id=0.
log-218074_20220215165540039.log:44902:[ERROR] HCCL(2138674.pythons):2022-02-15 17:08:59.590.472 [task_exception_handler.cc:104][2138674][2168866][EXEC][EXEC][TaskExceptionHandler][callback]Task failed base information is streamID:[0], taskID:[4], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_12].
log-218086_2022021516552944.log:10259:[ERROR] RUNTIME(2138692.pythons):2022-02-15 17:08:59.635.785 [task.cc:2904][2168866] PrintErrorInfo:[EXEC][EXEC]Notify wait execute failed, device_id=0, stream_id=12, task_id=0, flip_num=0, notify_id=0.
log-218086_2022021516552944.log:10260:[ERROR] HCCL(2138692.pythons):2022-02-15 17:08:59.636.904 [task_exception_handler.cc:104][2138692][2168866][EXEC][EXEC][TaskExceptionHandler][callback]Task failed base information is streamID:[12], taskID:[13], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_12].
log-218086_2022021516552944.log:10261:[ERROR] RUNTIME(2138695.pythons):2022-02-15 17:08:59.626.992 [task.cc:2904][2168865] PrintErrorInfo:[EXEC][EXEC]Notify wait execute failed, device_id=0, stream_id=12, task_id=20, flip_num=0, notify_id=0.
log-218086_2022021516552944.log:10262:[ERROR] HCCL(2138695.pythons):2022-02-15 17:08:59.628.108 [task_exception_handler.cc:104][2138695][2168865][EXEC][EXEC][TaskExceptionHandler][callback]Task failed base information is streamID:[12], taskID:[20], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_12].
log-218092_20220215164014081.log:130245:[ERROR] RUNTIME(2138927.pythons):2022-02-15 17:08:59.624.806 [task.cc:2904][2168863] PrintErrorInfo:[EXEC][EXEC]Notify wait execute failed, device_id=0, stream_id=16, task_id=11, flip_num=0, notify_id=0.
log-218092_20220215164014081.log:130254:[ERROR] HCCL(2138927.pythons):2022-02-15 17:08:59.625.987 [task_exception_handler.cc:104][2138927][2168864][EXEC][EXEC][TaskExceptionHandler][callback]Task failed base information is streamID:[0], taskID:[11], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_12].
log-218092_20220215164014081.log:13043:[ERROR] RUNTIME(2138680.pythons):2022-02-15 17:08:59.627.607 [task.cc:2904][2168863] PrintErrorInfo:[EXEC][EXEC]Notify wait execute failed, device_id=0, stream_id=16, task_id=4, flip_num=0, notify_id=0.
log-218092_20220215164014081.log:13052:[ERROR] HCCL(2138680.pythons):2022-02-15 17:08:59.628.743 [task_exception_handler.cc:104][2138680][2168863][EXEC][EXEC][TaskExceptionHandler][callback]Task failed base information is streamID:[0], taskID:[4], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_12].
log-218093_20220215165539789.log:44901:[ERROR] HCCL(2139031.pythons):2022-02-15 17:08:59.629.950 [task_exception_handler.cc:104][2139031][2168867][EXEC][EXEC][TaskExceptionHandler][callback]Task failed base information is streamID:[13], taskID:[13], taskType[NotifyWait], tag[hccal@reducer_6029421139219749105_12].
log-218093_20220215165539789.log:44902:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.658.400 [engine.cc:967][2168862] ReportExecProc:[EXEC][EXEC]Task exception device_id=2, stream_id=1, task_id=3, type=1, retCode=0x01, info_len=25, msg:info:IteratorV2(IteratorGetNext), extend_info:info:task_id=4, info_len=25, msg:info:IteratorV2(IteratorGetNext).
log-218093_20220215165539789.log:17458:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.660.318 [task.cc:2748][2168862] ReportErrorInfo:[EXEC][EXEC]model execute error, retCode=0x01, [the model stream execute failed].
log-218093_20220215165539789.log:17461:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.660.322 [task.cc:2702][2168862] PrintErrorInfo:[EXEC][EXEC]model execute task failed, device_id=2, model task_id=3, flip_num=0, notify_id=0.
log-218093_20220215165539789.log:17463:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.661.560 [stream.cc:730][2168862] GetError:[EXEC][EXEC]Stream Synchronize failed, stream_id=1, retCode=0x01, [the model stream execute failed].
log-218093_20220215165539789.log:17467:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.661.631 [logger.cc:281][2168862] StreamSynchronize:[EXEC][EXEC]Stream Synchronize failed, stream = &rtStreamS2[26].
log-218093_20220215165539789.log:17459:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.661.655 [api.cc:610][2168862] rtStreamSynchronize:[EXEC][EXEC]rtStreamSynchronize failed, desc=[the model stream execute failed], innerCode=0x7150050.
log-218093_20220215165539789.log:17466:[ERROR] RUNTIME(2138686.pythons):2022-02-15 16:59:14.661.663 [error_message_manage.cc:42][216862] func@rtStreamSynchronize:[EXEC][EXEC]rtStreamSynchronize execute failed, reason=[the model stream execute failed].
```

原因分析

HCCL算子的task会在指定集群的每个Device上执行，并通过notify进行状态同步，若任何一个rank或者通信链路在执行前/中发生异常，则会导致集群同步失败，剩余卡会出现notify wait timeout。常见的原因主要有：

1. 部分卡被某些耗时较长的任务阻塞，在超过设置的执行同步等待时间（可通过环境变量HCCL_EXEC_TIMEOUT配置）后才继续往下执行。
2. 部分rank未执行到notify同步阶段。
3. 网络模型等原因导致某些rank间的task执行序列不一致。
4. 节点间通信链路不稳定。
5. 训练场景下，用户设置的HCCL_EXEC_TIMEOUT小于HCCL_CONNECT_TIMEOUT时，建链失败，先报notify wait超时。

解决方法

收集所有卡的plog日志后，按以下步骤排查：

- 步骤1** 检查所有卡的报错情况，若有卡未报notify超时错误，请通过训练或者plog日志检查此卡是否存在业务进程报错退出、卡死或core宕机的情况。
- 步骤2** 若所有卡均上报notify超时错误，则检查错误日志中最早报错和最晚报错时间差是否超过算子执行超时阈值，若超过阈值请定位报错时间最晚的rank执行阻塞原因（如save checkpoint）或通过export HCCL_EXEC_TIMEOUT=3600（示例，请根据需要配置）调整超时阈值。
- 步骤3** 检查集群中是否存在Device网口通信链路不稳定的情况，搜索所有卡的Device侧日志，若存在error cq的打印，时间位于业务区间内，这种情况可能是发生网络拥塞，需要排查交换机配置是否合理、训练过程中是否有网口link down等情况。

搜索命令样例：grep -rn "err cq" | grep HCCL

```
19987:[ERROR] HCCL(85111.python3):2023-02-18 09:44:55.431.692 [heartbeat.cc:547][85111][94635][Heartbeat]cq err status[12], time:[2023-02-18 09:44:55.369944], ip:[192.0.1.7]
```

----结束

9.1.3 集合通信操作参数无效（EI0003）

问题现象

执行日志报错：EI0003 "value [***] for parameter [***] is invalid"，如下所示。

```
custom_group :None
test_type :all
dtype :float32
data :1024
iter :1
timeout : None
profiling :false
staged :false
pid :1036690
[2024-04-24 06:30:56.081654: F ge_plugin.cc:338] [GePlugin] Initialize ge failed, ret : failed
Error Messageis:
EI0003: 2024-04-24-06:30:52.388.386 In [HcomInitByFile], value [0] for parameter [rankTablePath] is invalid. Reason: The collective communication operator has an invalid argument. Reason[0]
Solution: Try again with a valid argument.
TraceBack (most recent call last):
PluginManager InvokeAll failed.[FUNC:Initialize] [FILE:ops_kernel_manager.cc][LINE :89]
OpsManager initialize failed.[FUNC:InnerInitialize] [FILE:gelib.cc] [LINE:241]
```



```
GELib::InnerInitialize failed.[FUNC :Initialize][FILE:gelib.cc][LINE:169]  
GEInitialize failed. [FUNC:GEInitialize] [FILE:ge_api. cc][LINE :307]
```

原因分析

该报错常见于输入数据校验阶段，通常可能有以下几种原因：

1. 接口入参为指针，但传入指针为空。
2. 接口传入的参数超过预期枚举值范围。
3. 接口传入的字符串无效，或长度不符合预期。
4. 接口传入的路径无效，或者路径文件不符合预期。

具体报错字段及报错原因可以参考日志提示信息“value[]”和“parameter[]”进行确认。

解决方法

针对日志提示字段“value[]”和“parameter[]”进行排查，排查原因见上述[原因分析](#)。

9.1.4 无效的 RankTable 配置（EI00004）

问题现象

执行日志报错：EI0004 “The ranktable or rank is invalid”，如下所示。

```
custom_group :None  
dtype :float32  
data :1  
iter :1  
profiling :false  
pid :1040540  
[2024-04-24 06:31:38.087571: F ge_plugin. cc:338] [GePlugin] Initialize ge failed, ret :failed  
Error Message is:  
EI0004: 2024-04-24-06:31:33.915.634 The ranktable or rank is invalid, Reason:[The ip in ranktable is not  
a valid ip address]. Please check the configured ranktable. [The ranktable path configured in the training  
can be found in the plogs.]  
Solution: Try again with a valid cluster configuration in the ranktable file. Ensure that the  
configuration matches the operating environment.  
TraceBack (most recent call last):  
PluginManager InvokeAll failed. [FUNC:Initialize][FILE:ops_kernel_manager. cc][LINE :89]  
OpsManager initialize failed. [FUNC:InnerInitialize][FILE:gelib. cc] [LINE :241]  
GELib::InnerInitialize failed. [FUNC:Initialize] [FILE:gelib.cc][LINE:169]  
GEInitialize failed.[FUNC:GEInitialize][FILE:ge_api. cc][LINE :307]
```

原因分析

该报错常见于rank id或ranktable的数据校验阶段，通常有以下几种原因：

1. rank id的值不符合预期（过大的非法值、非数值、超过ranktable中的rank个数），或者同Server内已有rank id重复。
2. ranktable的格式或参数错误：版本不对，路径不对，格式不对，起始rank不为0，Server内的rank个数不正确，server_count、server_list、super_pod_id、server_id、server_index、ip等参数为空或配置不正确。

解决方法

针对执行日志提示字段中的“Reason”，可以基本看出问题方向，基于此排查配置下发参数的rank id或者ranktable，即可确认问题点。

9.1.5 卡间集合通信参数不一致（EI0005）

问题现象

执行日志报错：EI0005 “The arguments for collective communication are inconsistent between ranks”，如下所示。

```
custom_group :None
test_type : sum
dtype :float32
data :1024
fusion :0
fusion_id :1
fusion_num :2
iter :1
profiling :false
para_err_type :1
pid :1052803
ranksize is 8, rankid is 4.
time start: 2024-04-24 06:32:20.702705
ERROR : GeOp3_OGEOP::DoRunAsync Failed
Error Message is:
EI0005: 2024-04-24-06:32:27.781.599 The arguments for collective communication are inconsistent
between ranks:tag[HcomAllReduce_6629421139219749105_0], parameter[count], local[16512],
remote [8320]
    Solution: Check whether the training script and ranktable of each NPU are consistent.
    TraceBack (most recent call last):
    Transport init error. Reason: [Create] [DestLink]Create Dest error! creakLink para:rank[5]-
localUsererrank[4]-localIpAddr[192.1.1. 243], dst_rank[6]-remoteUsererrank[7]-remote_ip_addr[192.1.1.243]
    Transport init error. Reason: [Create] [DestLink]Create Dest error! creakLink para:rank[5]-
localUsererrank[4]-localIpAddr[192.1.1. 243], dst_rank[4]-remoteUsererrank[5]-remote_ip_addr[192.1.1.243]
    call hccl op:HcomAllReduce(HcomAllReduce) load task fail[FUNC:Distribute][FILE:hccl_task_info.cc]
[LINE:329]
[[[{{node GeOp3_0}}]]]
train fail
pid is 1052803
```

原因分析

该报错是本端和对端的校验参数不一致导致，问题一般出现在建链阶段。建链阶段本端接收到对端发过来的校验帧，然后和本端数据进行对比较验，确认数据是否一致。

通常原因是上层框架或用户调用传参有问题（取决于HCCL上层为框架还是用户直调），需要结合报错提示的“tag”确定出错算子，“parameter”确定出错算子参数。

解决方法

- 若出错的集合通信算子为用户直调场景，请根据错误信息中的“parameter”字段，确认算子接口入参的正确性。
- 若是HCCL上层框架调用场景，您可以获取日志后单击[Link](#)联系技术支持。

9.1.6 建链接超时（EI0006）

问题现象

常见于算子加载阶段，有以下2种情况：

- Server间的建链超时现象，出现“LINK_ERROR_INFO”报错信息，如下所示：

```
[ERROR] HCCL(1473737.python3):2022-09-29 15:31:05.319.815 [comm_base.cc:934] [1473737][1482272][LOAD][LOAD] LINK_ERROR_INFO
[ERROR] HCCL(1473737.python3):2022-09-29 15:31:05.319.827 [comm_base.cc:936] [1473737][1482272][LOAD][LOAD] comm_error_device[0] num[]
[ERROR] HCCL(1473737.python3):2022-09-29 15:31:05.319.831 [comm_base.cc:937] [1473737][1482272][LOAD][LOAD] dest_ip(rank_id) src_ip(rank_id) Role Status
[ERROR] HCCL(1473737.python3):2022-09-29 15:31:05.319.848 [comm_base.cc:962] [1473737][1482272][LOAD][LOAD] 192.2.1.28(14) 192.2.1.10(6) server no connect
[ERROR] HCCL(1473737.python3):2022-09-29 15:31:05.319.853 [comm_base.cc:950] [1473737][1482272][LOAD][LOAD] the connection failure between this device and target device may be due to the following reasons:
[ERROR] HCCL(1473737.python3):2022-09-29 15:31:05.319.856 [comm_base.cc:951] [1473737][1482272][LOAD][LOAD] the connection between this device and the target device is abnormal.
```

- Server内的建链超时现象，日志报错信息如下所示：

```
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.793 [exchanger_network.cc:695] [1869739][1880385][LOAD][LOAD] device[0] userrank[0] exchanger Status: run_step[]
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.796 [exchanger_network.cc:696] [1869739][1880385][LOAD][LOAD] dest_dev userrank Role connStatus
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.803 [exchanger_network.cc:697] [1869739][1880385][LOAD][LOAD] 0 0 NA NA
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.806 [exchanger_network.cc:716] [1869739][1880385][LOAD][LOAD] 1 1 server NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.810 [exchanger_network.cc:716] [1869739][1880385][LOAD][LOAD] 2 2 client NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.814 [exchanger_network.cc:716] [1869739][1880385][LOAD][LOAD] 3 3 server NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.817 [exchanger_network.cc:716] [1869739][1880385][LOAD][LOAD] 4 4 client NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.823 [exchanger_network.cc:716] [1869739][1880385][LOAD][LOAD] 5 5 server NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.827 [exchanger_network.cc:716] [1869739][1880385][LOAD][LOAD] 6 6 client NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.830 [exchanger_network.cc:721] [1869739][1880385][LOAD][LOAD] 7 7 server NO
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.838 [exchanger_network.cc:722] [1869739][1880385][LOAD][LOAD] the connection failure between this device and the target device may be due to the following reasons:
[ERROR] HCCL(1869739.python3):2022-09-27 15:57:37.497.841 [exchanger_network.cc:723] [1869739][1880385][LOAD][LOAD] the connection between this device and the target device is abnormal.
```

原因分析

HCCL会在指定集群的每个Device上运行，并在集群间建立socket链接，若任一个rank或者通信链路在建链前/中发生异常，则会导致集群建链失败。常见的原因包括：

- 部分rank未执行到正确的建链阶段。
- 部分卡被某些耗时较长的任务阻塞，在超过socket建链超时等待时间（可通过HCCL_CONNECT_TIMEOUT配置）后才执行到对应阶段。
- 脚本等原因导致某些rank间的通信算子数量或者排序不一致。
- 节点间通信链路不通或者不稳定。

解决方法

收集所有卡的plog日志后，按以下步骤排查：

- 步骤1** 检查所有卡的报错情况，若有卡未报建链超时错误，请通过训练或者plog日志检查此卡是否存在业务卡死情况，若存在请定位根因。
- 步骤2** 若所有卡均上报建链超时错误，则检查错误日志中最早和最晚时间差异是否超过超时阈值，若超过阈值请定位最晚报错的卡执行较慢的原因或通过增大建链超时时间尝试解决（例如通过配置export HCCL_CONNECT_TIMEOUT=3600，请根据需要调整超时阈值）。
- 步骤3** 若时间差异未超过阈值，请检查各rank执行的集合通信算子数量是否一致，算子加载的顺序是否一致。HCCL要求一个通信域内各个rank上算子加载顺序保持一致。

如需查询tag和计算图中node name的对应关系，需开启INFO日志，在Host日志中搜索：

关键字1：GenerateOpTag:graph

关键字2：GenerateTask:graph

关键字1可以看到node的hash值，关键字2可以看到nodHash对应的NodeName，协助计算图排查。

- 步骤4** 检查集群中是否存在Device网口通信链路不通的情况，比较常见的原因：

1. IP不在同一网段或子网掩码配置存在问题。
2. IP冲突，集群中存在IP相同的两个rank。
3. 交换机配置不在同一个vlan。
4. 链路不通。
5. 各rank的TLS(安全增强)设置不一致时也会导致建链失败。

可通过hccn tool命令来确认TLS是否一致，如不一致请参考随产品发布的《HCCN Tool接口参考》进行TLS配置。

《HCCN Tool接口参考》的获取方式为：单击[Link](#)进入企业业务网站的“昇腾计算文档中心”，然后在“中心训练硬件”栏选择对应的硬件型号，单击进入对应的文档页面，即可在其中找到配套版本的《HCCN Tool接口参考》。

查询TLS状态命令：

```
hccn_tool -i 0 -tls -g
hccn_tool -i 1 -tls -g
hccn_tool -i 2 -tls -g
hccn_tool -i 3 -tls -g
hccn_tool -i 4 -tls -g
hccn_tool -i 5 -tls -g
hccn_tool -i 6 -tls -g
hccn_tool -i 7 -tls -g
```

TLS switch值为0表示关闭，1表示开启。如果提示no certificate found，也表示TLS功能关闭。如果各个rank的TLS情况不一致，可使用hccn_tool工具配置TLS，或者关闭所有rank的TLS功能。

关闭TLS功能命令：

```
hccn_tool -i 0 -tls -s enable 0
hccn_tool -i 1 -tls -s enable 0
hccn_tool -i 2 -tls -s enable 0
hccn_tool -i 3 -tls -s enable 0
hccn_tool -i 4 -tls -s enable 0
hccn_tool -i 5 -tls -s enable 0
hccn_tool -i 6 -tls -s enable 0
hccn_tool -i 7 -tls -s enable 0
```

----结束

9.1.7 CANN 版本不一致，HCCL 返回错误码 EI0008

问题现象

常见于通信建链阶段，屏显日志报错：EI0008: The CANN versions are inconsistent。

```
ERROR : GeOp5_0GEOp::DoRunAsync Failed
Error Message is :
EI0008: The CANN versions are inconsistent: tag [HcomAllReduce_6629421139219749105_0], local_version [1.83.T10.0.B206], remote_version [1.83.T10.0.B207]
Solution: Install the same CANN version.
Traceback (most recent call last):
Call ops_kernel_info_store LoadTask fail[FUNC:Distribute][FILE:hccn_task_info.cc][LINE:213]
```

原因分析

集合通信建链时会校验本端与对端rank的CANN版本一致性，如果CANN版本不一致，HCCL会返回错误并打印错误码EI0008。

解决方法

确认不同服务器上安装的CANN软件版本是否一致，若不一致则需要安装一致的版本。

步骤1 在plog日志（INFO级别）中grep关键字CannVersion，确认各节点的CANN软件版本。

步骤2 重新安装一致的CANN软件版本。

----结束

9.2 HCCP 常见问题总结

9.2.1 EJ0001 打屏报错

问题现象

plog日志中，TDT首报错为：[DeviceMsgProcess][tid:1241254] [TsdClient] DeviceMsgProc errcode[EJ0001]

```
[ERROR] TDT(685010,all_reduce_test):2023-11-29-11:55:41.334.702 [process_mode_manager.cpp:587]
[DeviceMsgProcess][tid:685010] [TsdClient] DeviceMsgProc errcode[EJ0001]
[ERROR] TDT(685010,all_reduce_test):2023-11-29-11:55:41.334.873 [process_mode_manager.cpp:269]
[WaitRsp][tid:685010] tsd client wait response fail, device response code[1]. unknown device error.
[ERROR] TDT(685010,all_reduce_test):2023-11-29-11:55:41.334.893 [process_mode_manager.cpp:123]
[OpenProcess][tid:685010] Wait open response from device failed.
[ERROR] TDT(685010,all_reduce_test):2023-11-29-11:55:41.334.897 [tsd_client.cpp:31][TsdOpen]
[tid:685010] TsdOpen failed, deviceId[4].
```

EJ0001错误只能说明拉起device HCCP进程失败，具体失败原因需要根据device报错进一步区分，在debug目录下，使用/usr/local/Ascend/driver/tools/msnpureport -f导出device日志，grep -rn ERROR * | grep HCCP查看HCCP首报错，Device报错有以下两种场景。

- Device报错 “Create Server failed, ret(61)”
EJ0001报错，通常看到HCCP报错：[ra_adp.c:14c_server_init(1425) : Create Server failed, ret(61)]
[ERROR] HCCP(13722,hccp_service.bin):2023-11-29-11:55:41.806.183
[ra_adp.c:1425]tid:13722,ra_hdc_server_init(1425) : Create Server failed, ret(61)
[ERROR] HCCP(13722,hccp_service.bin):2023-11-29-11:55:41.806.200
[ra_adp.c:1546]tid:13722,hccp_init(1546) : chip_id[0] ra_hdc_server_init failed, ret[-22]
[ERROR] HCCP(13722,hccp_service.bin):2023-11-29-11:55:41.806.265
[main.c:224]tid:13722,main(224) : hccp init error[-22]
- Device报错 “certificate is not yet valid”

原因分析与解决方法（Device 报错 “Create Server failed, ret(61)”）

若Device报错 “Create Server failed, ret(61)”，可能有以下两种原因：

- 人为操作问题（偶现），上一次训练未完全退出，就拉起新的训练。

原因分析：

在Host侧执行如下hccn_tool命令：

```
for i in {0..7}; do hccn_tool -i $i -process -g ; done
```

查看Device侧是否存在hccp进程。

解决办法：

稍等一会（通常停掉训练任务脚本，会进资源销毁释放流程），确认进程完全退出后再拉起训练进程。

- 业务脚本问题（必现），每次拉起训练时，会在一个device上拉起2个及以上的hccp进程。

原因分析:

执行如下命令，导出日志，并确认是否在临近间隔时间，通常是ms级时间间隔内，有两次及以上hccp进程拉起的日志记录。

```
cd /root/ascend/log/run/  
grep -rn "hccp init" *
```

如下所示，即可判定是由于业务脚本在一个Device上拉起2个及以上的HCCP进程导致的，需要用户排查业务脚本。

```

logdev-os-0/debug/device-os/device-os_20231017120017555.log:432: [EVENT] HCCP[29410,hccp_service.bin]:2023-10-17-12:00:19.137.676 [main.c:200]tid:29410,main(200) : hccp init st
logdev-os-0/debug/device-os/device-os_20231017120017555.log:435: [EVENT] HCCP[29422,hccp_service.bin]:2023-10-17-12:00:19.181.966 [main.c:200]tid:29422,main(200) : hccp init st
logdev-os-0/debug/device-os/device-os_20231017120017555.log:456: [EVENT] HCCP[15549,hccp_service.bin]:2023-10-17-11:04:30.430.577 [main.c:200]tid:15549,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:5980: [EVENT] HCCP[18201,hccp_service.bin]:2023-10-17-11:49:29.117.380 [main.c:200]tid:18201,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:5985: [EVENT] HCCP[18285,hccp_service.bin]:2023-10-17-11:49:29.166.936 [main.c:200]tid:18285,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:5994: [EVENT] HCCP[19021,hccp_service.bin]:2023-10-17-11:50:05.746.911 [main.c:200]tid:19021,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:6165: [EVENT] HCCP[21189,hccp_service.bin]:2023-10-17-11:54:27.939.275 [main.c:200]tid:21189,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:6168: [EVENT] HCCP[21193,hccp_service.bin]:2023-10-17-11:54:27.978.754 [main.c:200]tid:21193,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:6395: [EVENT] HCCP[23300,hccp_service.bin]:2023-10-17-11:58:22.742.250 [main.c:200]tid:23300,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:6398: [EVENT] HCCP[23304,hccp_service.bin]:2023-10-17-11:58:22.786.282 [main.c:200]tid:23304,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:6500: [EVENT] HCCP[24802,hccp_service.bin]:2023-10-17-12:00:01.199.157 [main.c:200]tid:24802,main(200) : hccp init
logdev-os-0/debug/device-os/device-os_20231017095935567.log:6503: [EVENT] HCCP[24806,hccp_service.bin]:2023-10-17-12:00:01.239.712 [main.c:200]tid:24806,main(200) : hccp init

```

解决方法:

需要自行排查业务训练脚本中问题，如下是几个常见案例。

- a. 单机16卡跑hccl test报错: -p与-n不配套导致多次拉起。

```
root@box-16:/usr/local/Ascend/ascend-toolkit/latest/tool/hool_test# mpirun -n 16 /bin/all_reduce_test -b 8K -e 1G -f 2 -d int8 -o sum -p 0
The max_iters is 8192, max_bytes is 1073741824, iters is 20, warmup iters is 5
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
This is an error in init_hoolComm.
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
This is an error in init_hoolComm.
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
This is an error in init_hoolComm.
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
hool interface return err:returerr err ./common/src/hool_test_common.c:499, retcode: 11
This is an error in init_hoolComm.
This is an error in init_hoolComm.
This is an error in init_hoolComm.
This is an error in init_hoolComm.
This is an error in init_hoolComm.
```

- b. “pytorch nproc_per_node”参数需要填写节点上要启动的训练进程数，假设只有2个卡可用，但此参数填写的8，所以重复拉起4次，导致出错。如下所示，对应日志中看到重复拉起了4次：

```
log/dev-os/0/debug/device-os/device-os_2023092712108529.log:1946: [EVENT] HCCP/26609.hccp.service/bin/1923:09-27-15:26:48.633.761 [main:c200]tgid:26609,main(200) : hccp unit start!
```

原因分析及解决方法（Device 报错 “certificate is not yet valid”）

- **原因分析:**

若Device报错“certificate is not yet valid”，则原因为：

Host时钟异常，导致系统时间早于TLS证书有效时间，TLS开关打开情况下校证书失败，拉起训练失败。

如下所示，查询TLS证书看到证书有效起始时间是：2023/09/26

```
[root@GXXYA9-404-A-19-A1P3-SEV-G5680-04U07 log]# for i in (0..0); do hcnn_tool -i $i -tls -g ; done
dev_id:0, tls switch[1](0:disable, 1:enable), tls preconfigured[1](0:non-preset, 1:preset), tls alarm time threshold[60]days
dev_id:0, [pub cert] info:
  issuer[/C=CN/O=Huawei/CN=Huawei IT Product CA]
  start_time[Tue Sep 26 07:57:04 2023 GMT]
  end_time[Sat Sep 25 07:57:04 2038 GMT]
  tls expiration status[0](0:normal, 1:near expiration, 2:has expired)
dev_id:0, [cal cert] info:
  issuer[/C=CN/O=Huawei/CN=Huawei Equipment CA]
  start_time[Tue Dec 6 07:34:23 2011 GMT]
  end_time[Thu Nov 28 07:34:23 2041 GMT]
  tls expiration status[0](0:normal, 1:near expiration, 2:has expired)
dev_id:0, [ca2 cert] info:
  issuer[/C=CN/O=Huawei/CN=Huawei Equipment CA]
  start_time[Tue Oct 18 06:50:53 2016 GMT]
  end_time[Sat Oct 12 06:50:53 2041 GMT]
  tls expiration status[0](0:normal, 1:near expiration, 2:has expired)
```


对应HCCP进程拉起时，报错证书无效，校验失败，拉起HCCP失败，看到系统时间是：2019/09/26

```
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.434 [rs_ssl.c:351]tid:7209,rs_ssl_verify_cert_chain[351] : X509_verify_cert fail, ret = 0, err:9, certificate is not yet valid
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.452 [rs_ssl.c:383]tid:7209,rs_ssl_check_cert_chain[383] : verify_cert_chain failed,ret -22
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.462 [rs_ssl.c:402]tid:7209,rs_ssl_check_mng_and_cert_chain[402] : ssl check cert chain failed,ret -22
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.469 [rs_ssl.c:500]tid:7209,rs_ssl_put_certs[500] : check mng and cert chain failed -22
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.494 [rs_ssl.c:550]tid:7209,rs_ssl_get_ca_data[550] : put certs err,ret -22
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.500 [rs_ssl.c:607]tid:7209,rs_ssl_load_ca[607] : get ca data err -13
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.508 [rs_ssl.c:939]tid:7209,rs_ssl_ca_key_init[939] : load ca err,ret -13
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.515 [rs_ssl.c:978]tid:7209,rs_ssl_inner_init[978] : SSL server check cert and key failed,ret -13
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.531 [rs_ssl.c:1020]tid:7209,rs_ssl_inner_enable[1020] : dev 0 ssl init failed, ret -22
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.537 [rs.c:304]tid:7209,rs_init_rscb_cfg[304] : init ssl failed, ret[-22]
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.543 [rs.c:361]tid:7209,rs_init[361] : rs init rscb configure fail,ret:-22
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.548 [ra_rdev.c:1520]tid:7209,hccp_init[1520] : rs_init failed [ret=-22]
[ERROR] HCCP(7209,hccp_service.bin):2019-09-26-14:14:02.096.654 [main.c:224]tid:7209,main[224] : hccp snit error[-22]
```

- 解决方法：
同步正常时间后，可以恢复。

9.2.2 EJ0002 打屏报错

问题现象

拉起训练进程时，报EJ0002 Environment Error，此报错通常由于环境异常，导致rdev初始化失败，拉起训练进程失败。

查看HCCP初始化ra_rdev阶段报错，对应HDC接口报错信息为：**ra hdc message process failed ret(-67)**

```
[ERROR] HCCP(46430,alltoallv_test):2023-09-21-03:43:49.546.469 [ra
hdc.c:1270]tid:46430,ra_hdc_rdev_init(1270) : [init][ra_hdc_rdev]ra hdc message process failed ret(-67)
phy_id(3)
[ERROR] HCCP(46430,alltoallv_test):2023-09-21-03:43:49.546.488
[ra_host.c:621]tid:46430,ra_rdev_init(621) : [init][ra_rdev]ra rdev init failed. ret(-67)
```

原因分析

网卡down导致初始化时，HCCP调用HDC接口返回-67，对应错误码定义（#define ENOLINK 67 /* Link has been severed */）

解决方法

1. 执行如下命令，检查网口状态。
for i in {0..7}; do hccn_tool -i \$i -link -g; done
2. 用户自行排查物理链路是否连通，检查软件配置是否正确。
 - 重新配置ip和netmask（有可能未配置ip）
hccn_tool -a -cfg recovery
基于/etc/hccn.conf中配置恢复环境配置
 - 查询光模块是否在位（evb环境无光模块）
for i in {0..7}; do hccn_tool -i \$i -optical -g; done
 - 查询交换机信息（交换机信息会有缺失）
for i in {0..7}; do hccn_tool -i \$i -lldp -g; done
 - 咨询环境人员光纤类型，是否交换机打开了对应的FEC策略、CDR版本是否过老，需要升级、是否光模块问题，将有问题的光模块+光纤和无问题的交换验证

9.2.3 EJ0003 打屏报错

问题现象

建链阶段报EJ0003的错误，HCCP调系统函数bind失败，常见报错**error: 98**或**error: 99**。

该报错对应Linux标准报错定义如下：

```
# define EADDRINUSE 98 /* Address already in use */  
# define EADDRNOTAVAIL 99 /* Cannot assign requested address */
```

查看plog日志中关于HCCP的报错：

grep -rn ERROR * | grep HCCP | head

报错1：

```
plog/plog-3541138_20231113190511833.log:30551:[ERROR]  
HCCP(3541138,host_cpu_executor):2023-11-13-19:05:49.833.681  
[rs_socket.c:671]tid:3541138,rs_socket_listen_bind_listen(671) : bind fail! family:2, IP:127.0.0.1, port:50000,  
sock:27, ret:0xffffffff, error:98, Possible Cause: the IP address and port have been bound already  
plog/plog-3541138_20231113190511833.log:30552:[ERROR]  
HCCP(3541138,host_cpu_executor):2023-11-13-19:05:49.833.687  
[rs_socket.c:860]tid:3541138,rs_socket_listen_start(860) : bind and listen fail, err_no:98, listen_fd:27, listen  
state:1, IP(127.0.0.1) server_port:50000  
plog/plog-3541138_20231113190511833.log:30554:[ERROR]  
HCCP(3541138,host_cpu_executor):2023-11-13-19:05:49.833.704  
[ra_peer.c:114]tid:3541138,ra_peer_socket_listen_start(114) : [listen_start][ra_peer_socket]ra listen start  
failed ret(-98).
```

报错2：

```
[ERROR] HCCP(160639,all_gather_test):2023-10-17-07:28:13.178.137  
[rs_socket.c:671]tid:160639,rs_socket_listen_bind_listen(671) : bind fail! ::350d:755b:ada4:ada6, port:60000,  
sock:25, ret:0xffffffff, error:99, Possible Cause: the IP address and port have been bound already
```

报错3：

```
75346:[ERROR] HCCP(13854,python3):2023-09-04-14:37:34.444.964  
[rs_socket.c:1124]tid:21717,rs_connect_bind_client(1124) : client bind fail! IP:fe80::4b79:e6c9:31eb:f019,  
sock:530, ret:-1, error:99  
75347:[ERROR] HCCP(13854,python3):2023-09-04-14:37:34.444.968  
[rs_socket.c:1164]tid:21717,rs_socket_state_reset(1164) : rs_connect_bind_client failed, ret[-99]
```

原因分析

- 配置错误，可能有以下情况：
 - 未配置网卡IP。
 - 网卡IP与业务配置的IP不匹配。
 - 对一个IP地址及端口重复监听了多次。
- Host侧未指定网卡建链，导致进程绑定IP地址报错。
- 网卡状态异常。

解决方法

步骤1 检查配置。

- 查看Device网卡的IP地址是否存在。
执行**hccn_tool -i dev_id -ip -g**命令查询对应的Device网卡的IP地址是否存在。
若不存在，请执行如下命令进行配置。
hccn_tool -i dev_id -ip -s address 192.168.0.3 netmask 255.255.255.0
- 检查ranktable文件中配置的IP地址与网卡实际IP是否一致，若不一致，请修改。
说明：可以在Device日志中搜索关键字“listen fail”查看监听失败的IP地址。
- 检查应用程序端口与HCCL系统进程端口是否冲突。

分布式训练场景下，HCCL默认会使用Host服务器的60000-60015端口收集集群信息，若通过环境变量HCCL_IF_BASE_PORT指定了Host网卡起始端口，则需要预留以该端口起始的16个端口。

可通过`lsof -i:60000`确认占用端口的进程信息，通过`netstat -ant | grep WAIT`命令查看端口状态。

若确认是应用程序端口与HCCL系统进程端口冲突，可通过如下命令预留系统端口。

```
sysctl -w net.ipv4.ip_local_reserved_ports=60000-60015
```

- 检查是否存在IP与端口重复监听的问题。

设置日志级别为EVENT，然后在日志中`grep -rn "socket bind" *`，确认当前训练端口监听情况。

步骤2 若绑定网卡报错，可通过环境变量HCCL_SOCKET_IFNAME指定初始化Host通信网卡，HCCL可通过该网卡名获取Host IP，完成通信域创建。

关于环境变量HCCL_SOCKET_IFNAME的详细说明可参见《[环境变量参考](#)》，配置示例如下：

```
export HCCL_SOCKET_IFNAME==eth0
```

步骤3 执行`hccn_tool -i dev_id -link -g`命令查看对应的Device网卡状态，若网卡状态异常，请排查网卡错误。

----结束

9.2.4 EJ0004 打屏报错

问题现象

训练拉起失败，报EJ0004错误，查看日志，HCCP首报错如下：

```
[ERROR] HCCP(1609,python3):2023-10-19-12:33:13.313.594  
[ra_host.c:544]tid:1609,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(-16) the IP address(29.4.84.36) in the  
ranktable is inconsistent with the IP(29.4.76.85)address of the network adapter, please make sure they're  
consistent. num(1)
```

搜索Error日志：`grep -rn ERROR | grep HCCP | grep inconsistent`

发现当前节点ranktable中的ip和device ip不一致，如下所示。

```
plog/plog-1601_20231019123251864.txt:21:[ERROR] HCCP(1601,python3):2023-10-19-12:33:11.645.514  
[ra_host.c:544]tid:1601,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(192) the IP address(29.4.47.87) in  
the ranktable is inconsistent with the IP(29.4.76.155)address of the network adapter, please make sure  
they're consistent. num(1)  
plog/plog-1590_20231019123251766.txt:21:[ERROR] HCCP(1590,python3):2023-10-19-12:33:11.313.520  
[ra_host.c:544]tid:1590,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(160) the IP address(29.4.159.125) in  
the ranktable is inconsistent with the IP(29.4.67.170)address of the network adapter, please make sure  
they're consistent. num(1)  
plog/plog-1612_20231019123251802.txt:21:[ERROR] HCCP(1612,python3):2023-10-19-12:33:11.365.521  
[ra_host.c:544]tid:1612,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(161) the IP address(29.4.156.176) in  
the ranktable is inconsistent with the IP(29.4.69.45)address of the network adapter, please make sure  
they're consistent. num(1)  
plog/plog-1585_20231019123251689.txt:21:[ERROR] HCCP(1585,python3):2023-10-19-12:33:11.377.587  
[ra_host.c:544]tid:1585,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(-162) the IP address(29.4.59.254) in  
the ranktable is inconsistent with the IP(29.4.0.99)address of the network adapter, please make sure they're  
consistent. num(1)  
plog/plog-1593_20231019123251798.txt:21:[ERROR] HCCP(1593,python3):2023-10-19-12:33:11.785.581  
[ra_host.c:544]tid:1593,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(-80) the IP address(29.4.187.47) in  
the ranktable is inconsistent with the IP(29.4.132.121)address of the network adapter, please make sure  
they're consistent. num(1)  
plog/plog-1609_20231019123251900.txt:21:[ERROR] HCCP(1609,python3):2023-10-19-12:33:13.313.594  
[ra_host.c:544]tid:1609,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(-16) the IP address(29.4.84.36) in the  
ranktable is inconsistent with the IP(29.4.76.85)address of the network adapter, please make sure they're
```

```
consistent. num(1)
plog/plog-1598_20231019123251821.txt:21:[ERROR] HCCP(1598,python3):2023-10-19-12:33:11.735.192
[ra_host.c:544]tid:1598,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(64) the IP address(29.4.146.67) in
the ranktable is inconsistent with the IP(29.4.184.7)address of the network adapter, please make sure
they're consistent. num(1)
plog/plog-1606_20231019123251824.txt:21:[ERROR] HCCP(1606,python3):2023-10-19-12:33:11.525.543
[ra_host.c:544]tid:1606,ra_rdev_init_check_ip(544) : [check][ip]fail, ret(96) the IP address(29.4.21.78) in the
ranktable is inconsistent with the IP(29.4.85.223)address of the network adapter, please make sure they're
consistent. num(1)
```

原因分析

此错误通常由于业务配置错误导致。

解决方法

请检查ranktable配置，确认是否存在以下问题：

- 存在重复的rank id。
若存在重复rank id，请修改，确保rank id全局唯一。
- 配置的Device IP地址与实际Device的网卡地址不一致。

可通过如下命令查询Device IP：

```
for i in `seq 0 7`; do echo "=====> dev$i, NPU$((i+1))"; hccn_tool -i $i -ip -g; done
```

若不一致，请修改配置的Device IP地址为实际的网卡地址。

9.2.5 参与集合通信的服务器 TLS 信息不一致，HCCL 初始化失败

问题现象

分布式训练场景下，集合通信建链失败，HCCL关键日志信息如下：

```
[EVENT] HCCP(23457,all_reduce_test):2023-10-16-08:39:56.291.195
[ra_host.c:1672]tid:23468,ra_socket_white_list_add(1672) : Input parameters: phy_id[0],
local_ip[192.168.100.101], num[1]
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.748 [comm_base.cc:967][23468]
LINK_ERROR_INFO
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.771 [comm_base.cc:968][23468] | comm
error, device[0] num[1]
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.777 [comm_base.cc:969][23468] |
dest_ip(rank_id) | src_ip(rank_id) | Role | Status |
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.783 [comm_base.cc:970][23468]
|-----|-----|-----|-----|
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.797 [comm_base.cc:1018][23468] |
192.168.100.100(1) | 192.168.100.101(0) | server | no connect |
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.804 [comm_base.cc:983][23468]

[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.809 [comm_base.cc:984][23468]the
connection failure between this device and target device may be due to the following reasons:
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.815 [comm_base.cc:985][23468]1. the
connection between this device and the target device is abnormal.
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.820 [comm_base.cc:986][23468]2. an
exception occurred at the target devices.
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.825 [comm_base.cc:988][23468]3. the
time difference between the execution of hcom on this device and the target device exceeds the
timeout threshold. make sure this by keyword [Entry-]
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.830 [comm_base.cc:990][23468]4. the
behavior of executing the calculation graph on this device and the target device is inconsistent.
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.836 [comm_base.cc:991][23468]5. The TLS
switch is inconsistent, or the TLS certificate expires.
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.841 [comm_base.cc:992][23468]6. If src
and dst IP address can be pinged, check whether the device IP address conflicts.
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.847 [comm_base.cc:437][23468][Get]
```

```
[RaSocket]in comm, get rasocket error role[0], rank[0], num[1], gotten[0], timeout[120]
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.853 [comm_base.cc:768][23468]call trace:
hcclRet -> 9
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.861 [comm_base.cc:827][23468]call trace:
hcclRet -> 9
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.867 [comm_base.cc:199][23468]call trace:
hcclRet -> 9
[ERROR] HCCL(23457,all_reduce_test):2023-10-16-08:41:56.291.873 [comm_base.cc:82][23468]call trace:
hcclRet -> 9
```

查询HCCP Debug日志信息，关键报错信息如下：

```
[ERROR] HCCP(6594,hccp_service.bin):2023-10-16-08:39:56.335.828
[rs_ssl.c:206]tid:6610,rs_ssl_err_string(206) ; ssl fd 42 err 1 errno 0, err code 167773208, err msg
error:0A000418:SSL routines::tlsv1 alert unknown ca, The possible cause is that the TLS switch is
inconsistent.
```

原因分析

参与集合通信的各服务器TLS状态开关不一致，或者当TLS状态开关统一打开时，TLS证书信息不一致。

解决方法

步骤1 查询集合通信的各服务器TLS状态开关。

在服务器中执行如下命令，获取TLS开关使能状态。

```
hccn_tool -i <device_id> -tls -g [host]
```

其中<device_id>为Device设备的逻辑ID，您也可以通过如下for语句，一次性查询所有Device设备的TLS信息。

```
for i in `seq 0 7`; do hccn_tool -i $i -tls -g; done # 0, 7分别为需要查询的Device ID的起始与结束值。
```

回显信息如下所示：

```
dev_id:0, tls switch[0] (0:disable, 1:enable), tls alarm time threshold[60]days
dev_id:0, [pub cert] info:
    issuer[/C=CN/ST=GD/O=HUAWEI/OU=2012/CN=2_1thCA]
    start_time[Wed Feb 19 03:19:21 2020 GMT]
    end_time[Sat Feb 16 03:19:21 2030 GMT]
dev_id:0, [ca1 cert] info:
    issuer[/C=CN/ST=GD/L=SZ/O=HUAWEI/CN=1thCA]
    start_time[Wed Feb 19 03:19:07 2020 GMT]
    end_time[Sat Feb 16 03:19:07 2030 GMT]
dev_id:0, [ca2 cert] info:
    issuer[/C=CN/ST=GD/L=SZ/O=HUAWEI/CN=1thCA]
    start_time[Wed Feb 19 03:19:10 2020 GMT]
    end_time[Sat Feb 16 03:19:10 2030 GMT]
dev_id:1, tls switch[0] (0:disable, 1:enable), tls alarm time threshold[60]days
dev_id:1, [pub cert] info:
    issuer[/C=CN/ST=GD/O=HUAWEI/OU=2012/CN=2_1thCA]
    start_time[Wed Feb 19 03:19:21 2020 GMT]
    end_time[Sat Feb 16 03:19:21 2030 GMT]
dev_id:1, [ca1 cert] info:
    issuer[/C=CN/ST=GD/L=SZ/O=HUAWEI/CN=1thCA]
    start_time[Wed Feb 19 03:19:07 2020 GMT]
    end_time[Sat Feb 16 03:19:07 2030 GMT]
dev_id:1, [ca2 cert] info:
    issuer[/C=CN/ST=GD/L=SZ/O=HUAWEI/CN=1thCA]
    start_time[Wed Feb 19 03:19:10 2020 GMT]
    end_time[Sat Feb 16 03:19:10 2030 GMT]
... ..
```

其中tls switch[0]，代表TLS状态为关闭，switch[1]代表TLS状态为使能。

步骤2 判断各服务器中所有Device的TLS状态开关是否一致。

- 若不一致，建议统一修改TLS状态为使能。若TLS开关关闭，集合通信时会存在信息被窃听、篡改、仿冒的风险。

您可以通过如下命令修改TLS状态开关：

```
hccn_tool -i <device_id> -tls -s enable 1
```

enable为使能开关，配置为1代表使能，配置为0代表关闭。

- 若一致且状态为使能，建议您继续执行**步骤3**判断各节点的TLS证书信息是否一致。

步骤3 查看所有服务器中各Device的TLS证书信息是否一致。

您可以通过**步骤1**中的信息判断各Device TLS证书信息是否一致。若不一致，您可以通过如下命令替换证书套件。

```
hccn_tool -i 0 -tls -s path /root pri pri.pem pub pub.pem ca1 ca1.pem ca2 ca2.pem crl xxx.crl
```

-i为Device ID，-path为指定证书/私钥/吊销列表存放路径，pri为私钥名字，pub为设备证书文件名，ca1/ca2/crl分别为根证书、二级根证书、吊销列表文件名。

关于hccn_tool工具的更多用法及参数解释，可查看对应设备的《HCCN Tool 接口参考》。

《HCCN Tool接口参考》的获取方式为：单击[Link](#)进入企业业务网站的“昇腾计算 文档中心”，然后在“中心训练硬件”栏选择对应的硬件型号，单击进入对应的文档页面，即可在其中找到配套版本的《HCCN Tool接口参考》。

----结束

9.3 HCCL Test 常见问题总结

9.3.1 单机带宽低

问题现象

使用HCCL Test工具测试单机带宽时，单机带宽峰值低于预期值。

原因分析

1. 日志等级设置不是error。
若某些卡的日志等级设置不是error，可能会导致对应卡的带宽降低（某些场景下，可能会导致带宽降低2GB左右）。
2. 某条链路带宽低。
某条链路带宽低会导致单机整机带宽低。

解决步骤

1. 若是日志级别设置不是error导致的带宽降低，可通过重新设置Host侧与Device侧日志级别为error解决。

Host侧日志级别：

```
export ASCEND_GLOBAL_LOG_LEVEL=3 // (0:debug 1:info 2:warning 3:error)
```

Device侧日志操作：

```
# 查询
for i in {0..7}; do /usr/local/Ascend/driver/tools/msnpureport -r -d $i; done
# 设置
for i in {0..7}; do msnpureport -g error -d $i; done
```

2. 若重设日志级别后，带宽仍然较低，可分析是否某条链路HCCS带宽低，或者PCIe带宽低。

可通过ascend-dmi命令测p2p的带宽，命令示例如下：

- 测试卡0到卡8的链路带宽：
ascend-dmi --bw -t p2p --ds 0 --dd 8
- 测试全量p2p带宽：
ascend-dmi --bw -t p2p
全量p2p带宽测试时间较长，预计20分钟左右。

获取以上数据后，单击[Link](#)联系技术支持。

9.3.2 多机峰值带宽低

问题现象

使用HCCL Test工具测试多机带宽时，多机带宽峰值低于预期值。

原因分析

- 使用HCCL Test性能测试工具时，未设置HCCL_BUFFSIZE环境变量，或者测试命令中设置的测试数据量较小。
- 开启了Profiling功能或日志等级未设置为默认的ERROR级别，会导致带宽较低。
- 某台机器带宽较低，导致多机带宽低。
- 交换机负载分担方式配置不合理，产生拥塞。
- spine和leaf交换机之间的某个端口状态异常。

解决步骤

步骤1 确认HCCL_BUFFSIZE环境变量是否设置。

环境变量HCCL_BUFFSIZE用于调整两个NPU之间共享数据的缓存区大小，两个NPU之间共享数据的缓存区大小默认为200M，使用HCCL Test工具进行性能测试的场景下，往往通信数据量较大，此种场景下，可适当增大HCCL_BUFFSIZE的值，提升数据通信效率与带宽。

配置示例：

```
export HCCL_BUFFSIZE=2048
```

步骤2 确认hccl_test测试命令中的参数“-e”是否配置过小。

“-e”代表测试数据大小的结束值，若“-e”较小，则带宽会较小，建议增大-e参数的值，例如：

```
mpirun -n 8 ./bin/all_reduce_test -b 8K -e 4G -f 2 -d fp32 -o sum -p 8
```

步骤3 查看是否开启了Profiling性能数据采集功能。

Profiling性能数据采集功能会导致带宽变低，请关闭此功能。

每种业务场景的性能数据采集功能开启方法不同，详细的方法可参见《[性能调优工具指南](#)》中的“性能分析”章节。

步骤4 查看日志级别是否为“ERROR”。

查看Host与Device的日志级别：

- Host侧日志级别可通过**echo \$ASCEND_GLOBAL_LOG_LEVEL**命令查看，0代表debug，1代表info，2代表warning，3代表error。
- Device侧日志级别可通过如下命令查看：
for i in {0..7}; do /usr/local/Ascend/driver/tools/msnpureport -r -d \$i; done

若日志级别不为默认值ERROR，则执行如下命令分别修改HOST与DEVICE的日志级别为“ERROR”。

- Host侧日志级别修改
在测试主机设置即可，MPI会将环境变量带到所有测试机器：
export ASCEND_GLOBAL_LOG_LEVEL=3 // (0:debug 1:info 2:warning 3:error)
- Device日志级别修改
for i in {0..7}; do msnpureport -g error -d \$i; done

步骤5 多机带宽较低还可能是某台机器带宽较低或网络配置不一致导致的。

此种场景下，可以通过二分法找到这台机器，然后参见[9.3.1 单机带宽低](#)进行可能原因的排查，若单机测试均无问题可尝试通过“**cat /etc/hccn.conf**”命令检查所有服务器配置是否一样，某台机器网络配置不一致，有可能会造成单机测试无问题（单机不使用外部网络），但多机带宽低的情况。

步骤6 检查交换机负载分担方式是否配置合理。

执行如下命令，查看服务器统计信息：

```
for i in $(seq 0 15); do echo "=====> $i"; hccn_tool -i $i -stat -g | grep pfc ;done
```

统计信息中有很多“rx pfc”，标识交换机负载分担不均衡，产生了拥塞。

可通过如下方法尝试解决：

- 首先解决交换机的pfc问题，要做到pfc很少甚至没有pfc。
- 查看交换机流量是否不均衡，有没有出现多打一（多条流量走一个端口出），有些端口流量大有些端口流量小，产生拥塞的情况。
- 假如是通过udp端口进行哈希选路，查看是否某些机器的udp port没配置。

----结束

9.3.3 小数据量时延大

问题现象

小数据量场景下时延较大，例如数据量小于4M时单机8卡allreduce时延大于300us。

原因分析

Device的日志级别如果不是默认值ERROR，会导致时延较大。

可通过如下命令查询Device日志等级：

```
for i in {0..7}; do /usr/local/Ascend/driver/tools/msnpureport -r -d $i; done
```

解决步骤

执行如下命令，将Device日志等级设置为error。

```
for i in {0..7}; do msnpureport -g error -d $i; done
```

9.3.4 多机测试大数据量带宽骤降为 0

问题现象

多机大数据量进行性能测试时出现以下情况：

- 带宽骤降为0点几。
- 查看统计数据，重传次数（roce_new_pkt_rty_num）会增长。
- 设置export HCCL_RDMA_TIMEOUT=10 缩短重传等待时间，带宽不会骤降为0点几。

原因分析

数据转发过程中发生丢包，产生了重传，从而拖慢时间。

解决步骤

解决此问题的根因是找到丢包所在的位置，例如spine交换机、leaf交换机、服务器、线缆等，可按照如下顺序排查丢包的地方。

步骤1 丢包在线缆，端口闪断丢包。

查询端口link记录

```
for i in $(seq 0 7); do echo "=====> $i"; hccn_tool -i $i -link_stat -g ;done
```

找到闪断的端口，然后排查硬件闪断原因。

```
[devid 7]current time Wed Dec 20 17:13:50 2023  
[devid 7]link up count 2  
[devid 7]link down count 1  
[devid 7]link change records  
[devid 7] Thu Dec : 14 10:57:34 2023 LINK UP  
[devid 7] Thu Dec 14 10:57:33 2023 LINK DOWN  
[devid 7] Thu Dec 14 10:57:19 2023 LINK UP
```

步骤2 丢包在交换机 ——交换机配置原因。

需要排查交换机配置，分别登录spine和leaf交换机检查丢包原因，可能存在如下原因：

- 交换机buffer水线设置不合理，流控方式不对。
- 只配了spine的PFC，没有配leaf交换机PFC，spine反压给leaf，leaf没有降速，导致丢包。
- ECN和PFC水线配置配合不合理导致丢包，若不是一定要用ECN，建议关掉ECN。

步骤3 丢包在交换机——交换机和服务器配合原因。

交换机和服务器流控参数配置不匹配，流量进错队列丢包。

交换机和服务器的dscp、pfc优先级队列等需要协商一致，原理请参见《[Ascend Training Solution 组网指南](#)》中的“[拥塞控制与纠错配置策略](#)”章节。

排查交换机配置：

可以在交换机侧看对应的报文dscp对不对，有没有走到对应的队列

在交换机看是否进到对应队列，命令：display qos queue statistics interface 10ge 1/0/1

排查服务器流控参配置：

```
查询 dscp_to_tc
for i in $(seq 0 7); do echo "=====> $i"; hccn_tool -i $i -dscp_to_tc -g ;done
查询 PFC
for i in $(seq 0 7); do echo "=====> $i"; hccn_tool -i $i -pfc -g;done
设置dscp_to_tc
for i in $(seq 0 7); do echo "=====> $i"; hccn_tool -i $i -dscp_to_tc -s dscp 33 tc 2;done
设置PFC
for i in $(seq 0 7); do echo "=====> $i"; hccn_tool -i $i -pfc -s bitmap 0,0,0,0,1,0,0,0;done
```

排查流控环境变量：

测试之前要确保这两个环境变量已经设置，不配置采用默认参数（TC=132，SL=4）

```
export HCCL_RDMA_TC=132 # 数值为dscp左移两位（乘以4），根据和交换机协商配置
export HCCL_RDMA_SL=4 # 数值为PFC的队列，根据和交换机协商配置
```

步骤4 丢包在服务器（TC buffer溢出导致丢包）。

原因可能是服务器buffer设置不合理，或者是交换机未响应服务器的PFC，最大使用的buffer达到了TCbuffer最大值，buffer溢出导致服务器丢包。

- 排查服务器统计是否有tx pfc。

```
for i in $(seq 0 15); do echo "=====> $i"; hccn_tool -i $i -stat -g ;done | grep pfc
```
- 排查buffer和上下水线，判断使用的最大buffer是否达到了设置的buffer上限。
#查询tc buffer配置

```
for i in $(seq 0 15); do hccn_tool -i $i -tc_cfg -g;done
```


#查询某个TC使用的最大buffer（读清）

```
for i in $(seq 0 15); do hccn_tool -i $i -cur_tc_buf -g tc 1;done #tc根据使用的tc改
```

----结束

9.3.5 测试命令卡数与实际卡数不一致，返回 retcode 11

问题现象

执行HCCL Test测试命令时，返回“return code 11”错误，例如：

```
“hccl interface return errreturn err ./common/src/hccl_test_common.cc:499,
retcode: 11”
```

原因分析

HCCL Test测试命令中配置的卡数与实际的卡数不一致。如下所示为错误命令示例：

```
mpirun -n 16 ./bin/all_reduce_test -b 8K -e 1G -f 2 -d int8 -o sum -p 8 -c 0
```

- -n：需要启动的NPU总数。
- -p：单个计算节点上参与训练的NPU个数。

示例命令错误的原因：这是一个单机测试命令，“-n 16”说明要启动的NPU总数为16个，“-p 8”单个节点上参与训练的NPU个数为8，这个节点的实际卡数达不到配置中的NPU总数16。

解决步骤

修改测试命令，检查“-n”与“-p”的NPU个数（进程数）是否配置正确。

- -n：需要启动的NPU总数。
- -p：单个计算节点上参与训练的NPU个数。

9.3.6 未设置 hostname 与 ip 对应关系导致 MPI error 报错

问题现象

报错MPI error，报错类似如下：

```
Fatal error in MPI_Init_thread: Other MPI error, error stack:
MPIR_Init_thread(474)
MPID_Init(190) channel initialization failed
MPIDI_CH3_Init(89)
MPID_nem_init(320).
MPID_nem_tcp_init(173)
MPID_nem_tcp_get_business_card(420):
MPID_nem_tcp_init(379).
.: gethostbyname failed, Georgreens-MacBook-Pro.local (errno 1)
```

原因分析

“/etc/hosts”文件中未配置对应的主机IP地址和主机名。

解决步骤

/etc/hosts添加主机IP地址（Host IP）和主机名（Host Name），如下所示：

```
172.16.0.100 HW-AI-LC-1-1
```

9.3.7 HCCLComm 初始化失败

问题现象

执行HCCL Test测试工具时，报“This is an error in init_hcclComm”的错误，如下图所示：

```
mv all gather test all reduce test alltoallv test alltoall test broadcast test reduce_scatter_test reduce_test ./bin
[root@node-96-59 hccl_test]# mpirun -n 8 ./bin/all_reduce_test -b 8K -e 4G -f 2 -o sum -p 8
the minbytes is 8192, maxbytes is 4294967296, iters is 20, warmup_iters is 5
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
This is an error in init_hcclComm.
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
This is an error in init_hcclComm.
This is an error in init_hcclComm.
This is an error in init_hcclComm.
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
This is an error in init_hcclComm.
This is an error in init_hcclComm.
This is an error in init_hcclComm.
This is an error in init_hcclComm.

=====
= BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
= PID 2290334 RUNNING AT node-96-59
= EXIT CODE: 139
= CLEANING UP REMAINING PROCESSES
=====
```

原因分析

某些卡被进程占用，导致无法使用HCCL Test工具进行测试。

解决步骤

步骤1 执行“npu-smi info”命令查看卡的占用情况。

某些场景下，npu-smi info未显示卡被占用，但片上内存使用非常高，此种情况下，也会引起HCCL Test测试工具执行失败。

步骤2 确认被占用卡上的进程释放后，再重新执行HCCL Test测试工具。

通常停掉训练任务脚本，会进入资源释放销毁流程，可在Host侧执行如下命令，确认进程完全退出后，再执行测试工具。

```
for i in {0..7}; do hccn_tool -i $i -process -g ; done
```

----结束

9.3.8 防火墙未关闭

问题现象

测试报错如下，查询机器防火墙发现防火墙未关闭

```
[root@node-87-66 hccl_test]# mpirun -f hostfile -n 16 ./bin/all_reduce_test -b 8K -e 4G -f 2 -d fp32 -p 8
the minbytes is 8192, maxbytes is 4294967296, iters is 20, warmup_iters is 5
Fatal error in PMPI_Barrier: Unknown error class, error stack:
PMPI_Barrier(425).....: MPI_Barrier(MPI_COMM_WORLD) failed
MPIR_Barrier_impl(332).....: Failure during collective
MPIR_Barrier_impl(327).....:
MPIR_Barrier(292).....:
MPIR_Barrier_intra(150).....:
barrier_smp_intra(96).....:
MPIR_Barrier_impl(332).....: Failure during collective
MPIR_Barrier_impl(327).....:
MPIR_Barrier(292).....:
MPIR_Barrier_intra(169).....:
MPIIDI_CH3U_Recvq_FDU_or_AEP(629): Communication error with rank 8
barrier_smp_intra(111).....:
MPIR_Bcast_impl(1452).....:
MPIR_Bcast(1476).....:
MPIR_Bcast_intra(1287).....:
MPIR_Bcast_binomial(310).....: Failure during collective
```

原因分析

发生此问题的原因一般是防火墙未关闭。

不同系统防火墙查询命令略有不同，例如：

systemctl status firewalld

解决步骤

通过systemctl命令关闭防火墙，不同系统设置命令略有不同，命令示例如下：

- 关闭防火墙：
systemctl stop firewalld

- 禁用防火墙（开机不启动）：
`systemctl disable firewalld`
`sudo ufw status`

9.3.9 SSH 连接超时错误

问题现象

多机场景下，执行HCCL Test测试工具时，报如下错误：

```
ssh: connect to host xx.xx.xx.xx port 22: Connection time out
```

原因分析

此错误原因是测试主机无法远程登录到其他所有机器导致。

多机测试场景，需要配置操作节点到集群通信所有节点的SSH信任关系，以支持集群通信节点远程登录。

解决步骤

- 在当前操作节点生成密钥信息（如若环境中存在，可不重复执行）：
`ssh-keygen -t rsa`
例如密钥信息生成后，存储在/root/.ssh/id_rsa.pub文件中。
- 将操作节点公钥复制到集群通信其他节点，实现SSH密钥登录远程主机。
`ssh-copy-id -i /root/.ssh/id_rsa.pub node1_address`
`ssh-copy-id -i /root/.ssh/id_rsa.pub node2_address`
- 在操作节点上SSH远程登录其他节点，确认是否可以直接登录。

9.3.10 多台机器 MPI 版本不一致问题

问题现象

多机场景下，HCCL Test测试工具执行过程中发生概率性失败，coredump等随机问题。

原因分析

可能是多台机器的MPI软件版本不一致导致，可通过如下命令查询MPI软件版本：

```
which mpirun find / -name mpirun
```

解决步骤

当前版本，如果通信网卡使用IPv4协议，建议安装**MPI 3.2.1**版本；如果通信网卡使用IPv6协议，建议安装**Open MPI-4.1.5**版本。

且所有机器的MPI软件版本需要保持一致。

9.3.11 Device 网络不通报错 retcode 4

问题现象

多机场景下，HCCL Test工具执行时，报错“retcode: 4”，如下图所示：

```
-bash: hccn: command not found
[root@node-9 hccn_test]# hccn_tool -i0 -ip -s address **** netmask 255.255.255.0
[root@node-9 hccn_test]# mpirun -f hostfile -n 16 ./bin/all_reduce_test -b 8K -e 64M -f 2 -d fp32 -o sum -p 8

Authorized users only. All activities may be monitored and reported.
the minbytes is 8192,maxbytes is 67108864,items is 20,warmup_items is 5
hccn interface return error:err/opbase_test/hccn_allreduce_rootinfo_test.cc:135,retcode:4
hccn_opbaseexecute failed,Detailed logs are stored in path:/root/ascend/log/This is an error in
opbase_test_by_data_size.
hccn interface return error:err/opbase_test/hccn_allreduce_rootinfo_test.cc:135,retcode:4
hccn_op_base execute failed,Detailed logs are stored in path:/root/ascend/log/This is an error in
opbase_test_by_data_size.
hccn interface return error:err/opbase_test/hccn_allreduce_rootinfo_test.cc:135,retcode:0
hccn_op_base execute failed, Detailed logs are stored in path:/root/ascend/log/This is an error in
opbase_test_by_data_size.
```

原因分析

Device网络不通，导致建链失败。

解决步骤

在Host侧执行如下命令，分别ping每张卡，确认是否网络连通。

```
hccn_tool -i 0 -ping -g address 192.169.150.60
```

同平面卡需要两两互通，即所有机器的同号卡间要互相ping通（如：两两之间0卡ping0卡，1卡ping1卡，以此类推），同时如果为单机16卡，所有机器的0卡和8卡，1卡和9卡，2卡和11卡，以此类推都需要两两互通。

命令如下：

```
hccn_tool -i 0 -ping -g address 192.169.150.60 #当前机器的0卡ping另外一台机器的一张卡的device ip。
```

说明

若修改了IP地址，但未对应修改gateway，也会导致device之间不通，ip、netmask、gateway需要对应配置。

9.3.12 host ip 之间两两不通报 MPI 错误

问题现象

多机场景下，执行HCCL Test测试工具时，报错误：“Fatal error in PMPI_Barrier: Unknown error class, error stack”，如下图所示。

```
root@A01-R15-I235-228-0011955:/usr/local/Ascend/ascend-toolkit/latest/tools/hccn_test# mpirun -f hostfile -n 160 ./bin/all_reduce_test -b 8K -e 4G -f 2 -d fp32 -o sum
p 16
the minbytes is 8192, maxbytes is 4294967296, items is 20, warmup_items is 5
Fatal error in PMPI_Barrier: Unknown error class, error stack:
PMPI_Barrier(425)..... MPI_Barrier(MPI_COMM_WORLD) failed
MPIR_Barrier_impl(332)..... Failure during collective
MPIR_Barrier(292).....
MPIR_Barrier(292).....
MPIR_Barrier_intra(150).....
barrier_smp_intra(96).....
MPIR_Barrier_impl(332)..... Failure during collective
MPIR_Barrier_impl(327).....
MPIR_Barrier(292).....
MPIR_Barrier_intra(169).....
MPIR_Barrier_intra(169)..... Communication error with rank 64: Connection timed out
MPIR_Barrier_intra(169)..... Communication error with rank 64
MPIR_Barrier_intra(169)..... Communication error with rank 64
barrier_smp_intra(111).....
MPIR_Bcast_impl(1452).....
MPIR_Bcast(1476).....
MPIR_Bcast_intra(1287).....
MPIR_Bcast_binomial(310)..... Failure during collective
Fatal error in PMPI_Barrier: Unknown error class, error stack:
PMPI_Barrier(425)..... MPI_Barrier(MPI_COMM_WORLD) failed
MPIR_Barrier_impl(332)..... Failure during collective
MPIR_Barrier(292).....
MPIR_Barrier_intra(150).....
barrier_smp_intra(111).....
MPIR_Bcast_impl(1452).....
MPIR_Bcast(1476).....
MPIR_Bcast_intra(1287).....
MPIR_Bcast_binomial(310)..... Failure during collective
Fatal error in PMPI_Barrier: Unknown error class, error stack:
PMPI_Barrier(425)..... MPI_Barrier(MPI_COMM_WORLD) failed
MPIR_Barrier_impl(332)..... Failure during collective
MPIR_Barrier(292).....
MPIR_Barrier_intra(150).....
barrier_smp_intra(111).....
```

原因分析

HCCL Test测试场景，要求所有机器的Host网卡两两ping通，此问题一般是由于某台机器的Host IP与其他机器网络不通导致。

解决步骤

步骤1 定位到网络不通的Host机器。

可通过二分的方法，逐步添加机器进行测试，当添加到某台机器报错时，可登录此机器，然后ping其他机器的Host IP，若不通，即确认是此机器网络原因。

步骤2 解决此机器与其他机器的Host IP无法ping通的问题。

----结束

9.3.13 网口 down 导致报错 ret(-67)

问题现象

HCCL Test工具执行报错，在Host日志目录“~/ascend/log”下执行命令**grep -rn "ERROR"**，有关键报错：ra hdc message process failed ret(-67)。

原因分析

可能是某网口状态为down导致其他正常卡与其通信时，建链超时。

解决步骤

在Host侧执行如下命令，查询网口状态：

```
for i in {0..7}; do hccn_tool -i $i -link -g ; done
```

若某网卡link状态为down，初始化网卡时会失败，如下图所示，从而导致其他正常卡与其通信时，上报建链超时的错误。

```
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:44:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.469 [ra_hdc.c:1270]tid:214,ra_hdc_rdev_in
it(1270) : [init][ra_hdc_rdev]ra hdc message process failed ret(-67) phy_id(1)
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:45:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.485 [ra_host.c:621]tid:214,ra_rdev_init(6
21) : [init][ra_rdev]ra rdev init failed, ret(-67)
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:46:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.558 [adapter_hccp.cc:378][hcccl-214-0-1692
255918-hcccl_world_group][i][init][RaRdma]errNo[0x00000000000000013] rdma init fail. params: mode[1]. return: ret[328004]
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:47:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.565 [network_manager.cc:374][hcccl-214-0-1
692255918-hcccl_world_group][i][init][DeviceRDMA]errNo[0x000000000000000b] ra rdma init failed, devid[1] ip[0x6a65a8c0]. return[19]
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:48:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.569 [network_manager.cc:291][hcccl-214-0-1
692255918-hcccl_world_group][i][Start][Nic]errNo[0x00000000000000013] ra nic init rdma failed, devid[1], return[11]
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:49:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.573 [hcccl_impl_base.cc:970][hcccl-214-0-16
92255918-hcccl_world_group][i][init][Nic]strrt nic ipaddr[0x6a65a8c0] failed
/root/ascend/log/debug/plog/plog-214.20230817070517473.log:50:[ERROR] HCCL(214,alltoally_test):2023-08-17-07:05:18.878.577 [hcccl_impl.cc:1918][hcccl-214-0-169225
5918-hcccl_world_group][i][call trace: hccclRet -> 19
```

定位到问题网口后，即可联系硬件工程师排查网口问题；若需紧急恢复可尝试重启或重新插拔光模块。

9.3.14 Device IP 冲突导致 get socket 超时

问题现象

网络正常连通且TLS状态一致的场景下，日志存在get socket超时的错误，使用hccn_tool进行卡之间ping操作，正向ping与反向ping存在丢包情况。

原因分析

可能是由于两台Device IP地址冲突导致。

解决步骤

重新配置Device IP，避免IP地址冲突。

9.3.15 未指定网卡名报错 retcode 9

问题现象

多机测试场景下，执行HCCL Test测试工具时，进程执行卡住，然后报错retcode 9，如下图所示：

```
# mpirun -f hostfile -n 32 ./bin/all_reduce_test -b 1 -e 2G -p 16 -f 2 -o sum -c 0
the minbytes is 1, maxbytes is 2147483648, iters is 20, warmup_iters is 5
hccl interface return errreturn err ./common/src/hccl_test_common.cc:499, retcode:9
hccl interface return errreturn err ./common/src/hccl_test_common.cc:499, retcode:9
hccl interface return errreturn err ./common/src/hccl_test_common.cc:499, retcode:9
hccl interface return errreturn err ./common/src/hccl_test_common.cc:499, retcode:9
...
This is an error in init_hcclComm.
```

原因分析

未配置初始化root通信网卡名导致。

解决步骤

多机场景下，执行HCCL Test测试工具前需要在操作节点配置HCCL初始化root通信网卡的网卡，HCCL可通过该网卡获取Host IP，完成通信域创建。

配置方式如下：

```
# 配置HCCL的初始化root通信网卡使用的IP协议版本，AF_INET: IPv4；AF_INET6: IPv6
export HCCL_SOCKET_FAMILY=AF_INET

# 支持以下格式的网卡名配置（4种规格自行选择1种即可，环境变量中可配置多个网卡，多个网卡间使用英文逗号分隔，取最先匹配到的网卡作为root网卡）

# 精确匹配网卡
export HCCL_SOCKET_IFNAME==eth0,enp0 # 使用指定的eth0或enp0网卡
export HCCL_SOCKET_IFNAME!=eth0,enp0 # 不使用eth0与enp0网卡

# 模糊匹配网卡
export HCCL_SOCKET_IFNAME=eth,enp # 使用所有以eth或enp为前缀的网卡
export HCCL_SOCKET_IFNAME^=eth,enp # 不使用任何以eth或enp为前缀的网卡
```

需要注意：

MPI工具执行时，会将环境变量同步到所有节点，如果参与集合通信的不同节点的网卡名字不同，例如node1的网卡名为eth1，node2的网卡名为eth2，则需要使用模糊匹配网卡的方式进行环境变量配置。

9.3.16 hostfile 与测试命令不匹配报错 retcode 11

问题现象

多机场景下执行HCCL Test测试工具时，报错：This is an error in init_hcclComm.retcode: 11，如下所示：

```
[root@node-96-61 hccl_test]# vim hostfile
[root@node-96-61 hccl_test]# mpirun -f hostfile -n 16 ./bin/all_reduce_test -b 8K -e 64M -f 2 -d fp32 -o sum -p 8

Authorized users only. All activities may be monitored and reported.
the minbytes is 8192, maxbytes is 67108864, iters is 20, warmup iters is 5
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
This is an error in init_hcclComm.
This is an error in init_hcclComm.
This is an error in init_hcclComm.
hccl interface return errreturn err ./common/src/hccl_test_common.cc:503, retcode: 11
This is an error in init_hcclComm.
```

原因分析

- hostfile文件中配置的每个节点使用的卡数和测试命令-p不匹配，“-p”代表每个节点使用多少张卡（即每个节点的进程数）。
- hostfile文件中配置的节点数量和mpirun -n数量不匹配，“-n”代表总共使用多少张卡（即节点数量*每个节点的卡数）。

mpirun命令示例：

```
mpirun -f hostfile -n 16 ./bin/all_reduce_test -b 8K -e 4G -f 2 -d fp32 -o sum -p 8
```

解决步骤

修改hostfile配置文件或者mpirun测试命令，使二者配置的总卡数以及每个节点使用的卡数保持一致。

9.3.17 大规模集群场景报建链失败错误

问题现象

大规模集群场景执行HCCL Test测试工具，报建链失败的错误，但是拆分成小规模集群分别测试正常。

原因分析

- Master节点建链时处理的数据量较大，建链默认等待超时时间120s若无法满足需求，会建链失败。
- Master节点允许处理的并发建链数受linux内核参数“somaxconn”与“tcp_max_syn_backlog”的限制，所以，针对大规模集群组网，若“somaxconn”与“tcp_max_syn_backlog”取值较小会导致部分客户端概率性提前异常退出，导致集群初始化失败。

解决步骤

步骤1 调大建链超时等待时间。

通过环境变量HCCL_CONNECT_TIMEOUT（单位为s）进行设置，建议根据集群组网的规模适当调大。例如：

```
# 3K卡场景
export HCCL_CONNECT_TIMEOUT=240

# 5K卡场景
export HCCL_CONNECT_TIMEOUT=600
```

步骤2 根据集群数量调整“somaxconn”与“tcp_max_syn_backlog”参数的值。

需要注意，所有机器的OS都需要配置，包括裸机、镜像环境，配置示例如下：


```
sysctl -w net.core.somaxconn=65535  
sysctl -w net.ipv4.tcp_max_syn_backlog=65535
```

---结束

9.3.18 大规模集群场景 HCCP 报 errno 24

问题现象

大规模集群场景执行HCCL Test测试工具，HCCP报“errno:24”的错误，错误示例如下所示，但是拆分成小规模集群分别测试正常。

```
[ERROR] HCCP ( 2726987,all_reduce_test) :2024-01-12-17:52:59.187.615  
[rs_socket.c:570]tid:2727604,rs_epoll_event_listen_in_handle(570) : IP:*** accept( ) failed!errno:24  
[ERROR] HCCP ( 2726987,all_reduce_test) :2024-01-12-17:52:59.187.615  
[rs_socket.c:570]tid:2727604,rs_epoll_event_listen_in_handle(570) : IP:*** accept( )  
failed!errno:24
```

执行“ulimit -a”命令查询“open files”的值（默认为1024），而测试卡数的规模接近open files的值。

```
[root@hccl137 ~]# ulimit -a  
real-time non-blocking time (microseconds, -R) unlimited  
core file size (blocks, -c) 0  
data seg size (kbytes, -d) unlimited  
scheduling priority (-e) 0  
file size (blocks, -f) unlimited  
pending signals (-i) 3092387  
max locked memory (kbytes, -l) 65536  
max memory size (kbytes, -m) unlimited  
open files (-n) 1024  
pipe size (512 bytes, -p) 8  
POSIX message queues (bytes, -q) 819200  
real-time priority (-r) 0  
stack size (kbytes, -s) 8192  
cpu time (seconds, -t) unlimited  
max user processes (-u) 3092387  
virtual memory (kbytes, -v) unlimited  
file locks (-x) unlimited  
[root@hccl137 ~]#
```

原因分析

open files太小，测试卡数规模接近open files，导致测试报错。

解决步骤

修改所有机器（包括裸机、镜像环境）的open files大小，建议配置为1000000，配置方法如下：

```
echo "ulimit -n 1000000">> /etc/profile  
source /etc/profile
```

执行“cat/proc/<PID>/limits”查看某个进程的soft limit和hard limit限制，有可能soft limit少但ulimit -a查出来是很大，该情况open files大小需要同步修改，进程PID可通过npu-smi工具查看。


```
(base) root@box-24:~# cat /proc/27512/limits
Limit                Soft Limit           Hard Limit           Units
Max cpu time         unlimited            unlimited            seconds
Max file size        unlimited            unlimited            bytes
Max data size        unlimited            unlimited            bytes
Max stack size       8388608             unlimited            bytes
Max core file size   0                   unlimited            bytes
Max resident set     unlimited            unlimited            bytes
Max processes        2062502             2062502             processes
Max open files       1024                1048576             files
Max locked memory    67598909440         67598909440         bytes
Max address space    unlimited            unlimited            bytes
Max file locks       unlimited            unlimited            locks
Max pending signals  2062502             2062502             signals
Max msgqueue size    819200              819200              bytes
Max nice priority    0                   0
Max realtime priority 0                   0
Max realtime timeout unlimited            unlimited            us
```